L1 Weeks 2–3: Paved road + Phoenix on halo

Service template · GitHub Actions + GHCR · Sealed Secrets · Phoenix Command Center as first tenant · CKA drills

W3 checkpoint: **the Command Center deploys through the template**

Builds on the Week 1 guide (Rev 2): k3s v1.36.5+k3s1 · Argo CD 10.11.6 (v3.5.6) · Helm v4.3.1

New: FastAPI 0.162.0 · Python 3.14.10 / 3.12.16 · uv 0.13.6 · Copier 9.19.1 · Sealed Secrets 0.41.2 · TimescaleDB 2.31.0-pg18 · Redis 8.10.3

Versions checked 2026-11-02 · Plan: Nov 12–15 (Part A), Nov 16–22 (Part B)

How to copy commands: every command box is numbered and colour-coded by where it runs. Select the whole box (from its first to last line) and paste it into the matching shell. File boxes write the file with `cat > 'path' <<'__EOF__'` from the repo root (`~/src/ai-platform-portfolio`).

WSL Ubuntu shell (bash) **Windows PowerShell** **Output / reference - not a command**

Text tags: [WSL] = WSL shell · [WIN] = PowerShell · [VS Code] · [BROWSER] · [GITHUB] = github.com · ☐ = checkbox

Contents

Overview: versions, decisions, architecture, plan

- Version table (checked 2026-11-02)
- The decisions (and why), in one table
- What I found in the Phoenix code (2.1.0)
- Architecture
- Time plan (90-minute blocks; each ends in a working state and a 15-minute drill)
- Conventions

Part A: The paved road service template (Nov 12–15)

- A0: Prerequisites and the cka scratch namespace (Nov 7–8 or start of A1)
- A1: Platform add-ons: Argo CD health check, Sealed Secrets, Retain storage (Thu Nov 12)
- A2: FastAPI template: code, tests, Makefile (Fri Nov 13)
- A3: Dockerfile and the shared Helm chart (Sat Nov 14, block 1)
- A4: Template deploy files, generator scripts, GHCR pull secret (Sat Nov 14, block 2)
- A5: GitHub Actions for the monorepo (Sun Nov 15, block 1)
- A6: Prove it: hello-platform end to end in under 15 minutes, then delete it (Sun Nov 15, block 2)

Part B: Phoenix Command Center as the first tenant (Nov 16–22)

- B1: Import the code (with history) and fit it to the template (Mon Nov 16, block 1)
- B2: Secrets and the data layer: TimescaleDB + Redis (Mon Nov 16, block 2)
- B3: Namespace guardrails and NetworkPolicies (Tue Nov 17, block 1)
- B4: The Command Center through the template (Tue Nov 17, block 2) — W3 checkpoint
- B5: Backups, restore test, runbook (Wed Nov 18, block 1)
- B6: Resilience tests and retiring the old repo (Wed Nov 18, block 2)

Part C: CKA drills, woven in

- C1: The rules
- C2: Where lab steps teach exam topics
- C3: Drill log (fill in; it's on the final checklist)
- C4: Print me: "10 minutes of kubectl a day" (7 days)

Appendix: troubleshooting, glossary, drill answers, checklist, what's next

- Troubleshooting
- Glossary
- Drill answers
- Final checklist
- What's next (out of scope here)

Overview: versions, decisions, architecture, plan

Weeks 2–3 of L1. Part A: build the paved road (template, chart, CI, secrets). **Part B:** move the Phoenix Command Center onto it as the first real tenant. **Part C:** CKA drills woven through both.

W3 checkpoint: *the Command Center deploys through the template.*

Starting point: Week 1 (Rev 2) is complete. You have cluster `halo`, repo `~/src/ai-platform-portfolio`, Argo CD app of apps (`root-halo`), AppProjects `platform` and `lab`, podinfo, and CI on GitHub Actions only.

Version table (checked 2026-11-02)

Everything from Week 1 is unchanged: WSL 3.0.5, k3s v1.36.5+k3s1, kubectl v1.36.6, Helm v4.3.1, Argo CD chart 10.11.6 / v3.5.6, argocd v3.5.6. New this week:

Component	Pinned	Released	Source
Python base image (template)	`python:3.14.10-slim-trixie@sha256:930e1abb…`	2026-11-02	https://hub.docker.com/_/python
Python base image (Phoenix)	`python:3.12.16-slim-trixie@sha256:b18b261e…`	2026-11-02	same
uv (+ `ghcr.io/astral-sh/uv:0.13.6`)	0.13.6	2026-10-27	https://github.com/astral-sh/uv/releases
FastAPI	0.162.0	2026-10-23	https://pypi.org/project/fastapi/
uvicorn	0.54.0	2026-09-25	https://pypi.org/project/uvicorn/
pydantic / pydantic-settings	2.14.2 / 2.16.2	2026-10-29 / 10-28	https://pypi.org/project/pydantic/
prometheus-client	0.26.0	2026-07-24	https://pypi.org/project/prometheus-client/
pytest / httpx2 / ruff / mypy	9.2.0 / 2.16.0 / 0.17.3 / 2.4.1	Oct 2026	PyPI
Copier	9.19.1	2026-10-13	https://github.com/copier-org/copier/releases
yq	v4.54.1	—	https://github.com/mikefarah/yq/releases
Sealed Secrets controller + kubeseal	0.41.2 (chart `sealed-secrets` 2.22.0)	2026-10-26	https://github.com/bitnami-labs/sealed-secrets/releases
TimescaleDB (PostgreSQL 18)	`timescale/timescaledb:2.31.0-pg18@sha256:0fcff576…`	2026-10-20	https://hub.docker.com/r/timescale/timescaledb
Redis	`redis:8.10.3-alpine@sha256:21090af9…`	2026-10-29	https://hub.docker.com/_/redis
gitleaks	8.30.1	—	https://github.com/gitleaks/gitleaks/releases
actions/checkout	v7.0.1 `3d3c42e5…`	—	https://github.com/actions/checkout/releases

Component	Pinned	Released	Source
astral-sh/setup-uv	v10.3.3 `68559e99…`	—	https://github.com/astral-sh/setup-uv/releases
docker/setup-buildx-action · login-action · metadata-action · build-push-action	v4.4.1 · v4.7.0 · v6.2.0 · v7.4.0	—	https://github.com/docker
azure/setup-helm	v5.0.1 `9bc31f4e…`	—	https://github.com/Azure/setup-helm/releases
Runners	`ubuntu-24.04`, `windows-2025` (GitHub-hosted)	—	GitHub docs
Considered, not used	CloudNativePG 1.30.1 · Argo CD Image Updater 1.4.0 · Valkey 9.1.3	—	see decisions

Every action is pinned to a full commit SHA, with the version as a comment. Dependabot proposes the bumps.

Recent changes worth knowing

- **Bitnami (Aug–Sep 2025):** versioned images moved to the unmaintained `bitnamilegacy` repository, and the free catalog shrank to a small set of `latest` hardened images. Bitnami charts are no longer a pinned, maintained option, so this guide uses official images. Sealed Secrets is a separate bitnami-labs project, still published with versioned tags.
- **PostgreSQL 18 images** changed the default `PGDATA` location. The manifests set `PGDATA` explicitly.
- **Starlette's `TestClient`** now warns that `httpx` is deprecated as its transport. The template uses `httpx2`.
- **Argo CD Image Updater 1.x** is configured with an `ImageUpdater` CRD (no longer annotations).
- **Supply chain:** after the 2025 compromise of a popular third-party "changed-files" action, this guide uses native `paths:` filters and SHA-pinned actions only.
- **Redis 8** added AGPLv3 as a license option. Fine for personal use; Valkey is the drop-in alternative.
- **GHCR auth:** at the time of writing, GitHub's docs say GHCR needs a **classic** personal access token (scope `read:packages`). My sandbox couldn't re-check docs.github.com, so if fine-grained tokens now work, use one with *Packages: read*.

The decisions (and why), in one table

Question	Decision	Why
Template generator	**Copier**	Records each service's answers, so `copier update` can roll template fixes out later (paved roads drift without this). Supports conditional files and custom delimiters (Helm and Actions already use `{{ }}`). cookiecutter only creates; a copy script drifts on day 2
Helm + Kustomize under Argo CD	**Argo CD Helm source** (`path: platform/helm-charts/service`) with **layered value files** `_shared/halo → apps/<svc>/base → apps/<svc>/overlays/halo`	Same `base` / `overlays/<env>` folder contract as Week 1; no `--enable-helm` flag; CI promotes by editing one YAML key. Kustomize stays for plain manifests (podinfo, phoenix-data)
CI layout	One **reusable workflow** (`_service-ci.yml`), four **composite actions**, one thin **caller per service** with native path filters	Each service shows as its own check, and its trigger is readable in one file. The contract with a service is `make ci` plus a Dockerfile

Question	Decision	Why
Image tags	Full git SHA, never `latest`	Immutable, traceable to a commit, rollback = `git revert`
Promotion	**CI commits the tag** into `overlays/halo/values.yaml`	Visible in Git history and PRs, no registry or Git write credentials in the cluster, no polling. Image Updater revisited for multi-env later
Pull from private GHCR	Classic PAT `read:packages`, sealed **once, cluster-wide**, rendered by the chart into every service namespace	Nothing secret in Git; a new service needs zero secret work (keeps the 15-minute target)
Secrets	**Sealed Secrets**	Native to Argo CD (a CRD, no plugin), public-key sealing (no decryption key on laptops or CI). SOPS + age needs a repo-server plugin with Argo CD; better kept for Terraform later
Phoenix data	Official **TimescaleDB** and **Redis** images as StatefulSets on a **Retain** StorageClass	Maintained and free; teaches StatefulSets; keeps the CNPG secret contract (`phoenix-db-app/uri`) so CNPG later is drop-in
Phoenix wiring	**Two Applications:** `phoenix-data` (wave 20) then `command-center` (wave 30)	Different lifecycles: the data layer must never be pruned or cascade-deleted with the app. Waves plus an Application health check make the app wait for the databases

What I found in the Phoenix code (2.1.0)

I read the code you sent, pushed it to `nikhilgs817/PHOENIX_TRADER` (`main`), and made the minimal fit on branch `platform-fit` (nikhilgs817/PHOENIX_TRADER#3, draft; 381 tests pass). **Merge it before Part B.**

Fact	Impact on this guide
FastAPI + Jinja/HTMX pages, **no separate frontend**; one image run four ways: `phoenix serve` (:8000, metrics :9100), `worker`, `agent`, `hook` (:8080)	Chart main process = `serve`, plus `worker` as a chart worker. **`agent` (MCP + models) and `hook` (internet-facing webhook) stay off this week** (L2 and the tunnel come later)
SQLAlchemy + Alembic (`phoenix db upgrade`), PostgreSQL via `PHOENIX_DATABASE_URL` (CNPG-style `postgresql://` uri)	TimescaleDB image (plain PostgreSQL to the app); migration Job before every rollout
No Redis client in 2.1	Redis is provisioned as planned (for the L2 agents). Delete `redis.yaml` from the overlay if you'd rather not run an idle service; nothing depends on it
Models through LiteLLM/Ollama (`PHOENIX_LLM_BASE_URL`)	Optional step B4.6 points the chat at Ollama on Windows. Full LLM work is L2
Alpaca **paper** keys, Schwab read-only keys, Google OAuth, Fidelity CSV imports	**None of them in the cluster this week.** Phoenix runs on synthetic demo data with the demo clock (`phoenix demo`), per the "public or synthetic data only" rule
Already non-root (uid 10001), read-only root FS works (`HOME=/tmp`), env-only config (`pydantic-settings`), host allowlist	Fits the chart's securityContext as is; set `PHOENIX_ALLOWED_HOSTS=phoenix.halo.localhost`
Code changes needed (in PR #3)	`/livez` + `/readyz` (and in `PUBLIC_PATHS`), `PHOENIX_LOG_FORMAT=json`, `demo --url postgresql://` fix, `Makefile` (`make ci`), Dockerfile base pinned by digest

Architecture

Output / reference - not a command

```
Developer (VS Code in WSL "halo")                          GitHub: nikhilgs817/ai-platform-portfolio (private)
  git push ------------------------------------------------> main
                                                             | path filter: projects/<svc>/**
                                                             v
                                  GitHub Actions, GitHub-hosted ubuntu-24.04
                                  svc-<svc>.yml  -->  _service-ci.yml (reusable)
                                    verify : make ci   (ruff, mypy, pytest)
                                    image  : buildx + GHA layer cache --> ghcr.io/nikhilgs817/<svc>:<git-sha>  (GHCR, private)
                                    promote: commit image.tag=<sha> to platform/kubernetes/apps/<svc>/overlays/halo/values.yaml
                                             [skip ci], GITHUB_TOKEN (cannot trigger workflows)
                                                             |
              +----------------------------------------------+   Argo CD polls Git (~3 min), read-only deploy key
              v
Windows 11 --> WSL2 VM --> Ubuntu "halo" --> k3s v1.36.5
  Argo CD  root-halo --> projects(-10) sealed-secrets(-5) storage-classes(-5) podinfo(10) <new services>(10)
                         phoenix-data(20) --> waits Healthy --> command-center(30)
  namespace phoenix  (PSA restricted, ResourceQuota, LimitRange, default-deny NetworkPolicy)
    command-center: serve :8000 (+ metrics :9100), worker, migrate Job --- 5432 --> timescale-0 (PVC 20Gi local-path-retain)
                                                                        --- 6379 --> redis-0     (PVC 2Gi,  local-path-retain)
    CronJob timescale-backup --> PVC timescale-backups (also mounted in timescale-0 for restores)
    Secrets: SealedSecret (ciphertext in Git) --> sealed-secrets-controller (kube-system) --> Secret
    image pulls from GHCR with Secret ghcr-pull
  Traefik (hostNetwork :80)  <--  WSL localhost relay  <--  Windows browser: http://phoenix.halo.localhost
```

Time plan (90-minute blocks; each ends in a working state and a 15-minute drill)

Date	Block	Content	Ends with
Sat Nov 7 / Sun Nov 8 (*only if Week 1 is done*)	A0	Tools (uv, yq, kubeseal), `cka` namespace, this guide's prerequisites	Tools pinned, `cka` context works
Mon Nov 9 – Wed Nov 11	—	**NCA-AIIO prep only. No lab.**	Exam Wed Nov 11
Thu Nov 12	A1	Platform add-ons: Argo CD app health check, Sealed Secrets, `local-path-retain`, key backup	Controller running, key in the password manager · Drill W2-1

Date	Block	Content	Ends with
Fri Nov 13	A2	FastAPI template (code, tests, Makefile)	`make ci` green on a generated test copy · Drill W2-2
Sat Nov 14	A3	Dockerfile + the shared Helm chart	Chart renders and passes a server-side dry run · Drill W2-3
Sat Nov 14	A4	Template deploy files, generator scripts, GHCR pull secret	`new-service.sh` works locally · Drill W2-4
Sun Nov 15	A5	GitHub Actions: reusable workflow, composite actions, guards	`repo-guards` and `platform-checks` green · Drill W2-5
Sun Nov 15	A6	**hello-platform end to end in < 15 min**, then delete it	Service built, promoted, synced, removed · Drill W2-6
Mon Nov 16 (PTO)	B1	Import Phoenix with history, CI caller, first image	`ghcr.io/…/command-center:<sha>` exists · Drill W3-1
Mon Nov 16	B2	Secrets + TimescaleDB + Redis (`phoenix-data`)	Both StatefulSets Ready · Drill W3-2
Tue Nov 17	B3	Guardrails + NetworkPolicies + proof they're enforced	Probe table matches · Drill W3-3
Tue Nov 17	B4	Command Center through the template + demo data	**W3 checkpoint** · Drill W3-4
Wed Nov 18	B5	Backups, restore test, runbook	Restore test all `OK` · Drill W3-5
Wed Nov 18	B6	Resilience tests a–e, retire the old repo	Everything Synced/Healthy · Drill W3-6
Thu Nov 19 – Sun Nov 22	—	L1 part 2 (Harbor, Kyverno) and the kubeadm build: separate guides. **Buffer for any Part A/B spill**	

Flags

- **Part A is tight:** 6 blocks in 4 days with a newborn. If A0 didn't happen on Nov 7–8, A0 folds into A1 (adds about 20 min). If anything slips, **A6 moves to Mon Nov 16** and Part B shifts by one block. That still fits the PTO week only if Harbor/Kyverno take ≤ 4 blocks and kubeadm ≤ 3.
- **A5 is the riskiest block** (first CI run, GHCR permissions). The troubleshooting appendix has the usual failures.

Conventions

- **Where to run:** [WSL] = Ubuntu shell in `~/src/ai-platform-portfolio` (unless stated) · [WIN] = PowerShell · [VS Code] · [BROWSER] · [GITHUB] = github.com UI.
- **Files:** every file is created with `cat > 'path' <<'__EOF__' … __EOF__` from the repo root. Copy the **whole** box. The same files are also in `labs/week2/repo-files/` in the `certifiaction` repo if you'd rather `cp` them.
- **Commit rhythm:** one commit per step, `git pull --rebase` before every push (CI pushes promotion commits to `main`).
- **Imperative exceptions** (everything else is Argo CD): the `cka` scratch namespace, one-off ops commands (`kubectl exec`, `create job --from`), and sealing secrets locally.

Part A: The paved road service template (Nov 12–15)

Outcome: one command creates a service that CI tests, builds and pushes, Git promotes, and Argo CD deploys. You prove it with `hello-platform`, then remove it cleanly.

A0: Prerequisites and the `cka` scratch namespace (Nov 7–8 or start of A1)

Goal: pinned tools for this guide, plus a practice namespace that Argo CD never touches.

Why: uv runs Python projects and Copier, yq edits YAML in scripts and CI, and kubeseal encrypts secrets for the cluster. The `cka` namespace matters because Argo CD's selfHeal reverts *anything it manages* within seconds. Drill objects in a managed namespace would be "fixed" (deleted or reset) while you practise. A namespace that is in no Application, and in no AppProject destination, is invisible to Argo CD.

Where: [WSL]

#001 WSL Ubuntu shell (bash) - copy & paste

```
sudo apt-get update -qq && sudo apt-get install -y -qq make git-filter-repo openssl
T=$(mktemp -d); cd "$T"
# uv 0.13.6 (also gives uvx, used to run Copier without installing it)
curl -fsSLO https://github.com/astral-sh/uv/releases/download/0.13.6/uv-x86_64-unknown-linux-gnu.tar.gz
curl -fsSLO https://github.com/astral-sh/uv/releases/download/0.13.6/uv-x86_64-unknown-linux-gnu.tar.gz.sha256
sha256sum --check uv-x86_64-unknown-linux-gnu.tar.gz.sha256
tar xzf uv-x86_64-unknown-linux-gnu.tar.gz && sudo install -m 0755 uv-x86_64-unknown-linux-gnu/uv uv-x86_64-unknown-linux-gnu/uvx /usr/local/bin/
# yq v4.54.1
curl -fsSLO https://github.com/mikefarah/yq/releases/download/v4.54.1/yq_linux_amd64
curl -fsSLO https://github.com/mikefarah/yq/releases/download/v4.54.1/checksums-bsd
echo "$(grep '^SHA256 (yq_linux_amd64)' checksums-bsd | awk '{print $4}')  yq_linux_amd64" | sha256sum --check
sudo install -m 0755 yq_linux_amd64 /usr/local/bin/yq
# kubeseal 0.41.2 (same version as the controller)
curl -fsSLO https://github.com/bitnami-labs/sealed-secrets/releases/download/v0.41.2/kubeseal-0.41.2-linux-amd64.tar.gz
curl -fsSLO https://github.com/bitnami-labs/sealed-secrets/releases/download/v0.41.2/sealed-secrets_0.41.2_checksums.txt
grep ' kubeseal-0.41.2-linux-amd64.tar.gz$' sealed-secrets_0.41.2_checksums.txt | sha256sum --check
tar xzf kubeseal-0.41.2-linux-amd64.tar.gz kubeseal && sudo install -m 0755 kubeseal /usr/local/bin/kubeseal
cd ~ && rm -rf "$T"
uv --version; yq --version; kubeseal --version; uvx copier@9.19.1 --version
```

Expected: `uv 0.13.6`, `yq … v4.54.1`, `kubeseal version: 0.41.2`, `copier 9.19.1`.

The `cka` namespace and its own context (created imperatively on purpose):

#002 WSL Ubuntu shell (bash) - copy & paste

```
kubectl create namespace cka
kubectl label namespace cka lab.platform/purpose=cka-practice
kubectl config set-context cka --cluster=halo --user=halo --namespace=cka
cat >> ~/.bashrc <<'EOF'
alias kcka='kubectl --context cka'          # drills: kcka get pods ...
EOF
source ~/.bashrc
kcka get all
```

Verify: `argocd app list -o name` doesn't mention `cka`, and `yq '.spec.destinations[].namespace' platform/kubernetes/argocd/projects/lab.yaml` doesn't list it.

If it fails

1. `sha256sum: WARNING: 1 computed checksum did NOT match` → incomplete download. Delete the file and re-run that block.
2. `uvx` can't download Copier → the WSL DNS/proxy issue from Week 1 T4. Check `curl -I https://pypi.org`.

- ☐ Tools pinned · ☐ `cka` namespace and context exist (and Argo CD doesn't know about them)

A1: Platform add-ons: Argo CD health check, Sealed Secrets, Retain storage (Thu Nov 12)

A1.1 Let the app of apps wait for child health

Goal: sync waves between child Applications (for example `phoenix-data` before `command-center`) actually *wait* for health.

Why: since Argo CD 1.8, an Application's own health no longer counts in its parent. Without this Lua check, the root applies wave 30 the moment wave 20 is *applied*, not when it's *healthy*. The official docs give this snippet to restore the behaviour. It goes into the Week 1 values file, so it's versioned like everything else.

Where: [WSL]

#003 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/argocd/install'
cat > 'platform/kubernetes/argocd/install/values.yaml' <<'__EOF__'
# platform/kubernetes/argocd/install/values.yaml
# Common Argo CD values for every cluster. Chart argo/argo-cd 10.11.6 (Argo CD v3.5.6).
# NO secrets here: admin password is set via CLI and stored in the password manager.
global:
  domain: argocd.example.invalid      # overridden per cluster (values-<env>.yaml)
```

```
  revisionHistoryLimit: 3

configs:
  params:
    server.insecure: true             # TLS terminates at the ingress; no TLS in week 1 (loopback only)
  cm:
    admin.enabled: true               # local admin until SSO/OIDC (production: false)
    exec.enabled: false               # no web terminal into pods
    statusbadge.enabled: false
    timeout.reconciliation: 180s      # Git poll interval (+ jitter); prod uses webhooks
    # Week 2: let a parent app-of-apps wait for each child Application to be Healthy before the next sync wave
    # (Argo CD removed this health check by default in 1.8; the docs give this Lua to restore it).
    resource.customizations.health.argoproj.io_Application: |
      hs = {}
      hs.status = "Progressing"
      hs.message = ""
      if obj.status ~= nil and obj.status.health ~= nil then
        hs.status = obj.status.health.status
        if obj.status.health.message ~= nil then
          hs.message = obj.status.health.message
        end
      end
      return hs
  rbac:
    policy.default: role:readonly     # any future SSO user is read-only unless granted more
  secret:
    createSecret: true                # chart creates argocd-secret; password is NOT set here

dex:
  enabled: false                      # no SSO yet

notifications:
  enabled: false                      # later: alert on sync failures / degraded health

controller:
  replicas: 1
  resources:
    requests: { cpu: 250m, memory: 512Mi }
```

```
    limits:   { memory: 2Gi }

repoServer:
  replicas: 1
  resources:
    requests: { cpu: 100m, memory: 256Mi }
    limits:   { memory: 1Gi }

server:
  replicas: 1
  resources:
    requests: { cpu: 50m, memory: 128Mi }
    limits:   { memory: 512Mi }
  ingress:
    enabled: true
    ingressClassName: traefik         # hostname defaults to global.domain
    tls: false

applicationSet:
  replicas: 1                         # kept for multi-cluster generators later
  resources:
    requests: { cpu: 25m, memory: 64Mi }
    limits:   { memory: 256Mi }

redis:
  resources:
    requests: { cpu: 50m, memory: 64Mi }
    limits:   { memory: 256Mi }
__EOF__
```

#004 WSL Ubuntu shell (bash) - copy & paste

```
helm upgrade argocd argo/argo-cd --version 10.11.6 -n argocd \
  -f platform/kubernetes/argocd/install/values.yaml -f platform/kubernetes/argocd/install/values-halo.yaml --wait
kubectl -n argocd get cm argocd-cm -o yaml | grep -c 'argoproj.io_Application'
```

Expected: `Release "argocd" has been upgraded`, then `1`. (If you did Week 1 Step 10, Argo CD manages itself: skip `helm upgrade`, the commit below applies it.)

A1.2 Sealed Secrets controller, Retain StorageClass, project updates

Goal: Argo CD installs the Sealed Secrets controller (chart 2.22.0) and a `local-path-retain` StorageClass.

Why: a **SealedSecret** is a Secret encrypted with the controller's public key. Only the controller (holding the private key) can turn it back into a Secret, inside the cluster. So the ciphertext is safe in a public repo, like an Ansible Vault file whose key never leaves the server. `local-path-retain` uses the same disk provisioner as Week 1, but deleting a PVC *keeps* the data (`Retain`), which is what databases need.

Where: [WSL]

#005 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/argocd/projects'
cat > 'platform/kubernetes/argocd/projects/platform.yaml' <<'__EOF__'
# platform/kubernetes/argocd/projects/platform.yaml
# Platform-owned components: root apps, Argo CD itself, cluster add-ons. Full cluster scope.
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: platform
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "-10"
    argocd.argoproj.io/sync-options: Prune=false   # never auto-delete a project
spec:
  description: Platform components owned by the platform team
  sourceRepos:
    - git@github.com:nikhilgs817/ai-platform-portfolio.git
    - https://argoproj.github.io/argo-helm          # Step 10: Argo CD manages its own chart
    - https://bitnami-labs.github.io/sealed-secrets # Week 2: Sealed Secrets controller chart
  destinations:
    - server: https://kubernetes.default.svc
      namespace: '*'
  clusterResourceWhitelist:
    - group: '*'
      kind: '*'
__EOF__
```

#006 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/argocd/projects'
cat > 'platform/kubernetes/argocd/projects/lab.yaml' <<'__EOF__'
```

```
# platform/kubernetes/argocd/projects/lab.yaml
# Tenant workloads (samples, Phoenix, LLM services). Least privilege: listed namespaces only,
# no cluster-scoped resources except the Namespace that CreateNamespace=true needs.
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: lab
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "-10"
    argocd.argoproj.io/sync-options: Prune=false
spec:
  description: Tenant workloads on the halo/aws platform
  sourceRepos:
    - git@github.com:nikhilgs817/ai-platform-portfolio.git
  destinations:                                  # one entry per tenant namespace (new-service.sh adds them)
    - server: https://kubernetes.default.svc
      namespace: podinfo
    - server: https://kubernetes.default.svc
      namespace: phoenix
  clusterResourceWhitelist:
    - group: ''
      kind: Namespace
  orphanedResources:
    warn: true                                   # flag anything in tenant namespaces not in Git
__EOF__
```

#007 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/apps/sealed-secrets/overlays/halo'
cat > 'platform/kubernetes/apps/sealed-secrets/overlays/halo/values.yaml' <<'__EOF__'
# platform/kubernetes/apps/sealed-secrets/overlays/halo/values.yaml — Sealed Secrets controller on halo.
fullnameOverride: sealed-secrets-controller   # the name kubeseal looks for by default (namespace kube-system)
image:
  registry: docker.io
  repository: bitnami/sealed-secrets-controller
  tag: "0.41.2"                               # pinned; bitnami-labs project, not part of the Bitnami catalog change
keyrenewperiod: "0"                           # lab: one sealing key to back up; rotate by hand (runbook)
resources:
  requests: {cpu: 20m, memory: 64Mi}
  limits: {memory: 256Mi}
__EOF__
```

#008 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/apps/storage-classes/base'
cat > 'platform/kubernetes/apps/storage-classes/base/local-path-retain.yaml' <<'__EOF__'
# platform/kubernetes/apps/storage-classes/base/local-path-retain.yaml
# Same provisioner as k3s's default "local-path", but deleting a PVC keeps the data (PV goes to Released).
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: local-path-retain
  annotations:
    argocd.argoproj.io/sync-options: Prune=false,Delete=false
provisioner: rancher.io/local-path
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: false
__EOF__
```

#009 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/apps/storage-classes/base'
cat > 'platform/kubernetes/apps/storage-classes/base/kustomization.yaml' <<'__EOF__'
# platform/kubernetes/apps/storage-classes/base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - local-path-retain.yaml
__EOF__
```

#010 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/apps/storage-classes/overlays/halo'
cat > 'platform/kubernetes/apps/storage-classes/overlays/halo/kustomization.yaml' <<'__EOF__'
# platform/kubernetes/apps/storage-classes/overlays/halo/kustomization.yaml (aws will use gp3 instead)
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base
__EOF__
```

#011 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/argocd/clusters/halo'
cat > 'platform/kubernetes/argocd/clusters/halo/sealed-secrets.yaml' <<'__EOF__'
# platform/kubernetes/argocd/clusters/halo/sealed-secrets.yaml — controller that turns SealedSecrets into Secrets.
# No finalizer and prune off: deleting this app must never delete the controller, its CRD or the sealing key.
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: sealed-secrets
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "-5"
spec:
  project: platform
  sources:
    - repoURL: https://bitnami-labs.github.io/sealed-secrets
      chart: sealed-secrets
```

```
      targetRevision: 2.22.0
      helm:
        releaseName: sealed-secrets
        valueFiles:
          - $values/platform/kubernetes/apps/sealed-secrets/overlays/halo/values.yaml
    - repoURL: git@github.com:nikhilgs817/ai-platform-portfolio.git
      targetRevision: main
      ref: values
  destination:
    server: https://kubernetes.default.svc
    namespace: kube-system
  syncPolicy:
    automated:
      prune: false
      selfHeal: true
    syncOptions:
      - ServerSideApply=true
__EOF__
```

#012 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/argocd/clusters/halo'
cat > 'platform/kubernetes/argocd/clusters/halo/storage-classes.yaml' <<'__EOF__'
# platform/kubernetes/argocd/clusters/halo/storage-classes.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: storage-classes
  namespace: argocd
  annotations:
    argocd.argoproj.io/sync-wave: "-5"
spec:
  project: platform
  source:
    repoURL: git@github.com:nikhilgs817/ai-platform-portfolio.git
    targetRevision: main
    path: platform/kubernetes/apps/storage-classes/overlays/halo
  destination:
```

```
    server: https://kubernetes.default.svc
  syncPolicy:
    automated:
      prune: false
      selfHeal: true
__EOF__
```

#013 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/argocd/clusters/halo'
cat > 'platform/kubernetes/argocd/clusters/halo/kustomization.yaml' <<'__EOF__'
# platform/kubernetes/argocd/clusters/halo/kustomization.yaml
# Everything that runs on cluster "halo" = the child Applications listed here (sync-wave order in each file).
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - projects.yaml           # wave -10  AppProjects
  - sealed-secrets.yaml     # wave -5   platform add-on
  - storage-classes.yaml    # wave -5   platform add-on
  - podinfo.yaml            # wave 10   Week 1 sample
__EOF__
```

#014 WSL Ubuntu shell (bash) - copy & paste

```
git add -A && git commit -m "feat(platform): Sealed Secrets, local-path-retain, app-of-apps health" && git pull --rebase && git push
argocd app get root-halo --refresh >/dev/null
until argocd app get sealed-secrets >/dev/null 2>&1; do sleep 5; done
argocd app wait sealed-secrets storage-classes --health --timeout 300
kubectl -n kube-system get deploy sealed-secrets-controller
kubectl get storageclass
```

Expected: `sealed-secrets-controller 1/1`, plus StorageClasses `local-path (default)` and `local-path-retain` (`Retain`, `WaitForFirstConsumer`).

A1.3 Back up the sealing key (do not skip)

Why: if the cluster is rebuilt and the controller starts without its old key, it creates a new one, and **every SealedSecret in Git becomes unreadable**. The key backup and its restore order are the one secret step of every rebuild. `keyrenewperiod: "0"` (in the values) means there's exactly one key to keep.

#015 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p ~/.secrets && chmod 700 ~/.secrets
kubectl -n kube-system get secret -l sealedsecrets.bitnami.com/sealed-secrets-key -o yaml \
  | yq 'del(.items[].metadata.resourceVersion, .items[].metadata.uid, .items[].metadata.creationTimestamp, .items[].metadata.managedFields)' \
  > ~/.secrets/sealed-secrets-key-halo.yaml
grep -c 'tls.key' ~/.secrets/sealed-secrets-key-halo.yaml      # 1
# Attach ~/.secrets/sealed-secrets-key-halo.yaml to the password-manager entry "Sealed Secrets key - halo" NOW, then:
shred -u ~/.secrets/sealed-secrets-key-halo.yaml
kubeseal --fetch-cert > platform/kubernetes/apps/sealed-secrets/pub-cert.pem   # PUBLIC cert: lets you seal offline
```

The bootstrap script restores the key automatically (before the controller starts) when the file exists at `~/.secrets/sealed-secrets-key-halo.yaml`:

#016 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/argocd/bootstrap'
cat > 'platform/kubernetes/argocd/bootstrap/bootstrap-argocd.sh' <<'__EOF__'
#!/usr/bin/env bash
# platform/kubernetes/argocd/bootstrap/bootstrap-argocd.sh — install Argo CD and hand the cluster to GitOps.
# Usage: platform/kubernetes/argocd/bootstrap/bootstrap-argocd.sh halo
#        (deploy key expected at ~/.ssh/argocd-<env>-deploy; override with DEPLOY_KEY=...)
set -euo pipefail
ENV="${1:-halo}"
CHART_VERSION="10.11.6"                     # <- the ONE place the Argo CD chart version is pinned for bootstrap
REPO_URL="git@github.com:nikhilgs817/ai-platform-portfolio.git"
DEPLOY_KEY="${DEPLOY_KEY:-$HOME/.ssh/argocd-${ENV}-deploy}"
SEALING_KEY="${SEALING_KEY:-$HOME/.secrets/sealed-secrets-key-${ENV}.yaml}"   # Week 2: from the password manager
A="$(git rev-parse --show-toplevel)/platform/kubernetes/argocd"

[ "$(kubectl config current-context)" = "$ENV" ] || { echo "kube context must be '$ENV'"; exit 1; }
[ -f "$DEPLOY_KEY" ] || { echo "Missing $DEPLOY_KEY: restore it from the password manager"; exit 1; }

echo ">> Argo CD chart ${CHART_VERSION}"
helm repo add argo https://argoproj.github.io/argo-helm --force-update >/dev/null
helm repo update argo >/dev/null
helm upgrade --install argocd argo/argo-cd --version "$CHART_VERSION" \
  --namespace argocd --create-namespace \
  -f "$A/install/values.yaml" -f "$A/install/values-${ENV}.yaml" \
  --wait --timeout 10m
```

```
echo ">> repository credential (in-cluster Secret only; never in Git)"
kubectl -n argocd create secret generic repo-ai-platform-portfolio \
  --from-literal=type=git --from-literal=name=ai-platform-portfolio \
  --from-literal=url="$REPO_URL" --from-file=sshPrivateKey="$DEPLOY_KEY" \
  --dry-run=client -o yaml \
| kubectl label --local -f - argocd.argoproj.io/secret-type=repository -o yaml \
| kubectl apply -f -

echo ">> Sealed Secrets key (must exist BEFORE the controller starts, or old SealedSecrets cannot be decrypted)"
if [ -f "$SEALING_KEY" ]; then
  kubectl apply -f "$SEALING_KEY"
else
  echo "   WARNING: $SEALING_KEY not found: the controller will generate a NEW key; re-seal every secret (runbook)"
fi

echo ">> AppProjects + root application"
kubectl apply -k "$A/projects"
kubectl apply -f "$A/bootstrap/root-${ENV}.yaml"

cat <<MSG
>> Bootstrap complete. Now:
   1) argocd login --username admin --password "\$(argocd admin initial-password -n argocd | head -1)"
   2) argocd account update-password   (use the password-manager value)
   3) kubectl -n argocd delete secret argocd-initial-admin-secret
MSG
__EOF__
chmod +x 'platform/kubernetes/argocd/bootstrap/bootstrap-argocd.sh'
```

#017 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'docs/runbooks'
cat > 'docs/runbooks/bootstrap-secrets.md' <<'__EOF__'
# Runbook: bootstrap secrets (what is done by hand on a rebuild)

Everything else comes back from Git. These items cannot: they are the keys that make Git's ciphertext usable.

## Inventory (password manager, vault "lab-halo")
```

```
| Item | Used by | If lost |
|---|---|---|
| `sealed-secrets-key-halo.yaml` (file attachment) | Sealed Secrets controller: decrypts every SealedSecret in Git | Re-seal everything (section "Key lost") |
| `Argo CD deploy key — halo` (private key) | Argo CD reads the repo | Generate a new deploy key, replace it on GitHub |
| `Argo CD admin — halo` (password) | Argo CD UI/CLI | `argocd account update-password` after bootstrap |
| `GHCR pull — halo` (classic PAT, read:packages, 90-day expiry) | `ghcr-pull` SealedSecret | New PAT -> `platform/scripts/seal-ghcr-pull.sh` |
| `Phoenix DB` (postgres + phoenix passwords) | Timescale, `phoenix-db-app` | Needed to reach an existing data directory |
| `Phoenix app` (basic auth, Fernet key), `Redis` | command-center, redis | Re-seal with new values |

## Rebuild order
1. Week 1 runbook `rebuild-halo.md` up to (not including) the Argo CD bootstrap.
2. Restore the sealing key file **before** the controller starts:
   `install -m 600 /dev/null ~/.secrets/sealed-secrets-key-halo.yaml` then paste the attachment into it
   (the bootstrap script applies it automatically).
3. Restore the deploy key to `~/.ssh/argocd-halo-deploy` (mode 600).
4. `platform/kubernetes/argocd/bootstrap/bootstrap-argocd.sh halo` -> rotate the admin password -> delete the initial secret.
5. Verify: `kubectl -n phoenix get secrets` lists timescale-auth, phoenix-db-app, redis-auth, phoenix-secrets, ghcr-pull.
6. `shred -u ~/.secrets/sealed-secrets-key-halo.yaml` (the password manager is the copy).

## Back up the sealing key (after the first install, and after any manual rotation)
```bash
mkdir -p ~/.secrets && chmod 700 ~/.secrets
kubectl -n kube-system get secret -l sealedsecrets.bitnami.com/sealed-secrets-key -o yaml \
  | yq 'del(.items[].metadata.resourceVersion, .items[].metadata.uid, .items[].metadata.creationTimestamp, .items[].metadata.managedFields)' \
  > ~/.secrets/sealed-secrets-key-halo.yaml
```
Attach the file to the password manager entry, then `shred -u ~/.secrets/sealed-secrets-key-halo.yaml`.

Key lost (controller generated a new key)
SealedSecrets show `no key could decrypt secret`. Re-run `seal-ghcr-pull.sh`, `seal-phoenix-db.sh` (reuse the DB passwords
from the password manager if the data directory survived: seal them with `seal-secret.sh` instead of generating),
`seal-secret.sh` for redis-auth and phoenix-secrets; commit; back up the new key.

Rotation calendar
GHCR PAT every 90 days (GitHub reminds you). Sealing key yearly (manual). Deploy key yearly.
__EOF__
```

#018 WSL Ubuntu shell (bash) - copy & paste

```
git add -A && git commit -m "docs(runbook): bootstrap secrets; restore sealing key in bootstrap" && git pull --rebase && git push
```

Verify:

#019 WSL Ubuntu shell (bash) - copy & paste

```
echo -n hello | kubectl create secret generic t --dry-run=client --from-file=v=/dev/stdin -o yaml | kubeseal --format yaml | head -5
```

Expected: `kind: SealedSecret` with `encryptedData: v: AgB…`.

If it fails

1. `sealed-secrets` app `ComparisonError … not permitted in project` → the `platform` project is missing the `https://bitnami-labs.github.io/sealed-secrets` repo. Push the projects first.
2. `kubeseal: cannot fetch certificate` → the controller name or namespace doesn't match. Check `fullnameOverride: sealed-secrets-controller`, namespace `kube-system`.
3. `phoenix-data` waits later with `waiting for healthy state of argoproj.io/Application` forever → that child really is unhealthy. Run `argocd app get <child>`.

- ☐ Health check active · ☐ controller running · ☐ key in the password manager, local copy shredded · ☐ `local-path-retain` exists

CKA corner: StorageClasses, `reclaimPolicy`, `volumeBindingMode` (Storage 10%): `kubectl get sc`, `kubectl describe pv`, `kubectl get pvc -A`. Secrets are base64, not encrypted; sealed and encrypted at rest are different layers.

Interview angle: "Secrets are sealed with the cluster's public key, so only ciphertext lives in Git, and the single private key is the one artefact the rebuild runbook treats as tier-0."

Phoenix angle: Timescale's passwords and Phoenix's Fernet key can now live in Git safely, and a Retain StorageClass means a mistaken PVC delete can't wipe the database.

Drill W2-1 (15 min, `kcka`): core concepts and pods. Answers in the appendix.

1. Create pod `web` (image `nginx:1.29`) with label `tier=frontend`.
2. Print `web`'s pod IP and node name in one command (jsonpath).
3. Without creating it, write `/tmp/busy.yaml` for pod `busy` (`busybox:1.37.0`, command `sleep 3600`), then create it from the file.
4. From `busy`, fetch the nginx welcome page from `web`'s IP.
5. Delete both pods immediately, without waiting for graceful termination.

A2: FastAPI template: code, tests, Makefile (Fri Nov 13)

Goal: the template's application half: typed schemas, `/healthz` `/readyz` `/metrics`, JSON logs, env-only config, pytest, ruff and mypy behind `make ci`.

Why: a paved road is opinions encoded once. **Liveness** (`/healthz`) only says the process answers, so a database outage never restarts pods. **Readiness** (`/readyz`) runs dependency checks and takes the pod out of the Service until they pass. Metrics use route *templates* as labels (`/items/{id}`), so cardinality stays bounded. The Makefile's `ci` target is the **only** contract CI knows, so Phoenix (pip) and template services (uv) share one pipeline.

Where: [WSL] / [VS Code]

Copier renders files ending in `.jinja` and file names containing `[[ … ]]`. Everything else is copied verbatim. The delimiters are `[[ ]]` because GitHub Actions and Helm already use `{{ }}`.

#020 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service'
cat > 'platform/templates/fastapi-service/copier.yml' <<'__EOF__'
# platform/templates/fastapi-service/copier.yml — the paved-road FastAPI service template.
# Run through platform/scripts/new-service.sh (it validates, registers the Argo CD app and locks deps).
_min_copier_version: "9.19.0"
_subdirectory: template
_answers_file: .copier-answers/service.yml      # new-service.sh passes -a .copier-answers/<name>.yml
_envops:                                        # [[ ]] instead of {{ }}: workflows and Helm already use {{ }}
  block_start_string: "[%"
  block_end_string: "%]"
  variable_start_string: "[["
  variable_end_string: "]]"
  comment_start_string: "[#"
  comment_end_string: "#]"
  keep_trailing_newline: true

service_name:
  type: str
  help: Service name (lowercase letters, digits, dashes; 3-30 chars)
  validator: >-
    [% if service_name != service_name|lower or ' ' in service_name or '_' in service_name
          or service_name|length < 3 or service_name|length > 30 %]use lowercase letters, digits and dashes (3-30)[% endif %]
description:
  type: str
  default: A service on the lab paved road
owner:
  type: str
  default: platform
```

```
github_owner:
  type: str
  default: nikhilgs817
namespace:
  type: str
  help: Kubernetes namespace (default = its own namespace, with guardrails)
  default: "[[ service_name ]]"
replicas:
  type: int
  default: 2
sync_wave:
  type: int
  default: 10
__EOF__
```

#021 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service'
cat > 'platform/templates/fastapi-service/README.md' <<'__EOF__'
# platform/templates/fastapi-service — Copier template

Creates, in one go: `projects/<name>/` (FastAPI app, tests, Makefile, Dockerfile), `platform/kubernetes/apps/<name>/`
(Helm values for `platform/helm-charts/service`), the Argo CD Application in `argocd/clusters/halo/`, and the
caller workflow `.github/workflows/svc-<name>.yml`. Use `platform/scripts/new-service.sh <name>`.
__EOF__
```

#022 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template'
cat > 'platform/templates/fastapi-service/template/[[ _copier_conf.answers_file ]].jinja' <<'__EOF__'
# Written by Copier: the answers this service was generated with (keep it; `copier update` needs it later).
[[ _copier_answers|to_nice_yaml ]]
__EOF__
```

#023 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/projects/[[ service_name ]]'
cat > 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/pyproject.toml.jinja' <<'__EOF__'
[project]
name = "[[ service_name ]]"
```

```
version = "0.1.0"
description = "[[ description ]]"
requires-python = ">=3.14,<3.15"
dependencies = [
  "fastapi==0.162.0",
  "uvicorn==0.54.0",
  "pydantic==2.14.2",
  "pydantic-settings==2.16.2",
  "prometheus-client==0.26.0",
]

[dependency-groups]
dev = [
  "pytest==9.2.0",
  "httpx2==2.16.0",                  # Starlette TestClient transport (httpx is deprecated there)
  "ruff==0.17.3",
  "mypy==2.4.1",
]

[tool.uv]
package = false                      # an app, not a library: uv installs dependencies only

[tool.ruff]
line-length = 120
target-version = "py314"

[tool.ruff.lint]
select = ["E", "F", "W", "I", "B", "UP", "S", "ASYNC"]

[tool.ruff.lint.per-file-ignores]
"tests/**" = ["S101"]                # assert is how pytest works

[tool.mypy]
strict = true
files = ["app"]
plugins = ["pydantic.mypy"]

[tool.pytest.ini_options]
```

```
testpaths = ["tests"]
pythonpath = ["."]                  # import `app` from the project root
addopts = "-q"
__EOF__
```

#024 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/projects/[[ service_name ]]'
cat > 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/Makefile' <<'__EOF__'
# projects/<service>/Makefile — the CI contract: the reusable workflow only ever runs `make ci`.
# Recipe lines start with ">" instead of a TAB (copy-paste from a PDF turns TABs into spaces).
.RECIPEPREFIX = >
.PHONY: install lint fmt typecheck test ci run

install:
> uv sync --frozen

lint:
> uv run ruff check .
> uv run ruff format --check .

fmt:
> uv run ruff format .
> uv run ruff check --fix .

typecheck:
> uv run mypy

test:
> uv run pytest

ci: install lint typecheck test

run:
> APP_ENVIRONMENT=local uv run uvicorn app.main:app --reload --port 8000
__EOF__
```

#025 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/projects/[[ service_name ]]'
cat > 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/README.md.jinja' <<'__EOF__'
# [[ service_name ]]

[[ description ]]

Generated from `platform/templates/fastapi-service` (Copier). Owner: [[ owner ]].

| What | Where |
|---|---|
| Code | `projects/[[ service_name ]]/app` |
| Deploy values | `platform/kubernetes/apps/[[ service_name ]]/{base,overlays/halo}/values.yaml` |
| Argo CD Application | `platform/kubernetes/argocd/clusters/halo/[[ service_name ]].yaml` |
| CI | `.github/workflows/svc-[[ service_name ]].yml` -> `_service-ci.yml` |
| URL (halo) | http://[[ service_name ]].halo.localhost |

```bash
make install && make ci      # lint, typecheck, test
make run                     # http://127.0.0.1:8000/docs
```
__EOF__
```

#026 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/app'
cat > 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/app/__init__.py' <<'__EOF__'
"""Service package. The import path is always `app` so the Dockerfile, Makefile and chart stay identical per service."""
__EOF__
```

#027 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/app'
cat > 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/app/config.py.jinja' <<'__EOF__'
"""All configuration comes from environment variables prefixed APP_. No config files, no secret defaults."""

from __future__ import annotations

from functools import lru_cache
```

```
from typing import Literal

from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_prefix="APP_", extra="ignore")

    service_name: str = "[[ service_name ]]"
    environment: str = "local"
    version: str = "dev"  # set at build time (Dockerfile ARG APP_VERSION = git SHA)
    log_level: Literal["DEBUG", "INFO", "WARNING", "ERROR"] = "INFO"
    greeting: str = "hello from the paved road"


@lru_cache
def get_settings() -> Settings:
    return Settings()
__EOF__
```

#028 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/app'
cat > 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/app/logging_setup.py' <<'__EOF__'
"""Structured JSON logs on stdout: one object per line, ready for Loki (Week 7)."""

from __future__ import annotations

import json
import logging
import sys
from datetime import UTC, datetime

_EXTRA_FIELDS = ("method", "path", "status", "duration_ms", "request_id")


class JsonFormatter(logging.Formatter):
    def __init__(self, service: str) -> None:
```

```
        super().__init__()
        self.service = service

    def format(self, record: logging.LogRecord) -> str:
        payload: dict[str, object] = {
            "ts": datetime.fromtimestamp(record.created, UTC).isoformat(timespec="milliseconds"),
            "level": record.levelname.lower(),
            "service": self.service,
            "logger": record.name,
            "msg": record.getMessage(),
        }
        for key in _EXTRA_FIELDS:
            if hasattr(record, key):
                payload[key] = getattr(record, key)
        if record.exc_info:
            payload["exc"] = self.formatException(record.exc_info)
        return json.dumps(payload, default=str)


def configure_logging(level: str, service: str) -> None:
    handler = logging.StreamHandler(sys.stdout)
    handler.setFormatter(JsonFormatter(service))
    root = logging.getLogger()
    root.handlers[:] = [handler]
    root.setLevel(level)
    for name in ("uvicorn", "uvicorn.error"):
        lg = logging.getLogger(name)
        lg.handlers[:] = []
        lg.propagate = True
    logging.getLogger("uvicorn.access").disabled = True  # the app middleware logs requests as JSON instead
__EOF__
```

#029 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/app'
cat > 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/app/metrics.py' <<'__EOF__'
"""Prometheus metrics. Labels use the route template (/items/{id}), never the raw path, to keep cardinality low."""
```

```
from __future__ import annotations

from prometheus_client import CONTENT_TYPE_LATEST, Counter, Histogram, generate_latest
from starlette.requests import Request
from starlette.responses import Response

REQUESTS = Counter("http_requests_total", "HTTP requests", ["method", "route", "status"])
LATENCY = Histogram("http_request_duration_seconds", "HTTP request latency", ["method", "route"])
UNOBSERVED = frozenset({"/healthz", "/readyz", "/metrics"})


def route_of(request: Request) -> str:
    route = request.scope.get("route")
    return str(getattr(route, "path", "unmatched"))


def metrics_response() -> Response:
    return Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)
__EOF__
```

#030 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/app'
cat > 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/app/schemas.py' <<'__EOF__'
"""Typed request/response models: the API contract, validated on the way in and on the way out."""

from __future__ import annotations

from typing import Literal

from pydantic import BaseModel, Field


class Health(BaseModel):
    status: Literal["ok"] = "ok"


class Ready(BaseModel):
```

```
    status: Literal["ready", "not-ready"]
    checks: dict[str, bool]


class Info(BaseModel):
    service: str
    version: str
    environment: str


class EchoIn(BaseModel):
    message: str = Field(min_length=1, max_length=200)


class EchoOut(BaseModel):
    message: str
    length: int
    service: str
__EOF__
```

#031 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/app'
cat > 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/app/main.py' <<'__EOF__'
"""FastAPI app factory: health, readiness, metrics, JSON access logs and a tiny typed API."""

from __future__ import annotations

import logging
import time
import uuid
from collections.abc import AsyncIterator, Awaitable, Callable
from contextlib import asynccontextmanager

from fastapi import FastAPI, Request, Response
from fastapi.responses import JSONResponse

from app.config import Settings, get_settings
```

```
from app.logging_setup import configure_logging
from app.metrics import LATENCY, REQUESTS, UNOBSERVED, metrics_response, route_of
from app.schemas import EchoIn, EchoOut, Health, Info, Ready

log = logging.getLogger("app")
ReadinessCheck = Callable[[], Awaitable[bool]]


def create_app(settings: Settings | None = None, checks: dict[str, ReadinessCheck] | None = None) -> FastAPI:
    """`checks` are async callables (database, cache, ...) that must all pass for /readyz to return 200."""
    cfg = settings or get_settings()
    configure_logging(cfg.log_level, cfg.service_name)
    readiness = checks or {}
    state = {"started": False}

    @asynccontextmanager
    async def lifespan(_: FastAPI) -> AsyncIterator[None]:
        state["started"] = True
        log.info("startup version=%s environment=%s", cfg.version, cfg.environment)
        yield
        state["started"] = False
        log.info("shutdown")

    app = FastAPI(title=cfg.service_name, version=cfg.version, lifespan=lifespan)

    @app.middleware("http")
    async def observe(request: Request, call_next: Callable[[Request], Awaitable[Response]]) -> Response:
        start = time.perf_counter()
        request_id = request.headers.get("x-request-id") or uuid.uuid4().hex
        response = await call_next(request)
        route = route_of(request)
        if route not in UNOBSERVED:
            elapsed = time.perf_counter() - start
            REQUESTS.labels(request.method, route, str(response.status_code)).inc()
            LATENCY.labels(request.method, route).observe(elapsed)
            log.info(
                "request",
                extra={
```

```
                "method": request.method,
                "path": route,
                "status": response.status_code,
                "duration_ms": round(elapsed * 1000, 1),
                "request_id": request_id,
            },
        )
    response.headers["x-request-id"] = request_id
    return response

@app.get("/healthz", response_model=Health)
async def healthz() -> Health:
    """Liveness: the process answers. Never checks dependencies (a DB outage must not restart pods)."""
    return Health()

@app.get("/readyz", response_model=Ready)
async def readyz() -> JSONResponse:
    """Readiness: started and every dependency check passes; otherwise 503 and the Service stops routing here."""
    results = {name: await check() for name, check in readiness.items()}
    ok = state["started"] and all(results.values())
    body = Ready(status="ready" if ok else "not-ready", checks=results)
    return JSONResponse(body.model_dump(), status_code=200 if ok else 503)

@app.get("/metrics", include_in_schema=False)
async def metrics() -> Response:
    return metrics_response()

@app.get("/api/v1/info", response_model=Info)
async def info() -> Info:
    return Info(service=cfg.service_name, version=cfg.version, environment=cfg.environment)

@app.post("/api/v1/echo", response_model=EchoOut)
async def echo(body: EchoIn) -> EchoOut:
    return EchoOut(message=f"{cfg.greeting}: {body.message}", length=len(body.message), service=cfg.service_name)

return app
```

```
app = create_app()
__EOF__
```

#032 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/tests'
cat > 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/tests/conftest.py' <<'__EOF__'
from __future__ import annotations

from collections.abc import Iterator

import pytest
from fastapi.testclient import TestClient

from app.config import Settings
from app.main import create_app


@pytest.fixture
def settings() -> Settings:
    return Settings(service_name="test-svc", environment="test", version="abc123")


@pytest.fixture
def client(settings: Settings) -> Iterator[TestClient]:
    with TestClient(create_app(settings)) as c:  # the context manager runs the lifespan (startup/shutdown)
        yield c
__EOF__
```

#033 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/tests'
cat > 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/tests/test_health.py' <<'__EOF__'
from __future__ import annotations

from fastapi.testclient import TestClient

from app.config import Settings
from app.main import create_app
```

```
def test_liveness(client: TestClient) -> None:
    r = client.get("/healthz")
    assert r.status_code == 200
    assert r.json() == {"status": "ok"}


def test_ready_when_started(client: TestClient) -> None:
    r = client.get("/readyz")
    assert r.status_code == 200
    assert r.json()["status"] == "ready"


def test_not_ready_when_a_dependency_fails(settings: Settings) -> None:
    async def db_down() -> bool:
        return False

    with TestClient(create_app(settings, checks={"db": db_down})) as c:
        r = c.get("/readyz")
    assert r.status_code == 503
    assert r.json() == {"status": "not-ready", "checks": {"db": False}}
__EOF__
```

#034 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/tests'
cat > 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/tests/test_api.py' <<'__EOF__'
from __future__ import annotations

from fastapi.testclient import TestClient


def test_info(client: TestClient) -> None:
    r = client.get("/api/v1/info")
    assert r.status_code == 200
    assert r.json() == {"service": "test-svc", "version": "abc123", "environment": "test"}
    assert r.headers["x-request-id"]


def test_echo_validates_input(client: TestClient) -> None:
    assert client.post("/api/v1/echo", json={"message": ""}).status_code == 422
    r = client.post("/api/v1/echo", json={"message": "hi"})
    assert r.status_code == 200
    assert r.json()["length"] == 2
__EOF__
```

#035 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/tests'
cat > 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/tests/test_metrics.py' <<'__EOF__'
from __future__ import annotations

from fastapi.testclient import TestClient


def test_metrics_use_route_templates(client: TestClient) -> None:
    client.get("/api/v1/info")
    body = client.get("/metrics").text
    assert 'http_requests_total{method="GET",route="/api/v1/info",status="200"}' in body
    assert 'route="/healthz"' not in body  # probes are not counted
__EOF__
```

#036 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/tests'
cat > 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/tests/test_logging.py' <<'__EOF__'
from __future__ import annotations

import json
import logging

from app.logging_setup import JsonFormatter


def test_json_log_line() -> None:
    record = logging.LogRecord("app", logging.INFO, __file__, 1, "request", None, None)
    record.status = 200
    line = json.loads(JsonFormatter("svc").format(record))
    assert line["level"] == "info"
    assert line["service"] == "svc"
    assert line["msg"] == "request"
    assert line["status"] == 200
__EOF__
```

Try the template without touching the repo. Render it into `/tmp` and run the CI contract:

#037 WSL Ubuntu shell (bash) - copy & paste

```
rm -rf /tmp/tpl-test && uvx copier@9.19.1 copy --quiet --defaults --data service_name=tpl-test \
  platform/templates/fastapi-service /tmp/tpl-test
cd /tmp/tpl-test/projects/tpl-test && uv lock --quiet && make ci
APP_VERSION=local uv run uvicorn app.main:app --port 8077 &
sleep 3; curl -s localhost:8077/readyz; echo; curl -s localhost:8077/api/v1/info; echo
curl -s -XPOST localhost:8077/api/v1/echo -H 'content-type: application/json' -d '{"message":"hi"}'; echo
curl -s localhost:8077/metrics | grep '^http_requests_total'; kill %1
cd ~/src/ai-platform-portfolio
```

Expected: `All checks passed!` · `12 files already formatted` · `Success: no issues found in 6 source files` · `7 passed`. Then `{"status":"ready","checks":{}}`, the info JSON with `"version":"local"`, the echo JSON, and two `http_requests_total{…route="/api/v1/…"…}` lines. The uvicorn output is one JSON object per line.

#038 WSL Ubuntu shell (bash) - copy & paste

```
git add -A && git commit -m "feat(template): FastAPI service template (code, tests, Makefile)" && git pull --rebase && git push
```

If it fails

1. `ModuleNotFoundError: No module named 'app'` in pytest → `pythonpath = ["."]` is missing in `pyproject.toml.jinja`.
2. `uv lock` downloads Python 3.14 and stalls → uv fetches managed Python builds from GitHub. Retry, or `uv python install 3.14` first.
3. Copier error `Question "service_name" is required` → you omitted `--data service_name=…` with `--defaults`.

- ☐ `make ci` green on a rendered copy · ☐ endpoints answer, logs are JSON

CKA corner: probes are core CKA material. Know `livenessProbe` vs `readinessProbe` vs `startupProbe`, and what each failure does (restart vs remove from endpoints): `kubectl describe pod` → *Conditions*, `kubectl get endpointslices`.

Interview angle: "Every service starts with the same health, readiness and metrics contract and the same `make ci`, so platform tooling (probes, dashboards, CI) works on day one without per-team glue."

Phoenix angle: the same split fixes a real Phoenix risk. Its old manifests used the database-checking `/healthz` for liveness, so a database blip would have restarted the app. PR #3 adds `/livez`.

Drill W2-2 (15 min, `kcka`): deployments, labels and selectors.

1. Create deployment `api` (`nginx:1.29`, 3 replicas).
2. List its pods with their labels shown as columns `app` and `pod-template-hash`.
3. Add label `env=lab` to the *pod template* (not to running pods) and watch new pods replace the old ones.
4. Count pods matching `app=api,env=lab`.
5. List pods whose `tier` label is either `frontend` or `backend` (set-based selector).

A3: Dockerfile and the shared Helm chart (Sat Nov 14, block 1)

A3.1 Dockerfile

Goal: a small, cacheable, non-root image that works with a read-only root filesystem.

Why: two stages. *Build* installs locked dependencies into `/app/.venv` (this layer only changes when `uv.lock` changes, so most CI builds reuse it). *Runtime* copies only the venv and the code: no compiler, no uv, no cache. The base image is pinned by tag **and digest**, so "python 3.14" can't silently change under you. `UV_COMPILE_BYTECODE` precompiles at build time, so nothing writes `.pyc` files at runtime (read-only FS).

Where: [WSL]

#039 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/projects/[[ service_name ]]'
cat > 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/Dockerfile' <<'__EOF__'
```

```
# projects/<service>/Dockerfile — multi-stage, pinned, non-root, read-only-root-filesystem friendly.
# Base images are pinned by tag AND digest; Dependabot (docker ecosystem) proposes bumps.
ARG PYTHON_IMAGE=python:3.14.10-slim-trixie@sha256:930e1abba7c4e8f96ed93e8012c35d174d7732752731bbd5e8e757d7ec6f3966

FROM ghcr.io/astral-sh/uv:0.13.6 AS uv

FROM ${PYTHON_IMAGE} AS build
ENV UV_COMPILE_BYTECODE=1 UV_LINK_MODE=copy UV_PYTHON_DOWNLOADS=never UV_PROJECT_ENVIRONMENT=/app/.venv
COPY --from=uv /uv /usr/local/bin/uv
WORKDIR /app
# Dependencies first: this layer is cached until pyproject.toml or uv.lock changes.
COPY pyproject.toml uv.lock ./
RUN --mount=type=cache,target=/root/.cache/uv uv sync --frozen --no-dev

FROM ${PYTHON_IMAGE} AS runtime
ARG APP_VERSION=dev
ENV PATH=/app/.venv/bin:$PATH \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    APP_VERSION=${APP_VERSION}
RUN useradd --uid 10001 --no-create-home --shell /usr/sbin/nologin app
WORKDIR /app
COPY --from=build /app/.venv /app/.venv
COPY app ./app
USER 10001
EXPOSE 8000
ENTRYPOINT ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--no-access-log"]
__EOF__
```

#040 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/projects/[[ service_name ]]'
cat > 'platform/templates/fastapi-service/template/projects/[[ service_name ]]/.dockerignore' <<'__EOF__'
.venv
**/__pycache__
**/*.pyc
.pytest_cache
.mypy_cache
.ruff_cache
tests
Makefile
README.md
.env
.env.*
__EOF__
```

There's no Docker in WSL on purpose (k3s uses its own containerd). The image is built by CI in A5. If you want a local build, Docker Desktop is the only Windows option, and Week 1 asked you not to install it.

A3.2 The shared chart `platform/helm-charts/service`

Goal: one chart every service uses: Deployment, Service, Ingress, ServiceAccount, ConfigMap, Secret references, probes, requests and limits, PDB, optional HPA, workers, a migration hook, one-shot jobs, and optional namespace guardrails.

Why: the chart is the "deployment half" of the paved road. Notable defaults:

- `restricted` Pod Security + Kyverno baseline: non-root, no privilege escalation, all capabilities dropped, seccomp `RuntimeDefault`, read-only root with an `emptyDir` `/tmp`.
- `maxUnavailable: 1` PDB, which never blocks a drain at 1 replica (a `minAvailable: 1` PDB would).
- No CPU limit: requests schedule, limits only throttle.
- **Empty `image.tag` renders `replicas: 0`**, so a brand-new service is Healthy before its first image exists.
- **Sync waves:** ServiceAccount, ConfigMap and pull secret at -1 → migration hook at 1 → Deployments at 2 → one-shot jobs at 3. A `PreSync` hook would run *before* the ServiceAccount and ConfigMap exist on a first install. **Where:** [WSL]

#041 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/helm-charts/service'
cat > 'platform/helm-charts/service/Chart.yaml' <<'__EOF__'
# platform/helm-charts/service/Chart.yaml — the paved-road chart: one HTTP service plus optional workers and jobs.
apiVersion: v2
name: service
description: Paved-road chart for HTTP services on the lab platform (halo now, aws later)
type: application
version: 0.1.0
appVersion: "0.1.0"
kubeVersion: ">=1.30.0-0"
__EOF__
```

#042 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/helm-charts/service'
cat > 'platform/helm-charts/service/values.yaml' <<'__EOF__'
# platform/helm-charts/service/values.yaml — defaults for every service.
# Layering (Argo CD valueFiles, last wins): this file -> apps/_shared/<env>/values.yaml
#   -> apps/<svc>/base/values.yaml -> apps/<svc>/overlays/<env>/values.yaml (CI writes image.tag here).
nameOverride: ""

image:
  repository: ""                # ghcr.io/<owner>/<service>
  tag: ""                       # the git SHA, written by CI. Empty = nothing built yet = replicas 0
  pullPolicy: IfNotPresent

imagePullSecret:
  create: false                 # true: render the cluster-wide SealedSecret "ghcr-pull" in this namespace
  name: ghcr-pull
  encryptedDockerConfigJson: ""  # kubeseal ciphertext (safe to commit); set in apps/_shared/<env>/values.yaml

replicaCount: 1
command: []                     # empty = the image ENTRYPOINT
args: []
containerPort: 8000

env: {}                         # plain, non-secret config -> ConfigMap -> envFrom
```

```
envFromSecrets: []              # Secret names; every key becomes an env var
secretEnv: []                   # [{name: ENV_VAR, secretName: s, key: k, optional: false}]
extraEnv: []                    # raw EnvVar list, rendered last; may use $(VAR) defined above

podLabels: {}                   # added to every pod (app, workers, jobs); NetworkPolicies select on these
podAnnotations: {}

probes:
  startup: {path: /healthz, periodSeconds: 2, failureThreshold: 30}    # up to 60 s to boot
  liveness: {path: /healthz, periodSeconds: 10, failureThreshold: 3}   # restart only if the process is stuck
  readiness: {path: /readyz, periodSeconds: 5, failureThreshold: 3}    # take out of the Service if a dependency fails

resources:
  requests: {cpu: 50m, memory: 128Mi}
  limits: {memory: 256Mi}       # no CPU limit on purpose: requests schedule, limits would only throttle

podSecurityContext:             # Pod Security "restricted" and Kyverno baseline/restricted compatible
  runAsNonRoot: true
  runAsUser: 10001
  runAsGroup: 10001
  fsGroup: 10001
  seccompProfile: {type: RuntimeDefault}
securityContext:
  allowPrivilegeEscalation: false
  readOnlyRootFilesystem: true
  capabilities: {drop: [ALL]}
tmpDir: {enabled: true, sizeLimit: 64Mi}   # writable /tmp on a read-only root filesystem

service:
  enabled: true
  port: 80

metrics:
  enabled: true                 # prometheus.io/* annotations for Week 7
  port: 0                       # 0 = same port as the app; otherwise an extra container port named "metrics"
  path: /metrics

ingress:
```

```
  enabled: false
  className: traefik
  host: ""
  path: /

serviceAccount:
  create: true
  automountToken: false          # no Kubernetes API token in the pod unless a service needs one

pdb:
  enabled: true
  maxUnavailable: 1              # never blocks a drain, even at replicas 1 (minAvailable: 1 would)

autoscaling:
  enabled: false                 # true: HPA owns replicas (and Argo CD stops caring about spec.replicas)
  minReplicas: 2
  maxReplicas: 4
  targetCPUUtilizationPercentage: 70

workers: []                      # extra Deployments from the same image: [{name, args, replicas, resources, strategy}]

migrations:
  enabled: false                 # Argo CD PreSync Job: runs before every sync, blocks the rollout on failure
  command: []
  args: []
  resources: {requests: {cpu: 50m, memory: 128Mi}, limits: {memory: 512Mi}}

jobs: []                         # one-shot Jobs [{name, args}]. Rename to run again (Jobs are immutable).

guardrails:
  enabled: false                 # namespace ResourceQuota + LimitRange + default-deny NetworkPolicy
  quota:
    requests.cpu: "1"
    requests.memory: 1Gi
    limits.memory: 2Gi
    pods: "10"
  ingressFromCIDRs: []           # who may reach the app port; set per env in apps/_shared/<env>/values.yaml
```

```
global: {}
__EOF__
```

#043 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/helm-charts/service/templates'
cat > 'platform/helm-charts/service/templates/_helpers.tpl' <<'__EOF__'
{{/* platform/helm-charts/service/templates/_helpers.tpl — names, labels and the pod spec shared by every workload */}}
{{- define "svc.name" -}}
{{- default .Release.Name .Values.nameOverride | trunc 63 | trimSuffix "-" -}}
{{- end -}}

{{- define "svc.labels" -}}
app.kubernetes.io/name: {{ include "svc.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
app.kubernetes.io/version: {{ .Values.image.tag | default "none" | trunc 12 | quote }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
helm.sh/chart: {{ printf "%s-%s" .Chart.Name .Chart.Version }}
{{- end -}}

{{/* selector labels never change after the first deploy (Deployment selectors are immutable) */}}
{{- define "svc.selector" -}}
app.kubernetes.io/name: {{ include "svc.name" .root }}
app.kubernetes.io/component: {{ .component }}
{{- end -}}

{{- define "svc.image" -}}
{{ required "image.repository is required" .Values.image.repository }}:{{ .Values.image.tag | default "pending" }}
{{- end -}}

{{- define "svc.env" -}}
{{- if or .Values.env .Values.envFromSecrets }}
envFrom:
  {{- if .Values.env }}
  - configMapRef:
      name: {{ include "svc.name" . }}-env
  {{- end }}
  {{- range .Values.envFromSecrets }}
```

```
  - secretRef:
      name: {{ . }}
  {{- end }}
{{- end }}
{{- if or .Values.secretEnv .Values.extraEnv }}
env:
  {{- range .Values.secretEnv }}
  - name: {{ .name }}
    valueFrom:
      secretKeyRef:
        name: {{ .secretName }}
        key: {{ .key }}
        optional: {{ .optional | default false }}
  {{- end }}
  {{- with .Values.extraEnv }}
  {{- toYaml . | nindent 2 }}
  {{- end }}
{{- end }}
{{- end -}}

{{/* pod spec. Call with: dict "root" $ "component" "api" "image" "<ref>" "command" [] "args" [] "resources" {} "main" true "restartPolicy" "" */}}
{{- define "svc.podSpec" -}}
{{- $v := .root.Values -}}
serviceAccountName: {{ include "svc.name" .root }}
automountServiceAccountToken: {{ $v.serviceAccount.automountToken }}
{{- if $v.imagePullSecret.name }}
imagePullSecrets:
  - name: {{ $v.imagePullSecret.name }}
{{- end }}
securityContext:
  {{- toYaml $v.podSecurityContext | nindent 2 }}
{{- with .restartPolicy }}
restartPolicy: {{ . }}
{{- end }}
containers:
  - name: {{ .component }}
    image: {{ .image | default (include "svc.image" .root) | quote }}
    imagePullPolicy: {{ $v.image.pullPolicy }}
```

```
{{- with .command }}
command:
  {{- toYaml . | nindent 6 }}
{{- end }}
{{- with .args }}
args:
  {{- toYaml . | nindent 6 }}
{{- end }}
{{- if .main }}
ports:
  - name: http
    containerPort: {{ $v.containerPort }}
  {{- if and $v.metrics.enabled $v.metrics.port }}
  - name: metrics
    containerPort: {{ $v.metrics.port }}
  {{- end }}
startupProbe:
  httpGet: {path: {{ $v.probes.startup.path }}, port: http}
  periodSeconds: {{ $v.probes.startup.periodSeconds }}
  failureThreshold: {{ $v.probes.startup.failureThreshold }}
livenessProbe:
  httpGet: {path: {{ $v.probes.liveness.path }}, port: http}
  periodSeconds: {{ $v.probes.liveness.periodSeconds }}
  failureThreshold: {{ $v.probes.liveness.failureThreshold }}
readinessProbe:
  httpGet: {path: {{ $v.probes.readiness.path }}, port: http}
  periodSeconds: {{ $v.probes.readiness.periodSeconds }}
  failureThreshold: {{ $v.probes.readiness.failureThreshold }}
{{- end }}
{{- include "svc.env" .root | nindent 4 }}
resources:
  {{- toYaml .resources | nindent 6 }}
securityContext:
  {{- toYaml $v.securityContext | nindent 6 }}
{{- if $v.tmpDir.enabled }}
volumeMounts:
  - name: tmp
    mountPath: /tmp
```

```
    {{- end }}
{{- if $v.tmpDir.enabled }}
volumes:
  - name: tmp
    emptyDir:
      sizeLimit: {{ $v.tmpDir.sizeLimit }}
{{- end }}
{{- end -}}
__EOF__
```

#044 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/helm-charts/service/templates'
cat > 'platform/helm-charts/service/templates/deployment.yaml' <<'__EOF__'
{{- /* platform/helm-charts/service/templates/deployment.yaml — the main HTTP process (component "api") */ -}}
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "svc.name" . }}
  labels:
    {{- include "svc.labels" . | nindent 4 }}
    app.kubernetes.io/component: api
  annotations:
    argocd.argoproj.io/sync-wave: "2"
spec:
  {{- if not .Values.autoscaling.enabled }}
  replicas: {{ if .Values.image.tag }}{{ .Values.replicaCount }}{{ else }}0{{ end }}
  {{- end }}
  revisionHistoryLimit: 5
  progressDeadlineSeconds: 180
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxUnavailable: 0, maxSurge: 1}
  selector:
    matchLabels:
      {{- include "svc.selector" (dict "root" . "component" "api") | nindent 6 }}
  template:
    metadata:
```

```
      labels:
        {{- include "svc.labels" . | nindent 8 }}
        app.kubernetes.io/component: api
        {{- with .Values.podLabels }}
        {{- toYaml . | nindent 8 }}
        {{- end }}
      annotations:
        checksum/env: {{ toJson .Values.env | sha256sum | trunc 16 | quote }}
        {{- if .Values.metrics.enabled }}
        prometheus.io/scrape: "true"
        prometheus.io/port: {{ .Values.metrics.port | default .Values.containerPort | quote }}
        prometheus.io/path: {{ .Values.metrics.path | quote }}
        {{- end }}
        {{- with .Values.podAnnotations }}
        {{- toYaml . | nindent 8 }}
        {{- end }}
    spec:
      {{- include "svc.podSpec" (dict "root" . "component" "api" "main" true
            "command" .Values.command "args" .Values.args "resources" .Values.resources) | nindent 6 }}
__EOF__
```

#045 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/helm-charts/service/templates'
cat > 'platform/helm-charts/service/templates/workers.yaml' <<'__EOF__'
{{- /* platform/helm-charts/service/templates/workers.yaml — background processes from the same image (no Service) */ -}}
{{- range .Values.workers }}
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "svc.name" $ }}-{{ .name }}
  labels:
    {{- include "svc.labels" $ | nindent 4 }}
    app.kubernetes.io/component: {{ .name }}
  annotations:
    argocd.argoproj.io/sync-wave: "2"
spec:
```

```
  replicas: {{ if $.Values.image.tag }}{{ if hasKey . "replicas" }}{{ .replicas }}{{ else }}1{{ end }}{{ else }}0{{ end }}
  revisionHistoryLimit: 5
  strategy:
    {{- toYaml (.strategy | default (dict "type" "Recreate")) | nindent 4 }}
  selector:
    matchLabels:
      {{- include "svc.selector" (dict "root" $ "component" .name) | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "svc.labels" $ | nindent 8 }}
        app.kubernetes.io/component: {{ .name }}
        {{- with $.Values.podLabels }}
        {{- toYaml . | nindent 8 }}
        {{- end }}
      annotations:
        checksum/env: {{ toJson $.Values.env | sha256sum | trunc 16 | quote }}
    spec:
      {{- include "svc.podSpec" (dict "root" $ "component" .name "main" false
            "command" .command "args" .args "resources" (.resources | default $.Values.resources)) | nindent 6 }}
{{- end }}
__EOF__
```

#046 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/helm-charts/service/templates'
cat > 'platform/helm-charts/service/templates/jobs.yaml' <<'__EOF__'
{{- /* platform/helm-charts/service/templates/jobs.yaml — migrations (Sync hook, every sync) and one-shot Jobs */ -}}
{{- if and .Values.migrations.enabled .Values.image.tag }}
apiVersion: batch/v1
kind: Job
metadata:
  name: {{ include "svc.name" . }}-migrate
  labels:
    {{- include "svc.labels" . | nindent 4 }}
    app.kubernetes.io/component: migrate
  annotations:
    # A Sync hook in wave 1, not PreSync: the ServiceAccount, ConfigMap and Secrets (wave -1) exist first,
```

```
    # and the Deployments (wave 2) only roll after the migration succeeded.
    argocd.argoproj.io/hook: Sync
    argocd.argoproj.io/sync-wave: "1"
    argocd.argoproj.io/hook-delete-policy: BeforeHookCreation
spec:
  backoffLimit: 3
  activeDeadlineSeconds: 600
  template:
    metadata:
      labels:
        {{- include "svc.selector" (dict "root" . "component" "migrate") | nindent 8 }}
        {{- with .Values.podLabels }}
        {{- toYaml . | nindent 8 }}
        {{- end }}
    spec:
      {{- include "svc.podSpec" (dict "root" . "component" "migrate" "main" false "restartPolicy" "Never"
            "command" .Values.migrations.command "args" .Values.migrations.args
            "resources" .Values.migrations.resources) | nindent 6 }}
{{- end }}
{{- range .Values.jobs }}
---
# One-shot Job: its image is pinned in values (not the app tag), so the spec never changes and Argo CD never
# tries to mutate an immutable Job. Rename it to run it again. Never set ttlSecondsAfterFinished here:
# Argo CD would recreate the deleted Job and run it twice.
apiVersion: batch/v1
kind: Job
metadata:
  name: {{ include "svc.name" $ }}-{{ .name }}
  labels:
    {{- include "svc.selector" (dict "root" $ "component" .name) | nindent 4 }}
  annotations:
    argocd.argoproj.io/sync-wave: "3"
spec:
  backoffLimit: 2
  template:
    metadata:
      labels:
        {{- include "svc.selector" (dict "root" $ "component" .name) | nindent 8 }}
```

```
        {{- with $.Values.podLabels }}
        {{- toYaml . | nindent 8 }}
        {{- end }}
    spec:
      {{- include "svc.podSpec" (dict "root" $ "component" .name "main" false "restartPolicy" "Never"
            "image" (required "jobs[].image must be pinned" .image) "command" .command "args" .args
            "resources" (.resources | default $.Values.resources)) | nindent 6 }}
{{- end }}
__EOF__
```

#047 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/helm-charts/service/templates'
cat > 'platform/helm-charts/service/templates/service.yaml' <<'__EOF__'
{{- /* platform/helm-charts/service/templates/service.yaml */ -}}
{{- if .Values.service.enabled }}
apiVersion: v1
kind: Service
metadata:
  name: {{ include "svc.name" . }}
  labels:
    {{- include "svc.labels" . | nindent 4 }}
spec:
  type: ClusterIP
  selector:
    {{- include "svc.selector" (dict "root" . "component" "api") | nindent 4 }}
  ports:
    - name: http
      port: {{ .Values.service.port }}
      targetPort: http
    {{- if and .Values.metrics.enabled .Values.metrics.port }}
    - name: metrics
      port: {{ .Values.metrics.port }}
      targetPort: metrics
    {{- end }}
{{- end }}
__EOF__
```

#048 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/helm-charts/service/templates'
cat > 'platform/helm-charts/service/templates/ingress.yaml' <<'__EOF__'
{{- /* platform/helm-charts/service/templates/ingress.yaml */ -}}
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ include "svc.name" . }}
  labels:
    {{- include "svc.labels" . | nindent 4 }}
spec:
  ingressClassName: {{ .Values.ingress.className }}
  rules:
    - host: {{ required "ingress.host is required when ingress.enabled" .Values.ingress.host }}
      http:
        paths:
          - path: {{ .Values.ingress.path }}
            pathType: Prefix
            backend:
              service:
                name: {{ include "svc.name" . }}
                port: {name: http}
{{- end }}
__EOF__
```

#049 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/helm-charts/service/templates'
cat > 'platform/helm-charts/service/templates/serviceaccount.yaml' <<'__EOF__'
{{- /* platform/helm-charts/service/templates/serviceaccount.yaml — one identity per service, no API token by default */ -}}
{{- if .Values.serviceAccount.create }}
apiVersion: v1
kind: ServiceAccount
metadata:
  name: {{ include "svc.name" . }}
  labels:
    {{- include "svc.labels" . | nindent 4 }}
  annotations:
    argocd.argoproj.io/sync-wave: "-1"
automountServiceAccountToken: {{ .Values.serviceAccount.automountToken }}
{{- end }}
__EOF__
```

#050 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/helm-charts/service/templates'
cat > 'platform/helm-charts/service/templates/configmap.yaml' <<'__EOF__'
{{- /* platform/helm-charts/service/templates/configmap.yaml — non-secret env config (secrets come from Secret refs) */ -}}
{{- if .Values.env }}
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ include "svc.name" . }}-env
  labels:
    {{- include "svc.labels" . | nindent 4 }}
  annotations:
    argocd.argoproj.io/sync-wave: "-1"
data:
  {{- range $k, $v := .Values.env }}
  {{ $k }}: {{ $v | toString | quote }}
  {{- end }}
{{- end }}
__EOF__
```

#051 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/helm-charts/service/templates'
cat > 'platform/helm-charts/service/templates/pdb.yaml' <<'__EOF__'
{{- /* platform/helm-charts/service/templates/pdb.yaml */ -}}
{{- if .Values.pdb.enabled }}
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: {{ include "svc.name" . }}
  labels:
    {{- include "svc.labels" . | nindent 4 }}
spec:
  maxUnavailable: {{ .Values.pdb.maxUnavailable }}
  selector:
    matchLabels:
      {{- include "svc.selector" (dict "root" . "component" "api") | nindent 6 }}
{{- end }}
__EOF__
```

#052 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/helm-charts/service/templates'
cat > 'platform/helm-charts/service/templates/hpa.yaml' <<'__EOF__'
{{- /* platform/helm-charts/service/templates/hpa.yaml — optional; needs metrics-server (bundled with k3s) */ -}}
{{- if .Values.autoscaling.enabled }}
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: {{ include "svc.name" . }}
  labels:
    {{- include "svc.labels" . | nindent 4 }}
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: {{ include "svc.name" . }}
  minReplicas: {{ .Values.autoscaling.minReplicas }}
  maxReplicas: {{ .Values.autoscaling.maxReplicas }}
```

```
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: {{ .Values.autoscaling.targetCPUUtilizationPercentage }}
{{- end }}
__EOF__
```

#053 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/helm-charts/service/templates'
cat > 'platform/helm-charts/service/templates/sealedsecret-ghcr.yaml' <<'__EOF__'
{{- /* platform/helm-charts/service/templates/sealedsecret-ghcr.yaml — GHCR pull credential, decrypted only in-cluster */ -}}
{{- if .Values.imagePullSecret.create }}
apiVersion: bitnami.com/v1alpha1
kind: SealedSecret
metadata:
  name: {{ .Values.imagePullSecret.name }}
  annotations:
    sealedsecrets.bitnami.com/cluster-wide: "true"
    argocd.argoproj.io/sync-wave: "-1"
spec:
  encryptedData:
    .dockerconfigjson: {{ required "run platform/scripts/seal-ghcr-pull.sh" .Values.imagePullSecret.encryptedDockerConfigJson }}
  template:
    type: kubernetes.io/dockerconfigjson
    metadata:
      name: {{ .Values.imagePullSecret.name }}
      annotations:
        sealedsecrets.bitnami.com/cluster-wide: "true"
{{- end }}
__EOF__
```

#054 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/helm-charts/service/templates'
cat > 'platform/helm-charts/service/templates/guardrails.yaml' <<'__EOF__'
```

```
{{- /* platform/helm-charts/service/templates/guardrails.yaml — namespace budget + default-deny network, for single-service namespaces */ -}}
{{- if .Values.guardrails.enabled }}
{{- if not .Values.guardrails.ingressFromCIDRs }}{{ fail "guardrails.ingressFromCIDRs must list the ingress source CIDRs" }}{{ end }}
apiVersion: v1
kind: ResourceQuota
metadata:
  name: tenant-budget
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
spec:
  hard:
    {{- toYaml .Values.guardrails.quota | nindent 4 }}
---
apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
spec:
  limits:
    - type: Container
      defaultRequest: {cpu: 50m, memory: 64Mi}
      default: {memory: 256Mi}
      max: {memory: 2Gi}
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
```

```
metadata:
  name: allow-dns-egress
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector:
            matchLabels: {kubernetes.io/metadata.name: kube-system}
          podSelector:
            matchLabels: {k8s-app: kube-dns}
      ports:
        - {port: 53, protocol: UDP}
        - {port: 53, protocol: TCP}
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-ingress-{{ include "svc.name" . }}
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
spec:
  podSelector:
    matchLabels:
      {{- include "svc.selector" (dict "root" . "component" "api") | nindent 6 }}
  policyTypes: [Ingress]
  ingress:
    - from:
        {{- range .Values.guardrails.ingressFromCIDRs }}
        - ipBlock: {cidr: {{ . }}}
        {{- end }}
      ports:
        - {port: {{ .Values.containerPort }}, protocol: TCP}
{{- end }}
__EOF__
```

Validate: lint, render, then a **server-side dry run** (the API server validates schemas, admission and Pod Security, and nothing is created):

#055 WSL Ubuntu shell (bash) - copy & paste

```
C=platform/helm-charts/service
helm lint $C --set image.repository=ghcr.io/nikhilgs817/demo
helm template demo $C -n cka --set image.repository=ghcr.io/nikhilgs817/demo --set image.tag=abc123 \
  --set ingress.enabled=true --set ingress.host=demo.halo.localhost --set imagePullSecret.name= \
  | kubectl apply --dry-run=server -n cka -f -
```

Expected: `1 chart(s) linted, 0 chart(s) failed`, then `… created (server dry run)` for poddisruptionbudget, serviceaccount, service, deployment and ingress.

#056 WSL Ubuntu shell (bash) - copy & paste

```
git add -A && git commit -m "feat(chart): shared service chart + template Dockerfile" && git pull --rebase && git push
```

If it fails

1. `Error: … image.repository is required` → intended: every service must set it (the template's `base/values.yaml` does).
2. Dry run `forbidden: violates PodSecurity` → only if you edited the security contexts. `cka` has no PSA label, but `phoenix` and generated namespaces enforce `restricted`.
3. `nil pointer evaluating interface {}.path` → a probe key is missing in your override. Override the whole `probes.<name>` map.

- ☐ Chart lints, renders, and passes a server-side dry run

> **Helm and Kustomize together, in one paragraph.** Week 1 used Kustomize `base` / `overlays/<env>` for plain manifests, and that stays (podinfo, phoenix-data). Services use the shared **Helm chart as the Argo CD source**, and their `base` / `overlays/<env>` folders hold **value files** instead of patches: `valueFiles: [_shared/halo, <svc>/base, <svc>/overlays/halo]`, last wins. Same folder contract, same mental model, and CI promotes by setting one key. The alternative, Kustomize `helmCharts:`, needs `--enable-helm` on the repo-server and fights the load restrictor when the chart lives elsewhere in the repo.

CKA corner: the rendered output is pure CKA (Workloads 15%). Read `kubectl explain deployment.spec.strategy.rollingUpdate` and `kubectl explain pod.spec.securityContext`, then compare with `helm template … | less`.

Interview angle: "One chart owned by the platform, versioned in the monorepo, rendered for every consumer in CI before a change merges, with security defaults that already pass the policy engine we add next week."

Phoenix angle: Phoenix's four hand-written Deployments become one values file. Its worker reuses the same image, env and security context through `workers:`.

Drill W2-3 (15 min, `kcka`): rollouts, undo, scaling.

1. Change `api`'s image to `nginx:1.28` and record a change cause on the Deployment.

2. Show the rollout history, then the details of revision 2.
3. Set the image to `nginx:9.9.9` (doesn't exist) and find why the rollout is stuck.
4. Undo to the previous working revision and confirm the image.
5. Scale `api` to 5, then back to 2, in two commands.

A4: Template deploy files, generator scripts, GHCR pull secret (Sat Nov 14, block 2)

A4.1 The deploy half of the template

Goal: Copier also writes the service's Helm values, its Argo CD Application and its CI caller.

Why: "under 15 minutes" only works if a new service needs **zero hand edits** outside its own code. A service in its own namespace gets a guardrailed namespace (quota, LimitRange, default-deny) and `CreateNamespace` with `restricted` PSA labels. A service joining an existing namespace (Phoenix) gets neither, because that namespace's owner provides them.

Where: [WSL]

#057 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/platform/kubernetes/apps/[[ service_name ]]/base'
cat > 'platform/templates/fastapi-service/template/platform/kubernetes/apps/[[ service_name ]]/base/values.yaml.jinja' <<'__EOF__'
# platform/kubernetes/apps/[[ service_name ]]/base/values.yaml — environment-neutral values for the service chart.
image:
  repository: ghcr.io/[[ github_owner ]]/[[ service_name ]]
env:
  APP_SERVICE_NAME: [[ service_name ]]
  APP_LOG_LEVEL: INFO
resources:
  requests: {cpu: 50m, memory: 96Mi}
  limits: {memory: 256Mi}
__EOF__
```

#058 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/platform/kubernetes/apps/[[ service_name ]]/overlays/halo'
cat > 'platform/templates/fastapi-service/template/platform/kubernetes/apps/[[ service_name ]]/overlays/halo/values.yaml.jinja' <<'__EOF__'
# platform/kubernetes/apps/[[ service_name ]]/overlays/halo/values.yaml — cluster "halo".
# image.tag is written by CI (.github/actions/promote-tag). Humans change it only via git revert.
image:
  tag: ""
replicaCount: [[ replicas ]]
env:
  APP_ENVIRONMENT: halo
ingress:
  enabled: true
  host: [[ service_name ]].halo.localhost
guardrails:
  enabled: [[ 'true' if namespace == service_name else 'false' ]]
__EOF__
```

#059 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/platform/kubernetes/apps/[[ service_name ]]/overlays/aws'
cat > 'platform/templates/fastapi-service/template/platform/kubernetes/apps/[[ service_name ]]/overlays/aws/README.md' <<'__EOF__'
# overlays/aws — placeholder until the AWS labs. Same keys as overlays/halo/values.yaml.
__EOF__
```

#060 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/platform/kubernetes/argocd/clusters/halo'
cat > 'platform/templates/fastapi-service/template/platform/kubernetes/argocd/clusters/halo/[[ service_name ]].yaml.jinja' <<'__EOF__'
# platform/kubernetes/argocd/clusters/halo/[[ service_name ]].yaml — generated by the service template.
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: [[ service_name ]]
  namespace: argocd
  labels:
    lab.platform/env: halo
    lab.platform/tenant: [[ namespace ]]
  annotations:
    argocd.argoproj.io/sync-wave: "[[ sync_wave ]]"
```

```
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: lab
  source:
    repoURL: git@github.com:[[ github_owner ]]/ai-platform-portfolio.git
    targetRevision: main
    path: platform/helm-charts/service
    helm:
      releaseName: [[ service_name ]]
      valueFiles:                                 # relative to the chart path; later files win
        - ../../kubernetes/apps/_shared/halo/values.yaml
        - ../../kubernetes/apps/[[ service_name ]]/base/values.yaml
        - ../../kubernetes/apps/[[ service_name ]]/overlays/halo/values.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: [[ namespace ]]
  revisionHistoryLimit: 10
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - PruneLast=true
[%- if namespace == service_name %]
      - CreateNamespace=true
    managedNamespaceMetadata:
      labels:
        pod-security.kubernetes.io/enforce: restricted
        pod-security.kubernetes.io/enforce-version: latest
        lab.platform/tenant: [[ namespace ]]
        lab.platform/env: halo
[%- endif %]
    retry:
      limit: 5
      backoff: {duration: 10s, factor: 2, maxDuration: 3m}
__EOF__
```

#061 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/templates/fastapi-service/template/.github/workflows'
cat > 'platform/templates/fastapi-service/template/.github/workflows/svc-[[ service_name ]].yml.jinja' <<'__EOF__'
# .github/workflows/svc-[[ service_name ]].yml — generated caller. All logic lives in _service-ci.yml.
name: svc-[[ service_name ]]

on:
  push:
    branches: [main]
    paths:
      - projects/[[ service_name ]]/**
      - .github/workflows/svc-[[ service_name ]].yml
      - .github/workflows/_service-ci.yml
      - .github/actions/**
  pull_request:
    paths:
      - projects/[[ service_name ]]/**
      - .github/workflows/svc-[[ service_name ]].yml
      - .github/workflows/_service-ci.yml
      - .github/actions/**
  workflow_dispatch:

permissions: {}                      # nothing by default; the job below grants exactly what it needs

concurrency:
  group: svc-[[ service_name ]]-${{ github.ref }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}

jobs:
  ci:
    uses: ./.github/workflows/_service-ci.yml
    with:
      service: [[ service_name ]]
      path: projects/[[ service_name ]]
      python-version: "3.14"
      publish: ${{ github.event_name != 'pull_request' }}
    permissions:
```

```
      contents: write                # promote job commits the new tag (GITHUB_TOKEN; cannot trigger workflows)
      packages: write                # push to ghcr.io
__EOF__
```

A4.2 Generator and removal scripts

Why: Copier can't edit existing files, so the script does the two registrations Copier can't: add the Application to `clusters/halo/kustomization.yaml`, and add the namespace to the `lab` AppProject. Removal takes **two commits**: first the app disappears (its finalizer deletes its resources), then the project destination goes. Removing the destination first leaves the app unable to delete its own resources.

#062 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/scripts'
cat > 'platform/scripts/new-service.sh' <<'__EOF__'
#!/usr/bin/env bash
# platform/scripts/new-service.sh — create a paved-road service: code, chart values, Argo CD app, CI caller.
# Usage (WSL, anywhere in the repo):  platform/scripts/new-service.sh <name> [namespace]
set -euo pipefail
NAME="${1:?usage: new-service.sh <name> [namespace]}"
NS="${2:-$NAME}"
COPIER_VERSION="9.19.1"
cd "$(git rev-parse --show-toplevel)"

[[ "$NAME" =~ ^[a-z][a-z0-9-]{1,28}[a-z0-9]$ ]] || { echo "name must match ^[a-z][a-z0-9-]{1,28}[a-z0-9]$"; exit 1; }
[ ! -e "projects/$NAME" ] || { echo "projects/$NAME already exists"; exit 1; }
[ -f platform/kubernetes/apps/_shared/halo/values.yaml ] || { echo "missing apps/_shared/halo/values.yaml (Step A4)"; exit 1; }

uvx "copier@${COPIER_VERSION}" copy --quiet --defaults \
  --data "service_name=$NAME" --data "namespace=$NS" \
  --answers-file ".copier-answers/$NAME.yml" \
  platform/templates/fastapi-service .

(cd "projects/$NAME" && uv lock --quiet)

K=platform/kubernetes/argocd/clusters/halo/kustomization.yaml
yq -i ".resources += [\"$NAME.yaml\"] | .resources |= unique" "$K"
L=platform/kubernetes/argocd/projects/lab.yaml
yq -i ".spec.destinations += [{\"server\": \"https://kubernetes.default.svc\", \"namespace\": \"$NS\"}]
```

```
       | .spec.destinations |= unique_by(.namespace)" "$L"

cat <<MSG
Created service "$NAME" (namespace "$NS"):
  projects/$NAME/   platform/kubernetes/apps/$NAME/   argocd/clusters/halo/$NAME.yaml   .github/workflows/svc-$NAME.yml
Next:
  (cd projects/$NAME && make ci)
  git add -A && git commit -m "feat($NAME): new service from the paved-road template" && git push
MSG
__EOF__
chmod +x 'platform/scripts/new-service.sh'
```

#063 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/scripts'
cat > 'platform/scripts/remove-service.sh' <<'__EOF__'
#!/usr/bin/env bash
# platform/scripts/remove-service.sh — remove a template service in two safe commits.
# Usage:  remove-service.sh <name>            step 1: delete code, values, CI caller and the Argo CD app
#         remove-service.sh <name> --finalize step 2 (after Argo CD pruned it): drop the AppProject destination
set -euo pipefail
NAME="${1:?usage: remove-service.sh <name> [--finalize]}"
cd "$(git rev-parse --show-toplevel)"
NS=$(yq '.namespace // ""' ".copier-answers/$NAME.yml" 2>/dev/null || true)
NS="${NS:-$NAME}"

if [ "${2:-}" = "--finalize" ]; then
  if [ "$NS" = "$NAME" ]; then
    yq -i "del(.spec.destinations[] | select(.namespace == \"$NS\"))" platform/kubernetes/argocd/projects/lab.yaml
  fi
  rm -f ".copier-answers/$NAME.yml"
  echo "Step 2 done. Commit + push, then: kubectl delete namespace $NS   (and delete the GHCR package)"
  exit 0
fi

rm -rf "projects/$NAME" "platform/kubernetes/apps/$NAME" \
       ".github/workflows/svc-$NAME.yml" "platform/kubernetes/argocd/clusters/halo/$NAME.yaml"
yq -i "del(.resources[] | select(. == \"$NAME.yaml\"))" platform/kubernetes/argocd/clusters/halo/kustomization.yaml
```

```
echo "Step 1 done. Commit + push, wait until 'argocd app get $NAME' says not found, then run with --finalize."
__EOF__
chmod +x 'platform/scripts/remove-service.sh'
```

#064 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/scripts'
cat > 'platform/scripts/seal-secret.sh' <<'__EOF__'
#!/usr/bin/env bash
# platform/scripts/seal-secret.sh — create a namespace-scoped SealedSecret without plaintext ever touching disk.
# Usage: seal-secret.sh <namespace> <secret-name> <out.yaml> KEY[=gen|=fernet] ...
#   KEY=gen     random 48-hex-char value, printed ONCE: store it in your password manager
#   KEY=fernet  random Fernet key (urlsafe base64, 32 bytes), printed ONCE
#   KEY         prompts for the value (no echo)
set -euo pipefail
NS="${1:?}"; NAME="${2:?}"; OUT="${3:?}"; shift 3
args=()
for spec in "$@"; do
  key="${spec%%=*}"
  case "$spec" in
    *=gen)    val=$(openssl rand -hex 24);                    echo "  $NAME/$key = $val   <- password manager" >&2 ;;
    *=fernet) val=$(openssl rand -base64 32 | tr '+/' '-_');  echo "  $NAME/$key = $val   <- password manager" >&2 ;;
    *)        read -rsp "  value for $NAME/$key: " val; echo >&2 ;;
  esac
  args+=("--from-literal=$key=$val")
done
mkdir -p "$(dirname "$OUT")"
kubectl -n "$NS" create secret generic "$NAME" "${args[@]}" --dry-run=client -o yaml \
  | kubeseal --format yaml > "$OUT"
echo "wrote $OUT (namespace-scoped ciphertext; safe to commit)"
__EOF__
chmod +x 'platform/scripts/seal-secret.sh'
```

#065 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/scripts'
cat > 'platform/scripts/seal-ghcr-pull.sh' <<'__EOF__'
#!/usr/bin/env bash
# platform/scripts/seal-ghcr-pull.sh — seal the GHCR pull credential once, for every namespace (cluster-wide scope).
# Needs: kubeseal + access to the cluster (it fetches the controller's public cert). Writes ciphertext only.
set -euo pipefail
OWNER="${1:-nikhilgs817}"
cd "$(git rev-parse --show-toplevel)"
OUT=platform/kubernetes/apps/_shared/halo/values.yaml
[ -f "$OUT" ] || { echo "create $OUT first (Step A4)"; exit 1; }

read -rsp "GHCR token (classic PAT, scope read:packages only): " PAT; echo
ENC=$(kubectl create secret docker-registry ghcr-pull \
        --docker-server=ghcr.io --docker-username="$OWNER" --docker-password="$PAT" \
        --dry-run=client -o yaml \
      | kubeseal --scope cluster-wide --format yaml \
      | yq '.spec.encryptedData.".dockerconfigjson"')
unset PAT
ENC="$ENC" yq -i '.imagePullSecret.encryptedDockerConfigJson = strenv(ENC)' "$OUT"
echo "Sealed into $OUT (ciphertext only; safe to commit)."
__EOF__
chmod +x 'platform/scripts/seal-ghcr-pull.sh'
```

A4.3 The GHCR pull credential (sealed once, for every namespace)

Why: GHCR images are private. Every namespace that runs them needs an `imagePullSecret`. Sealing **one** credential with `--scope cluster-wide` and letting the chart render it in each service namespace means new services need no secret work. Use a token that can **only read packages**: it's the least damaging thing to leak.

[GITHUB] *Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token (classic)*. Note `ghcr-pull-halo`, expiration **90 days**, scope **`read:packages`** **only**. Save it in the password manager as `GHCR pull - halo`.

#066 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/apps/_shared/halo'
cat > 'platform/kubernetes/apps/_shared/halo/values.yaml' <<'__EOF__'
# platform/kubernetes/apps/_shared/halo/values.yaml — facts about cluster "halo" that every service chart needs.
# Listed first in every service Application's valueFiles, so a service can still override any of it.
imagePullSecret:
  create: true
  name: ghcr-pull
  encryptedDockerConfigJson: ""      # filled by platform/scripts/seal-ghcr-pull.sh (ciphertext; safe to commit)
guardrails:
  ingressFromCIDRs:                  # Traefik runs hostNetwork: traffic arrives from the node, via cni0
    - 10.255.255.10/32               #   node IP (halo0)
    - 10.42.0.1/32                   #   cni0 gateway of the node's pod subnet
__EOF__
```

#067 WSL Ubuntu shell (bash) - copy & paste

```
platform/scripts/seal-ghcr-pull.sh nikhilgs817          # paste the token at the prompt (no echo)
yq '.imagePullSecret.encryptedDockerConfigJson | length' platform/kubernetes/apps/_shared/halo/values.yaml
git add -A && git commit -m "feat(platform): generator scripts, sealed GHCR pull credential" && git pull --rebase && git push
```

Expected: `Sealed into …values.yaml`, then a length around 600–900. The file holds ciphertext only.

Verify (dry): `platform/scripts/new-service.sh demo-check`, then `git status --short` shows the new files plus two modified registries. Throw it away with `git checkout -- . && git clean -fd projects/demo-check platform/kubernetes/apps/demo-check .copier-answers .github/workflows/svc-demo-check.yml platform/kubernetes/argocd/clusters/halo/demo-check.yaml`.

If it fails

1. `missing apps/_shared/halo/values.yaml` → create it (above) before sealing.
2. `yq: … strenv` errors → wrong yq (the Python `yq` package). `which yq` must be `/usr/local/bin/yq` (mikefarah v4).
3. `kubeseal: error: cannot get sealed secret service` → Sealed Secrets isn't running (A1.2).

- ☐ Template deploy files committed · ☐ scripts executable · ☐ GHCR credential sealed, token in the password manager

CKA corner: `kubectl create secret docker-registry … --dry-run=client -o yaml` is a classic exam one-liner. Know where `imagePullSecrets` goes (pod spec, or the ServiceAccount).

Interview angle: "Onboarding is one command: code, CI, Helm values, Argo CD app and namespace guardrails in one reviewed commit, with no secrets to hand out."

Phoenix angle: the Command Center pulls its private image with the same sealed credential, so there's no Phoenix-specific secret plumbing for images.

Drill W2-4 (15 min, `kcka`): explain, events, logs, exec.

1. With `kubectl explain`, find the default `maxSurge` for a Deployment rolling update.
2. Show only *Warning* events in `cka`, newest last.
3. Create pod `crash` (`busybox:1.37.0`, command `sh -c 'echo boom; exit 1'`) and read the logs of the **previous** container instance.
4. Print one environment variable (`HOSTNAME`) from inside `web` (recreate `web` if needed).
5. Create pod `huge` requesting 500 CPUs and find the exact scheduling failure message.

A5: GitHub Actions for the monorepo (Sun Nov 15, block 1)

Goal: the reusable workflow, four composite actions, repository guards and platform checks, all green.

Why (monorepo patterns):

- **One reusable workflow, many thin callers.** Each service has a 30-line caller with its own `paths:` filter, so a push to `projects/hello-platform/**` runs only that service. A change to `_service-ci.yml` or `.github/actions/**` runs *every* service (intended: platform changes are tested everywhere).
- **Composite actions** hold the repeated steps (verify, build-push, promote, secret scan). The reusable workflow holds the job graph and the permissions.
- **Least privilege.** `permissions: {}` at the top of every workflow. Each job asks for exactly what it needs: `contents: read` to test, `packages: write` to push, `contents: write` only in `promote`.
- **Concurrency.** PR runs cancel superseded runs. `main` runs queue. `promote-halo` serialises promotions and never cancels one.
- **Immutable tags.** `ghcr.io/nikhilgs817/<svc>:<full git sha>`, never `latest`.
- **No CI loops.** Three independent guards: pushes made with `GITHUB_TOKEN` never trigger workflows; caller filters exclude `platform/kubernetes/**`; the message carries `[skip ci]`. Promotion also refuses to go *backwards*: an older build that finishes last won't overwrite a newer tag.
- **Supply chain.** Every third-party action is pinned to a commit SHA. gitleaks scans the full history on every push. **Where:** [WSL], then [GITHUB]

#068 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p '.github/workflows'
cat > '.github/workflows/_service-ci.yml' <<'__EOF__'
# .github/workflows/_service-ci.yml — the reusable pipeline every service calls: verify -> image -> promote.
# Callers: .github/workflows/svc-<service>.yml (generated by the template). Shared steps: .github/actions/*.
name: service-ci (reusable)

on:
  workflow_call:
    inputs:
```

```
      service:
        description: Service name = image name = platform/kubernetes/apps/<service>
        type: string
        required: true
      path:
        description: Build context and `make ci` directory, e.g. projects/<service>
        type: string
        required: true
      python-version:
        type: string
        default: "3.14"
      publish:
        description: Push the image and promote the tag (false on pull requests)
        type: boolean
        default: true
      env:
        description: Overlay that receives the new tag
        type: string
        default: halo
    outputs:
      image:
        value: ${{ jobs.image.outputs.image }}
      tag:
        value: ${{ jobs.image.outputs.tag }}
      digest:
        value: ${{ jobs.image.outputs.digest }}

permissions: {}

jobs:
  verify:
    name: verify (make ci)
    runs-on: ubuntu-24.04
    timeout-minutes: 20
    permissions:
      contents: read
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
```

```
      with:
        persist-credentials: false
    - uses: ./.github/actions/python-verify
      with:
        path: ${{ inputs.path }}
        python-version: ${{ inputs.python-version }}

image:
  name: image
  needs: verify
  runs-on: ubuntu-24.04
  timeout-minutes: 20
  permissions:
    contents: read
    packages: write
  outputs:
    image: ${{ steps.bp.outputs.image }}
    tag: ${{ steps.bp.outputs.tag }}
    digest: ${{ steps.bp.outputs.digest }}
  steps:
    - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
      with:
        persist-credentials: false
    - id: bp
      uses: ./.github/actions/build-push
      with:
        service: ${{ inputs.service }}
        path: ${{ inputs.path }}
        push: ${{ inputs.publish }}

promote:
  name: promote -> ${{ inputs.env }}
  if: inputs.publish && github.ref == 'refs/heads/main'
  needs: image
  runs-on: ubuntu-24.04
  timeout-minutes: 10
  permissions:
    contents: write
```

```
    concurrency:                          # one promotion at a time per environment, never cancelled
      group: promote-${{ inputs.env }}
      cancel-in-progress: false
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          ref: main
          fetch-depth: 0                  # promote-tag compares commits to never deploy an older build
      - uses: ./.github/actions/promote-tag
        with:
          service: ${{ inputs.service }}
          env: ${{ inputs.env }}
          tag: ${{ needs.image.outputs.tag }}
__EOF__
```

#069 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p '.github/actions/python-verify'
cat > '.github/actions/python-verify/action.yml' <<'__EOF__'
# .github/actions/python-verify/action.yml — uv + Python, then the service's own `make ci`.
name: python-verify
description: Install uv and Python, then run `make ci` (the only contract between CI and a service)
inputs:
  path:
    description: Service directory
    required: true
  python-version:
    description: Python version for uv to provide
    required: true
runs:
  using: composite
  steps:
    - uses: astral-sh/setup-uv@68559e99cc807c1eb7e9238fd43a15bba460eec2 # v10.3.3
      with:
        version: "0.13.6"
        python-version: ${{ inputs.python-version }}
        enable-cache: true
        cache-dependency-glob: |
```

```
          ${{ inputs.path }}/uv.lock
          ${{ inputs.path }}/pyproject.toml
    - name: make ci
      shell: bash
      working-directory: ${{ inputs.path }}
      run: make ci
__EOF__
```

#070 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p '.github/actions/build-push'
cat > '.github/actions/build-push/action.yml' <<'__EOF__'
# .github/actions/build-push/action.yml — build with BuildKit layer cache; push ghcr.io/<owner>/<service>:<git sha>.
name: build-push
description: Build the service image (cached per service) and push it to GHCR tagged with the full git SHA
inputs:
  service:
    description: Image name
    required: true
  path:
    description: Docker build context
    required: true
  push:
    description: "'true' to log in and push"
    default: "true"
outputs:
  image:
    description: Image repository
    value: ${{ steps.ref.outputs.image }}
  tag:
    description: Immutable tag (the full git SHA)
    value: ${{ steps.ref.outputs.tag }}
  digest:
    description: Image digest (empty when not pushed)
    value: ${{ steps.build.outputs.digest }}
runs:
  using: composite
  steps:
```

```
    - id: ref
      shell: bash
      env:
        SERVICE: ${{ inputs.service }}
        OWNER: ${{ github.repository_owner }}
      run: |
        echo "image=ghcr.io/${OWNER,,}/${SERVICE}" >> "$GITHUB_OUTPUT"
        echo "tag=${GITHUB_SHA}" >> "$GITHUB_OUTPUT"
    - uses: docker/setup-buildx-action@f87e5991a6d7451dcb8d9637bfbc97413f497069 # v4.4.1
    - if: inputs.push == 'true'
      uses: docker/login-action@81c6f93d4bc991e510b342a00b419b3bb4efc5c6 # v4.7.0
      with:
        registry: ghcr.io
        username: ${{ github.actor }}
        password: ${{ github.token }}
    - id: meta
      uses: docker/metadata-action@dc802804100637a589fabce1cb79ff13a1411302 # v6.2.0
      with:
        images: ${{ steps.ref.outputs.image }}
        tags: type=raw,value=${{ steps.ref.outputs.tag }}
        labels: org.opencontainers.image.title=${{ inputs.service }}
    - id: build
      uses: docker/build-push-action@c3c9e263c25d99ce0380d002d59b67737d91b0dc # v7.4.0
      with:
        context: ${{ inputs.path }}
        push: ${{ inputs.push == 'true' }}
        tags: ${{ steps.meta.outputs.tags }}
        labels: ${{ steps.meta.outputs.labels }}
        build-args: APP_VERSION=${{ steps.ref.outputs.tag }}
        cache-from: type=gha,scope=${{ inputs.service }}
        cache-to: type=gha,mode=max,scope=${{ inputs.service }}
        provenance: false                # Week 4 adds SBOM (Syft) and signatures (cosign)
__EOF__
```

#071 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p '.github/actions/promote-tag'
cat > '.github/actions/promote-tag/action.yml' <<'__EOF__'
```

```
# .github/actions/promote-tag/action.yml — GitOps promotion: commit the new tag into overlays/<env>/values.yaml.
# Loop-safe: pushes made with GITHUB_TOKEN never start workflows, the callers' path filters exclude
# platform/kubernetes/**, and the message carries [skip ci].
name: promote-tag
description: Write image.tag for one service and environment, commit and push to main (retries on races)
inputs:
  service:
    description: Service name
    required: true
  env:
    description: Environment overlay (halo, aws)
    required: true
  tag:
    description: Image tag (git SHA) to deploy
    required: true
runs:
  using: composite
  steps:
    - shell: bash
      env:
        SERVICE: ${{ inputs.service }}
        ENV_NAME: ${{ inputs.env }}
        TAG: ${{ inputs.tag }}
      run: |
        set -euo pipefail
        file="platform/kubernetes/apps/${SERVICE}/overlays/${ENV_NAME}/values.yaml"
        git config user.name "github-actions[bot]"
        git config user.email "41898282+github-actions[bot]@users.noreply.github.com"
        for attempt in 1 2 3 4 5; do
          git fetch --quiet origin main
          git reset --quiet --hard origin/main
          current=$(yq '.image.tag // ""' "$file")
          if [ "$current" = "$TAG" ]; then echo "already at ${TAG:0:12}"; exit 0; fi
          if [ -n "$current" ] && git merge-base --is-ancestor "$TAG" "$current" 2>/dev/null; then
            echo "newer build ${current:0:12} already deployed; not going backwards"; exit 0
          fi
          TAG="$TAG" yq -i '.image.tag = strenv(TAG)' "$file"
          git commit --quiet -am "deploy(${SERVICE}/${ENV_NAME}): ${TAG:0:12} [skip ci]"
```

```
          if git push --quiet origin HEAD:main; then echo "promoted ${SERVICE} -> ${TAG:0:12}"; exit 0; fi
          echo "push raced with another commit; retry ${attempt}"; sleep $((attempt * 3))
        done
        echo "::error::could not promote ${SERVICE} after 5 attempts"; exit 1
__EOF__
```

#072 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p '.github/actions/secret-scan'
cat > '.github/actions/secret-scan/action.yml' <<'__EOF__'
# .github/actions/secret-scan/action.yml — gitleaks (pinned, checksum-verified) over the full git history.
name: secret-scan
description: Fail if any secret appears anywhere in the git history
runs:
  using: composite
  steps:
    - shell: bash
      env:
        GITLEAKS_VERSION: "8.30.1"
      run: |
        set -euo pipefail
        base="https://github.com/gitleaks/gitleaks/releases/download/v${GITLEAKS_VERSION}"
        tgz="gitleaks_${GITLEAKS_VERSION}_linux_x64.tar.gz"
        cd "$RUNNER_TEMP"
        curl -fsSLO "$base/$tgz"
        curl -fsSLO "$base/gitleaks_${GITLEAKS_VERSION}_checksums.txt"
        grep " ${tgz}\$" "gitleaks_${GITLEAKS_VERSION}_checksums.txt" | sha256sum --check --quiet
        tar xzf "$tgz" gitleaks
        cd "$GITHUB_WORKSPACE"
        "$RUNNER_TEMP/gitleaks" git --redact --no-banner --exit-code 1 .
__EOF__
```

#073 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p '.github/workflows'
cat > '.github/workflows/repo-guards.yml' <<'__EOF__'
# .github/workflows/repo-guards.yml — repository-wide rules that run on every push and pull request.
name: repo-guards
```

```
on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:

permissions: {}

concurrency:
  group: repo-guards-${{ github.ref }}
  cancel-in-progress: true

jobs:
  no-secrets:
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    permissions:
      contents: read
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          fetch-depth: 0
          persist-credentials: false
      - uses: ./.github/actions/secret-scan
      - name: No env files, databases, broker exports or Terraform state are tracked
        run: |
          if git ls-files | grep -E '(^|/)\.env$|(^|/)service\.env$|\.db$|\.sqlite$|(^|/)exports/|\.tfstate'; then
            echo "::error::these files must never be committed"; exit 1
          fi
      - name: Every Secret manifest in Git is a SealedSecret
        run: |
          bad=$(grep -rlE '^kind: Secret[[:space:]]*$' platform/kubernetes || true)
          if [ -n "$bad" ]; then echo "::error::plain Secret manifests found: $bad"; exit 1; fi
__EOF__
```

#074 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p '.github/workflows'
cat > '.github/workflows/platform-checks.yml' <<'__EOF__'
# .github/workflows/platform-checks.yml — a chart or GitOps change is rendered for EVERY consumer before it merges.
name: platform-checks

on:
  push:
    branches: [main]
    paths: [platform/helm-charts/**, platform/kubernetes/**, .github/workflows/platform-checks.yml]
  pull_request:
    paths: [platform/helm-charts/**, platform/kubernetes/**, .github/workflows/platform-checks.yml]

permissions: {}

concurrency:
  group: platform-checks-${{ github.ref }}
  cancel-in-progress: true

jobs:
  render:
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    permissions:
      contents: read
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          persist-credentials: false
      - uses: azure/setup-helm@9bc31f4ebc9c6b171d7bfbaa5d006ae7abdb4310 # v5.0.1
        with:
          version: v4.3.1
      - name: helm lint the shared chart
        run: helm lint platform/helm-charts/service --set image.repository=ghcr.io/example/lint
      - name: Render every Application that uses the shared chart
        run: |
          set -euo pipefail
          for app in platform/kubernetes/argocd/clusters/*/*.yaml; do
```

```
            [ "$(yq '.kind' "$app")" = "Application" ] || continue
            path=$(yq '.spec.source.path // ""' "$app")
            [ "$path" = "platform/helm-charts/service" ] || continue
            name=$(yq '.spec.source.helm.releaseName' "$app")
            ns=$(yq '.spec.destination.namespace' "$app")
            args=()
            while IFS= read -r vf; do args+=(-f "$path/$vf"); done < <(yq '.spec.source.helm.valueFiles[]' "$app")
            helm template "$name" "$path" -n "$ns" "${args[@]}" --set image.tag=ci-render > /dev/null
            echo "rendered $name ($app)"
          done
      - name: Build every Kustomization
        run: |
          set -euo pipefail
          find platform/kubernetes -name kustomization.yaml -print0 | while IFS= read -r -d '' k; do
            kubectl kustomize "$(dirname "$k")" > /dev/null && echo "built $(dirname "$k")"
          done
__EOF__
```

#075 WSL Ubuntu shell (bash) - copy & paste

```
cat > '.gitleaks.toml' <<'__EOF__'
# .gitleaks.toml — default rules plus a narrow allowlist for known, reviewed false positives.
[extend]
useDefault = true

[allowlist]
description = "Alpaca pagination cursors in test fixtures (not secrets)"
paths = ['''projects/command-center/tests/slice/fixtures/.*\.json''']
__EOF__
```

#076 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'docs/adrs'
cat > 'docs/adrs/0002-paved-road-service-template.md' <<'__EOF__'
# ADR-0002: Paved road for services — Copier template, shared Helm chart, GitHub Actions, Sealed Secrets

- **Status:** Accepted · **Date:** 2026-11-12

## Context
```

```
Every lab adds services (Phoenix, LLM gateway, RAG, MCP tools). Each must build, ship and run the same way, from one
monorepo, with no plaintext secrets in a repository that becomes public in January 2027. CI is GitHub Actions only.

## Decision
1. **Generator: Copier**, template in `platform/templates/fastapi-service`, run via `platform/scripts/new-service.sh`.
   It records answers per service, so `copier update` can roll template changes out later.
2. **One shared Helm chart** `platform/helm-charts/service` (Deployment, Service, Ingress, ServiceAccount, ConfigMap,
   PDB, HPA, workers, migration hook, one-shot jobs, optional namespace guardrails). Argo CD renders it with layered
   value files: `_shared/<env>` -> `apps/<svc>/base` -> `apps/<svc>/overlays/<env>`.
3. **CI: one reusable workflow** `_service-ci.yml` (verify -> image -> promote) plus composite actions; each service has
   a thin caller with path filters. The contract with a service is `make ci` and a Dockerfile.
4. **Images: GHCR**, private, tag = full git SHA, never `latest`. Harbor replaces GHCR in Week 4.
5. **Promotion: CI commits the tag** into `overlays/<env>/values.yaml` (not Argo CD Image Updater).
6. **Secrets: Sealed Secrets** (controller 0.41.2). Ciphertext in Git; the sealing key in the password manager.

## Alternatives
| Option | Why not now |
|---|---|
| cookiecutter | No update path for existing services; Copier has `copier update` and answer files |
| Plain copy script | Fast to write, but drift between services starts on day 2 |
| Kustomize `helmCharts` | Needs `--enable-helm` in Argo CD and a local chart path outside the kustomization root |
| Argo CD Image Updater (v1.x, CRD-based) | Needs registry and Git write credentials in the cluster, polls, and hides promotion from the PR flow |
| SOPS + age | Great for Terraform later; with Argo CD it needs a repo-server plugin (KSOPS/helm-secrets) to decrypt |

## Consequences
- A new service runs through Argo CD in under 15 minutes; the chart and workflow are fixed in one place for all.
- A chart change affects every service at once: `platform-checks.yml` renders all consumers on every chart PR.
- Losing the sealing key means re-sealing every secret (runbook `bootstrap-secrets.md`).
- The bot commits to `main`; requiring PRs on `main` later means switching promotion to a GitHub App or PR-based flow.
__EOF__
```

> **CI commit vs Argo CD Image Updater.** Image Updater (v1.x, `ImageUpdater` CRD) watches the registry from inside the cluster and writes tags back to Git or to the Application. It needs registry credentials *and* Git write credentials in the cluster, it polls, and the deploy shows up as a bot commit you didn't see coming. **CI committing the tag** is explicit: the same run that built and tested the image promotes it, and the commit is an audit record. Promoting to a second environment later is a second, PR-gated step. Recommendation: **CI commit now**; revisit Image Updater when `aws` needs "latest patch of a semver range".

[GITHUB] repo *Settings → Actions → General*: **Workflow permissions = "Read repository contents and packages permissions"** (the default; jobs request more explicitly). Leave "Allow GitHub Actions to create and approve pull requests" off.

#077 WSL Ubuntu shell (bash) - copy & paste

```
git add -A && git commit -m "ci: reusable service workflow, composite actions, repo guards, platform checks" && git pull --rebase && git push
```

Expected: [GITHUB] *Actions* shows `repo-guards` ✓ (secret scan, forbidden files, no plain Secrets) and `platform-checks` ✓ (chart lint, kustomize builds; no services to render yet).

Verify: `gh` isn't needed. In the browser, open the `repo-guards` run → `no-secrets` → the gitleaks step ends with `no leaks found`.

If it fails

1. `platform-checks` fails building `phoenix-data` → expected only before Part B (it doesn't exist yet; this run shouldn't see it). Otherwise run `kubectl kustomize <dir>` locally.
2. gitleaks flags a real-looking string → if it's a false positive, add a narrow `paths` or regex to `.gitleaks.toml`. **If it's real, rotate the credential first**, then rewrite history with `git filter-repo --replace-text`.
3. `Unexpected input(s) 'python-version'` from setup-uv → the action's inputs changed. Check its README at the pinned tag and adjust `python-verify`.

- ☐ Both workflows green · ☐ Actions default permissions are read-only

CKA corner: none directly, but the `--dry-run=server` habit from A3 is how you'd validate on the exam too.

Interview angle: "A monorepo pipeline built from one reusable workflow and composite actions, with per-service path filters, least-privilege tokens, SHA-pinned actions, immutable tags and loop-proof GitOps promotion. It's the GitHub Actions paved road I'd propose as the path off Jenkins."

Phoenix angle: Phoenix keeps its Windows and PostgreSQL test jobs as extra jobs next to the shared pipeline (B1), so nothing it tests today is lost.

Drill W2-5 (15 min, `kcka`): `--dry-run=client -o yaml`.

1. Generate a Deployment `shop` (`nginx:1.29`, 2 replicas, port 80) into `/tmp/shop.yaml` without creating it.
2. Add requests `cpu=50m,memory=32Mi` to it *locally* (no API call), then save.
3. Generate a ClusterIP Service for it on port 80 into the same file (`---` between documents).
4. Generate a Job `once` (`busybox:1.37.0`, `echo done`) into `/tmp/once.yaml`.
5. Apply both files in one command and show what was created.

A6: Prove it: `hello-platform` end to end in under 15 minutes, then delete it (Sun Nov 15, block 2)

Goal: generate → push → CI builds → tag promoted → Argo CD syncs → reachable from Windows, timed.

Why: this is the checkpoint a platform team demos: the time from idea to running service, with every gate on.

Where: [WSL], [GITHUB], [BROWSER]

#078 WSL Ubuntu shell (bash) - copy & paste

```
START=$(date +%s)
platform/scripts/new-service.sh hello-platform
(cd projects/hello-platform && make ci)
git add -A && git commit -m "feat(hello-platform): new service from the paved-road template" && git pull --rebase && git push
```

[GITHUB] *Actions → svc-hello-platform*: `verify` → `image` → `promote -> halo`. In the promote log: `promoted hello-platform -> <12 chars>`.

#079 WSL Ubuntu shell (bash) - copy & paste

```
# Argo CD sees the Application immediately (replicas 0: no image yet), then the promotion commit:
argocd app get root-halo --refresh >/dev/null
until [ -n "$(git fetch -q && git show origin/main:platform/kubernetes/apps/hello-platform/overlays/halo/values.yaml | yq '.image.tag')" ]; do sleep 15; done
git pull --rebase
argocd app get hello-platform --refresh >/dev/null
argocd app wait hello-platform --health --timeout 600
curl -s -H 'Host: hello-platform.halo.localhost' http://127.0.0.1/api/v1/info; echo
echo "minutes from generate to running: $(( ($(date +%s) - START) / 60 ))"
```

Expected: `{"service":"hello-platform","version":"<git sha>","environment":"halo"}` and **< 15 minutes** (typically 8–12; the first CI build is the slowest). [BROWSER] http://hello-platform.halo.localhost/docs shows the OpenAPI page.

Verify the guardrails and the image:

#080 WSL Ubuntu shell (bash) - copy & paste

```
kubectl get ns hello-platform --show-labels | grep -o 'enforce=restricted'
kubectl -n hello-platform get quota,limitrange,networkpolicy,pdb,sealedsecret,secret ghcr-pull
kubectl -n hello-platform get deploy hello-platform -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
git log --oneline -3      # your commit + "deploy(hello-platform/halo): <sha> [skip ci]"
```

[GITHUB] *Actions*: the promotion commit started **no** new workflow run (that's the loop guard working).

Delete it cleanly (two commits):

#081 WSL Ubuntu shell (bash) - copy & paste

```
platform/scripts/remove-service.sh hello-platform
git add -A && git commit -m "chore(hello-platform): remove demo service" && git pull --rebase && git push
argocd app get root-halo --refresh >/dev/null
until ! argocd app get hello-platform >/dev/null 2>&1; do sleep 5; done; echo "app gone"
platform/scripts/remove-service.sh hello-platform --finalize
git add -A && git commit -m "chore(hello-platform): drop project destination" && git pull --rebase && git push
kubectl delete namespace hello-platform
```

[GITHUB] *Your profile → Packages → hello-platform → Package settings → Delete this package.*

Expected: no `hello-platform` in `argocd app list`, in `kubectl get ns`, or under `projects/`.

If it fails

1. Pods `ImagePullBackOff` with `401 Unauthorized` → the PAT lacks `read:packages`, was revoked, or belongs to another account. Re-seal (A4.3). Appendix T1.
2. The workflow didn't start → the push didn't touch the caller's paths, or the caller file isn't on `main` yet. Appendix T2.
3. `promote` fails with `rejected … fetch first` five times → many pushes at once. Re-run the job; the retry loop normally absorbs this. Appendix T3.

- ☐ **Template creates a service that CI builds and Argo CD deploys**, timed < 15 min · ☐ removed without leftovers

CKA corner: `kubectl get events -n hello-platform --sort-by=.lastTimestamp` during the first sync shows the PDB, quota admission and image pull sequence. ResourceQuota is on the exam.

Interview angle: "From `new-service.sh` to a running, policy-compliant, observable service in about ten minutes, without a ticket or a credential handed out."

Phoenix angle: the next service you add for Phoenix (for example the L2 Morning Brief API) is this exact flow with `namespace=phoenix`.

Drill W2-6 (15 min, any namespace, read-only): jsonpath and custom-columns.

1. Every pod name in every namespace, one per line (jsonpath `range`).
2. For `kube-system`: columns `NAME`, `IMAGE`, `NODE` with custom-columns.
3. The node's `InternalIP` using a jsonpath filter expression.
4. All pods in the cluster sorted by restart count of their first container.
5. Every distinct container image running in the cluster, sorted and unique.

Part B: Phoenix Command Center as the first tenant (Nov 16–22)

Outcome: the Command Center runs in namespace `phoenix` through the template's chart and CI, on TimescaleDB and Redis that survive pod kills and WSL restarts, behind default-deny networking, with nightly backups and a tested restore. Synthetic demo data only, and no broker or order credentials in the cluster.

B1: Import the code (with history) and fit it to the template (Mon Nov 16, block 1)

B1.1 Merge the fit, review for personal data, import with history

Goal: `projects/command-center/` holds Phoenix 2.1 with its git history; `deploy/` and the repo-local `.github/` are gone (the platform replaces them).

Why history, and why filter-repo: the history is short (the 2.1.0 import plus the reviewed fit in PR #3), but it's the provenance of every change, and it costs nothing. `git filter-repo --to-subdirectory-filter` rewrites every commit as if the code had always lived in `projects/command-center/`, so `git log` and `git blame` work on the new paths with no `--follow` tricks. `git subtree add` would keep the old paths in old commits. Starting fresh would lose the review trail.

Where: [GITHUB], then [WSL]

1. [GITHUB] Open nikhilgs817/PHOENIX_TRADER#3, mark it *Ready for review*, and **merge** it.
2. **Publication check.** `ai-platform-portfolio` goes public in January, and Phoenix travels with it:

#082 WSL Ubuntu shell (bash) - copy & paste

```
cd /tmp && rm -rf phoenix-import && git clone -q git@github.com:nikhilgs817/PHOENIX_TRADER.git phoenix-import && cd phoenix-import
git log --oneline
grep -rnoE '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[a-z]{2,}' --include=*.py --include=*.md --include=*.yaml . | grep -v example | head
grep -n 'lesson' config/policy.yaml | head -5          # lessons "from your own record": personal, not secret
```

Expected: the 2 commits (import, fit), no personal e-mail addresses. **Decide before January** whether the `lesson` texts in `config/policy.yaml` and the PDFs in `docs/` should be public. If not, add a scrub step to the January publication checklist (What's next).

3. Rewrite and merge:

#083 WSL Ubuntu shell (bash) - copy & paste

```
cd /tmp/phoenix-import
git filter-repo --force --invert-paths --path deploy/ --path .github/
git filter-repo --force --to-subdirectory-filter projects/command-center
git log --oneline --stat | head -12                    # paths now start with projects/command-center/
cd ~/src/ai-platform-portfolio && git pull --rebase
git fetch /tmp/phoenix-import main:phoenix-import
git merge --allow-unrelated-histories -m "feat(command-center): import Phoenix Command Center 2.1 with history" phoenix-import
git branch -D phoenix-import
git log --oneline -- projects/command-center | head -5
```

Expected: the merge commit, then the two original commits rewritten under `projects/command-center/`. `ls projects/command-center` shows `src/ config/ tests/ Dockerfile Makefile pyproject.toml PhoenixFidelitySync/ …`, and no `deploy/` or `.github/`.

B1.2 What changed in the code (all in PR #3, already merged)

Change	Why the platform needs it	Size
`GET /livez` (no DB call) and `GET /readyz` (DB check) in `web/routes/system.py`; both in `PUBLIC_PATHS`	Liveness must not restart pods on a database blip; probes arrive with the pod IP as Host, so they must bypass the host allowlist	~15 lines
`core/logjson.py` + `PHOENIX_LOG_FORMAT=json` wired into `cli/main.py` (`setup_logging()`, uvicorn `log_config`)	Structured stdout logs for Loki later; Windows stays plain text	~60 lines
`phoenix demo --url postgresql://…` converts the URL (`to_sqlalchemy_url`)	Seed synthetic data into the cluster database	2 lines
`Makefile` with `make ci`	The reusable workflow's only contract	new file
`Dockerfile`: base pinned by tag + digest	Reproducible builds; Dependabot bumps	3 lines
`tests/test_probes.py`	Probes answer on a pod-IP Host; JSON log format	new file

Nothing else changes. Config is already env-only, the process already runs as uid 10001, and a read-only root works (`HOME=/tmp`).

B1.3 CI caller, Dependabot, chart values

Why: promotion writes into `apps/command-center/overlays/halo/values.yaml`, so that file must exist before the first CI run. Phoenix keeps its extra test jobs (PostgreSQL, Windows, the Fidelity helper) next to the shared pipeline. The PostgreSQL job now uses the **same TimescaleDB image** as the cluster instead of `postgres:16`.

#084 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p '.github/workflows'
cat > '.github/workflows/svc-command-center.yml' <<'__EOF__'
# .github/workflows/svc-command-center.yml — Phoenix Command Center: the shared pipeline + Phoenix-only test jobs.
name: svc-command-center

on:
  push:
    branches: [main]
    paths:
      - projects/command-center/**
      - .github/workflows/svc-command-center.yml
      - .github/workflows/_service-ci.yml
      - .github/actions/**
  pull_request:
    paths:
      - projects/command-center/**
      - .github/workflows/svc-command-center.yml
      - .github/workflows/_service-ci.yml
      - .github/actions/**
  workflow_dispatch:

permissions: {}

concurrency:
  group: svc-command-center-${{ github.ref }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}

jobs:
  ci:
    uses: ./.github/workflows/_service-ci.yml
    with:
      service: command-center
      path: projects/command-center
      python-version: "3.12"                    # what Phoenix supports today (3.11-3.12 tested)
      publish: ${{ github.event_name != 'pull_request' }}
    permissions:
      contents: write
```

```
    packages: write

postgres-tests:                              # the PostgreSQL tests, against the same engine as production
  runs-on: ubuntu-24.04
  timeout-minutes: 15
  permissions:
    contents: read
  services:
    db:
      image: timescale/timescaledb:2.31.0-pg18@sha256:0fcff5761363809d1242f2261a3482197b5a5337407f8b97a976fb174206d579
      env:
        POSTGRES_PASSWORD: postgres          # throwaway CI database, not a secret
        POSTGRES_DB: phoenix_test
      ports: ["5432:5432"]
      options: >-
        --health-cmd "pg_isready -U postgres" --health-interval 5s --health-timeout 5s --health-retries 12
  env:
    PHOENIX_TEST_PG_URL: postgresql://postgres:postgres@localhost:5432/phoenix_test
  defaults:
    run:
      working-directory: projects/command-center
  steps:
    - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
      with:
        persist-credentials: false
    - uses: astral-sh/setup-uv@68559e99cc807c1eb7e9238fd43a15bba460eec2 # v10.3.3
      with:
        version: "0.13.6"
        python-version: "3.12"
    - run: make install
    - run: .venv/bin/python -m pytest tests/test_web_db.py -rs

windows-tests:                               # Phoenix also runs natively on the Windows PC
  runs-on: windows-2025
  timeout-minutes: 20
  permissions:
    contents: read
  defaults:
```

```
      run:
        working-directory: projects/command-center
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          persist-credentials: false
      - uses: astral-sh/setup-uv@68559e99cc807c1eb7e9238fd43a15bba460eec2 # v10.3.3
        with:
          version: "0.13.6"
          python-version: "3.12"
      - run: uv venv .venv
      - run: uv pip install -p .venv -e ".[dev]"
      - run: .venv\Scripts\python -m pytest

  fidelity-helper:
    runs-on: windows-2025
    timeout-minutes: 15
    permissions:
      contents: read
    defaults:
      run:
        working-directory: projects/command-center/PhoenixFidelitySync
    steps:
      - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
        with:
          persist-credentials: false
      - uses: astral-sh/setup-uv@68559e99cc807c1eb7e9238fd43a15bba460eec2 # v10.3.3
        with:
          version: "0.13.6"
          python-version: "3.12"
      - run: uv venv .venv
      - run: uv pip install -p .venv -r requirements.txt pytest
      - run: .venv\Scripts\python -m pytest test_fidelity_sync.py -q
__EOF__
```

#085 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p '.github'
cat > '.github/dependabot.yml' <<'__EOF__'
# .github/dependabot.yml — weekly, grouped update PRs for the whole monorepo.
version: 2
updates:
  - package-ecosystem: github-actions
    directory: /
    schedule: {interval: weekly}
    groups:
      actions: {patterns: ["*"]}
  - package-ecosystem: uv                       # template services (they have uv.lock)
    directories: ["/projects/*"]
    schedule: {interval: weekly}
    groups:
      python: {patterns: ["*"]}
  - package-ecosystem: pip                      # Phoenix (pyproject + setuptools)
    directory: /projects/command-center
    schedule: {interval: monthly}
    groups:
      python: {patterns: ["*"]}
  - package-ecosystem: docker                   # base images pinned by digest in every Dockerfile
    directories: ["/projects/*"]
    schedule: {interval: weekly}
__EOF__
```

#086 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/apps/command-center/base'
cat > 'platform/kubernetes/apps/command-center/base/values.yaml' <<'__EOF__'
# platform/kubernetes/apps/command-center/base/values.yaml — Phoenix Command Center on the paved-road chart.
# One image, run several ways (the image ENTRYPOINT is `phoenix`): serve = this Deployment, worker = a worker.
nameOverride: command-center
image:
  repository: ghcr.io/nikhilgs817/command-center
args: [serve, --host, 0.0.0.0, --port, "8000", --metrics-port, "9100"]
containerPort: 8000
metrics:
  enabled: true
```

```
  port: 9100                      # Phoenix serves /metrics on its own port, outside the page login
  path: /metrics
probes:
  startup: {path: /livez, periodSeconds: 3, failureThreshold: 40}
  liveness: {path: /livez, periodSeconds: 15, failureThreshold: 4}
  readiness: {path: /readyz, periodSeconds: 10, failureThreshold: 3}
env:
  PHOENIX_LOG_FORMAT: json
  PHOENIX_IMPORT_WATCH: "off"      # no watched download folders inside a container
  PHOENIX_WORKER_INTERVAL: "300"
  PHOENIX_CONFIG_DIR: /app/config
secretEnv:
  - {name: PHOENIX_DATABASE_URL, secretName: phoenix-db-app, key: uri}
envFromSecrets:
  - phoenix-secrets               # PHOENIX_BASIC_AUTH, PHOENIX_SECRET_KEY. No broker or order keys this week.
podLabels:
  app.kubernetes.io/part-of: phoenix
  lab.platform/access-timescale: "true"
  lab.platform/access-redis: "true"
resources:
  requests: {cpu: 100m, memory: 256Mi}
  limits: {memory: 1Gi}
workers:
  - name: worker                  # bars, screen, ticket expiry (paper sync is off: no Alpaca keys in the cluster)
    args: [worker]
    resources:
      requests: {cpu: 100m, memory: 256Mi}
      limits: {memory: 1Gi}
migrations:
  enabled: true                   # `phoenix db upgrade` (Alembic) before every rollout
  args: [db, upgrade]
__EOF__
```

#087 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/apps/command-center/overlays/halo'
cat > 'platform/kubernetes/apps/command-center/overlays/halo/values.yaml' <<'__EOF__'
# platform/kubernetes/apps/command-center/overlays/halo/values.yaml — cluster "halo".
# image.tag is written by CI (.github/actions/promote-tag). Humans change it only via git revert.
image:
  tag: ""
replicaCount: 1                    # one API pod: Phoenix runs background threads (knowledge base, imports)
env:
  PHOENIX_CLOCK: "2026-10-22T10:40"          # demo clock + synthetic demo data (Phoenix rule: no real account data)
  PHOENIX_ALLOWED_HOSTS: phoenix.halo.localhost
ingress:
  enabled: true
  host: phoenix.halo.localhost
jobs: []                           # Step B4.4 adds the one-shot demo-data seed here
__EOF__
```

#088 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/apps/command-center/overlays/aws'
cat > 'platform/kubernetes/apps/command-center/overlays/aws/README.md' <<'__EOF__'
# overlays/aws — placeholder until the AWS labs.
__EOF__
```

#089 WSL Ubuntu shell (bash) - copy & paste

```
git add -A && git commit -m "ci(command-center): shared pipeline + Phoenix test jobs; chart values" && git pull --rebase && git push
```

[GITHUB] *Actions → svc-command-center*: `ci / verify` (≈3 min; 381 tests), `ci / image`, `ci / promote -> halo`, `postgres-tests`, `windows-tests` and `fidelity-helper`, all ✓. `repo-guards` ✓ (gitleaks over the imported history; the fixture false positive is allowlisted).

Verify:

#090 WSL Ubuntu shell (bash) - copy & paste

```
git pull --rebase
yq '.image.tag' platform/kubernetes/apps/command-center/overlays/halo/values.yaml     # a 40-char SHA
```

[GITHUB] *Profile → Packages → command-center* is **Private** and linked to `ai-platform-portfolio`.

If it fails

1. `ci / verify` fails on `make install` → `pyproject.toml` extras changed. `make ci` must pass locally in `projects/command-center` first.
2. `promote` says `file not found` → the `overlays/halo/values.yaml` commit isn't on `main`. Push it, then re-run.
3. `windows-tests` fails only on `exchange_calendars` downloads → transient. Re-run the job.

- ☐ Phoenix imported with history · ☐ all 6 CI jobs green · ☐ tag promoted · ☐ publication check noted

CKA corner: none in the cluster yet; spend the time on the drill.

Interview angle: "I migrated an existing app into the monorepo with its history rewritten under the new path, kept its platform-specific tests, and only changed what the platform contract needed: probes, logs and a CI entry point."

Phoenix angle: Phoenix now ships the same way every future service will, and its PostgreSQL tests run against the exact database engine it uses in the lab.

Drill W3-1 (15 min, `kcka`): ConfigMaps and Secrets.

1. Create ConfigMap `app-cfg` with `MODE=lab` and `LEVEL=debug`.
2. Run pod `cfg-env` (`busybox:1.37.0`, `sleep 3600`) with **all** keys of `app-cfg` as env vars; print `MODE` from inside.
3. Create Secret `db-cred` (`user=lab`, `pass=s3cr3t`) and mount it read-only at `/etc/db` in pod `sec-vol`; `cat` the `pass` file.
4. In pod `sec-env`, expose only `db-cred`'s `user` key as env var `DB_USER`.
5. Print the decoded value of `db-cred`'s `pass` with jsonpath + base64.

B2: Secrets and the data layer: TimescaleDB + Redis (Mon Nov 16, block 2)

B2.1 Seal the Phoenix secrets

Goal: four SealedSecrets in Git, with no plaintext on disk or in Git.

Why: each script prints a generated value **once**, for the password manager, then writes only ciphertext. The DB password lives in **two** Secrets that must agree: `timescale-auth` (the server side) and `phoenix-db-app/uri` (the app side, the CNPG naming, so a move to CloudNativePG later needs no app change). That's why one script generates both. Namespace-scoped sealing means the ciphertext only decrypts in namespace `phoenix` under that name.

Where: [WSL]

#091 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/scripts'
cat > 'platform/scripts/seal-phoenix-db.sh' <<'__EOF__'
#!/usr/bin/env bash
# platform/scripts/seal-phoenix-db.sh — one Phoenix DB password, sealed into the two Secrets that must agree:
#   timescale-auth (postgres-password, phoenix-password)  and  phoenix-db-app (uri, the CNPG-style contract)
set -euo pipefail
```

```
cd "$(git rev-parse --show-toplevel)"
D=platform/kubernetes/apps/phoenix-data/overlays/halo/sealed
mkdir -p "$D"
SUPER=$(openssl rand -hex 24); APP=$(openssl rand -hex 24)
echo "  postgres superuser password: $SUPER   <- password manager" >&2
echo "  phoenix app password:        $APP   <- password manager" >&2
kubectl -n phoenix create secret generic timescale-auth \
  --from-literal=postgres-password="$SUPER" --from-literal=phoenix-password="$APP" --dry-run=client -o yaml \
  | kubeseal --format yaml > "$D/timescale-auth.yaml"
kubectl -n phoenix create secret generic phoenix-db-app \
  --from-literal=uri="postgresql://phoenix:${APP}@timescale.phoenix.svc.cluster.local:5432/phoenix" \
  --dry-run=client -o yaml \
  | kubeseal --format yaml > "$D/phoenix-db-app.yaml"
unset SUPER APP
echo "wrote $D/timescale-auth.yaml and $D/phoenix-db-app.yaml"
__EOF__
chmod +x 'platform/scripts/seal-phoenix-db.sh'
```

#092 WSL Ubuntu shell (bash) - copy & paste

```
platform/scripts/seal-phoenix-db.sh
D=platform/kubernetes/apps/phoenix-data/overlays/halo/sealed
platform/scripts/seal-secret.sh phoenix redis-auth $D/redis-auth.yaml password=gen
platform/scripts/seal-secret.sh phoenix phoenix-secrets $D/phoenix-secrets.yaml PHOENIX_BASIC_AUTH PHOENIX_SECRET_KEY=fernet
#   PHOENIX_BASIC_AUTH prompt: type  nikhil:<a long password>   (this is the web login)
ls $D; grep -c 'kind: SealedSecret' $D/*.yaml
```

Expected: each printed value saved in the password manager (entries `Phoenix DB`, `Redis`, `Phoenix app`). Four files, each `1`.

B2.2 TimescaleDB, Redis, backups, guardrails

Goal: `phoenix-data` (wave 20) creates namespace `phoenix` (PSA `restricted`), its quota and LimitRange, two StatefulSets and the backup CronJob.

Why:

- **StatefulSet** = stable pod name (`timescale-0`), stable storage (a PVC per replica, from `volumeClaimTemplates`) and ordered start/stop, which is what databases need. A Deployment would give random pod names and shared volumes.
- **Headless Service** (`clusterIP: None`): DNS points straight at the pod.

- The **init script** runs once, on an empty data directory, and creates the non-superuser `phoenix` role and database. It's mounted with `subPath` so the image's own Timescale init scripts stay in place.
- The **backup PVC** is also mounted in `timescale-0`. A `WaitForFirstConsumer` PVC stays *Pending* until a pod uses it, and Argo CD would wait forever for it to be healthy. Mounting it in the database pod also lets you restore from inside that pod.
- `Prune=false,Delete=false` on the StatefulSets and the backup PVC, plus no finalizer on the Application: no Git mistake can delete the data.
  Where: [WSL]

#093 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/apps/phoenix-data/base'
cat > 'platform/kubernetes/apps/phoenix-data/base/kustomization.yaml' <<'__EOF__'
# platform/kubernetes/apps/phoenix-data/base/kustomization.yaml — namespace "phoenix": guardrails + data layer.
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
labels:
  - pairs:
      app.kubernetes.io/part-of: phoenix
    includeSelectors: false
resources:
  - guardrails.yaml
  - timescale.yaml
  - redis.yaml
  - backup.yaml
__EOF__
```

#094 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/apps/phoenix-data/base'
cat > 'platform/kubernetes/apps/phoenix-data/base/guardrails.yaml' <<'__EOF__'
# platform/kubernetes/apps/phoenix-data/base/guardrails.yaml — the tenant budget for namespace "phoenix".
apiVersion: v1
kind: ResourceQuota
metadata:
  name: tenant-budget
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
spec:
  hard:
    requests.cpu: "2"
```

```
    requests.memory: 4Gi
    limits.memory: 8Gi
    pods: "20"
    persistentvolumeclaims: "5"
    requests.storage: 60Gi
    services.loadbalancers: "0"
    services.nodeports: "0"
---
apiVersion: v1
kind: LimitRange
metadata:
  name: defaults
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
spec:
  limits:
    - type: Container
      defaultRequest: {cpu: 50m, memory: 64Mi}
      default: {memory: 256Mi}
      max: {memory: 4Gi}
__EOF__
```

#095 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/apps/phoenix-data/base'
cat > 'platform/kubernetes/apps/phoenix-data/base/timescale.yaml' <<'__EOF__'
# platform/kubernetes/apps/phoenix-data/base/timescale.yaml — TimescaleDB (PostgreSQL 18) as a 1-replica StatefulSet.
# Official image (Alpine, uid 70), PVC on local-path-retain, non-root, read-only root filesystem.
apiVersion: v1
kind: ConfigMap
metadata:
  name: timescale-init
  annotations:
    argocd.argoproj.io/sync-wave: "-1"
data:
  10-phoenix.sh: |
    #!/bin/sh
    # Runs ONCE, on an empty data directory: the app role and database. The app is not a superuser.
```

```
    set -eu
    psql -v ON_ERROR_STOP=1 --username "$POSTGRES_USER" --dbname postgres <<SQL
    CREATE ROLE phoenix LOGIN PASSWORD '${PHOENIX_DB_PASSWORD}';
    CREATE DATABASE phoenix OWNER phoenix;
    SQL
    psql -v ON_ERROR_STOP=1 --username "$POSTGRES_USER" --dbname phoenix -c "CREATE EXTENSION IF NOT EXISTS timescaledb;"
---
apiVersion: v1
kind: Service
metadata:
  name: timescale
  labels: {app.kubernetes.io/name: timescale}
spec:
  clusterIP: None                       # headless: DNS name timescale.phoenix.svc resolves to the pod
  selector: {app.kubernetes.io/name: timescale}
  ports:
    - {name: postgres, port: 5432, targetPort: postgres}
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: timescale
  labels: {app.kubernetes.io/name: timescale}
  annotations:
    argocd.argoproj.io/sync-wave: "0"
    argocd.argoproj.io/sync-options: Prune=false,Delete=false
spec:
  serviceName: timescale
  replicas: 1
  selector:
    matchLabels: {app.kubernetes.io/name: timescale}
  template:
    metadata:
      labels: {app.kubernetes.io/name: timescale, app.kubernetes.io/component: database}
    spec:
      automountServiceAccountToken: false
      terminationGracePeriodSeconds: 60          # time for a clean shutdown checkpoint
      securityContext:
```

```
      runAsNonRoot: true
      runAsUser: 70
      runAsGroup: 70
      fsGroup: 70
      fsGroupChangePolicy: OnRootMismatch
      seccompProfile: {type: RuntimeDefault}
    containers:
      - name: timescale
        image: timescale/timescaledb:2.31.0-pg18@sha256:0fcff5761363809d1242f2261a3482197b5a5337407f8b97a976fb174206d579
        ports:
          - {name: postgres, containerPort: 5432}
        env:
          - name: POSTGRES_PASSWORD
            valueFrom: {secretKeyRef: {name: timescale-auth, key: postgres-password}}
          - name: PHOENIX_DB_PASSWORD
            valueFrom: {secretKeyRef: {name: timescale-auth, key: phoenix-password}}
          - {name: PGDATA, value: /var/lib/postgresql/data/pgdata}   # explicit: PG 18 images moved the default
          - {name: TIMESCALEDB_TELEMETRY, value: "off"}
          - {name: TS_TUNE_MEMORY, value: 2GB}                        # tune for the container limit, not the VM
          - {name: TS_TUNE_NUM_CPUS, value: "2"}
        startupProbe:
          exec: {command: [pg_isready, -h, 127.0.0.1, -U, postgres]}
          periodSeconds: 5
          failureThreshold: 60
        readinessProbe:
          exec: {command: [pg_isready, -h, 127.0.0.1, -U, postgres]}
          periodSeconds: 10
        livenessProbe:
          exec: {command: [pg_isready, -h, 127.0.0.1, -U, postgres]}
          periodSeconds: 20
          failureThreshold: 6
        resources:
          requests: {cpu: 250m, memory: 512Mi}
          limits: {memory: 2Gi}
        securityContext:
          allowPrivilegeEscalation: false
          readOnlyRootFilesystem: true
          capabilities: {drop: [ALL]}
```

```
          volumeMounts:
            - {name: data, mountPath: /var/lib/postgresql/data}
            - {name: run, mountPath: /var/run/postgresql}
            - {name: tmp, mountPath: /tmp}
            - {name: init, mountPath: /docker-entrypoint-initdb.d/10-phoenix.sh, subPath: 10-phoenix.sh}
            - {name: backups, mountPath: /backups}     # binds the backup PVC now; restores run from here
      volumes:
        - {name: run, emptyDir: {}}
        - {name: tmp, emptyDir: {}}
        - name: backups
          persistentVolumeClaim: {claimName: timescale-backups}
        - name: init
          configMap: {name: timescale-init, defaultMode: 0555}
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: [ReadWriteOnce]
        storageClassName: local-path-retain
        resources:
          requests: {storage: 20Gi}
__EOF__
```

#096 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/apps/phoenix-data/base'
cat > 'platform/kubernetes/apps/phoenix-data/base/redis.yaml' <<'__EOF__'
# platform/kubernetes/apps/phoenix-data/base/redis.yaml — Redis 8 (official image) with AOF persistence on a PVC.
# Phoenix 2.1 has no Redis client yet: this is provisioned for the L2 agents (cache, rate limits, job state).
apiVersion: v1
kind: Service
metadata:
  name: redis
  labels: {app.kubernetes.io/name: redis}
spec:
  selector: {app.kubernetes.io/name: redis}
  ports:
    - {name: redis, port: 6379, targetPort: redis}
```

```
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
  labels: {app.kubernetes.io/name: redis}
  annotations:
    argocd.argoproj.io/sync-wave: "0"
    argocd.argoproj.io/sync-options: Prune=false,Delete=false
spec:
  serviceName: redis
  replicas: 1
  selector:
    matchLabels: {app.kubernetes.io/name: redis}
  template:
    metadata:
      labels: {app.kubernetes.io/name: redis, app.kubernetes.io/component: cache}
    spec:
      automountServiceAccountToken: false
      securityContext:
        runAsNonRoot: true
        runAsUser: 999
        runAsGroup: 1000
        fsGroup: 1000
        fsGroupChangePolicy: OnRootMismatch
        seccompProfile: {type: RuntimeDefault}
      containers:
        - name: redis
          image: redis:8.10.3-alpine@sha256:21090af93f8324db2f0f0f4f38a7c60369d47d917e000c3423d89d2ae2883537
          args:
            - --appendonly
            - "yes"
            - --appendfsync
            - everysec
            - --dir
            - /data
            - --maxmemory
            - 256mb
```

```
            - --maxmemory-policy
            - noeviction
            - --requirepass
            - $(REDIS_PASSWORD)                  # expanded by the kubelet from the env var below
          ports:
            - {name: redis, containerPort: 6379}
          env:
            - name: REDIS_PASSWORD
              valueFrom: {secretKeyRef: {name: redis-auth, key: password}}
            - name: REDISCLI_AUTH                # lets the probes run redis-cli without -a on the command line
              valueFrom: {secretKeyRef: {name: redis-auth, key: password}}
          readinessProbe:
            exec: {command: [redis-cli, ping]}
            periodSeconds: 10
          livenessProbe:
            exec: {command: [redis-cli, ping]}
            periodSeconds: 20
            failureThreshold: 6
          resources:
            requests: {cpu: 50m, memory: 128Mi}
            limits: {memory: 512Mi}
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: {drop: [ALL]}
          volumeMounts:
            - {name: data, mountPath: /data}
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: [ReadWriteOnce]
        storageClassName: local-path-retain
        resources:
          requests: {storage: 2Gi}
__EOF__
```

#097 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/apps/phoenix-data/base'
cat > 'platform/kubernetes/apps/phoenix-data/base/backup.yaml' <<'__EOF__'
# platform/kubernetes/apps/phoenix-data/base/backup.yaml — nightly pg_dump of database "phoenix" to its own PVC.
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: timescale-backups
  annotations:
    # same wave as the StatefulSet that mounts it: a WaitForFirstConsumer PVC stays Pending (Progressing)
    # until a pod uses it, and Argo CD would wait forever on an earlier wave
    argocd.argoproj.io/sync-wave: "0"
    argocd.argoproj.io/sync-options: Prune=false,Delete=false
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: local-path-retain
  resources:
    requests: {storage: 20Gi}
---
apiVersion: batch/v1
kind: CronJob
metadata:
  name: timescale-backup
  annotations:
    argocd.argoproj.io/sync-wave: "1"
spec:
  schedule: "15 21 * * *"                # 21:15 every night...
  timeZone: Etc/UTC                      # ...in this zone; change to your IANA zone, e.g. Europe/London
  concurrencyPolicy: Forbid
  startingDeadlineSeconds: 3600          # still runs if the PC was asleep at 21:15, within the hour
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      backoffLimit: 1
      activeDeadlineSeconds: 1800
      template:
        metadata:
```

```
    labels:
      app.kubernetes.io/name: timescale-backup
      lab.platform/access-timescale: "true"
  spec:
    restartPolicy: Never
    automountServiceAccountToken: false
    securityContext:
      runAsNonRoot: true
      runAsUser: 70
      runAsGroup: 70
      fsGroup: 70
      seccompProfile: {type: RuntimeDefault}
    containers:
      - name: pg-dump
        image: timescale/timescaledb:2.31.0-pg18@sha256:0fcff5761363809d1242f2261a3482197b5a5337407f8b97a976fb174206d579
        command: [/bin/sh, -c]
        args:
          - |
            set -eu
            f="/backups/phoenix-`date -u +%Y%m%dT%H%M%SZ`.dump"
            pg_dump --format=custom --dbname=phoenix --file="$f.partial"
            mv "$f.partial" "$f"
            find /backups -name 'phoenix-*.dump' -mtime +14 -delete
            echo "backup ok: $f"; ls -lh /backups | tail -5
        env:
          - {name: PGHOST, value: timescale.phoenix.svc.cluster.local}
          - {name: PGUSER, value: postgres}
          - name: PGPASSWORD
            valueFrom: {secretKeyRef: {name: timescale-auth, key: postgres-password}}
        resources:
          requests: {cpu: 50m, memory: 128Mi}
          limits: {memory: 512Mi}
        securityContext:
          allowPrivilegeEscalation: false
          readOnlyRootFilesystem: true
          capabilities: {drop: [ALL]}
        volumeMounts:
          - {name: backups, mountPath: /backups}
```

```
                - {name: tmp, mountPath: /tmp}
          volumes:
            - name: backups
              persistentVolumeClaim: {claimName: timescale-backups}
            - {name: tmp, emptyDir: {}}
__EOF__
```

#098 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/apps/phoenix-data/overlays/halo'
cat > 'platform/kubernetes/apps/phoenix-data/overlays/halo/kustomization.yaml' <<'__EOF__'
# platform/kubernetes/apps/phoenix-data/overlays/halo/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: phoenix
resources:
  - ../../base
  - sealed/timescale-auth.yaml          # platform/scripts/seal-phoenix-db.sh
  - sealed/phoenix-db-app.yaml          # platform/scripts/seal-phoenix-db.sh
  - sealed/redis-auth.yaml              # platform/scripts/seal-secret.sh
  - sealed/phoenix-secrets.yaml         # platform/scripts/seal-secret.sh
  # - ollama-external.yaml              # optional (Step B4.6): Phoenix chat -> Ollama on Windows
labels:
  - pairs:
      lab.platform/env: halo
    includeSelectors: false
__EOF__
```

#099 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/apps/phoenix-data/overlays/aws'
cat > 'platform/kubernetes/apps/phoenix-data/overlays/aws/README.md' <<'__EOF__'
# overlays/aws — placeholder until the AWS labs (CNPG or RDS for PostgreSQL there).
__EOF__
```

#100 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/argocd/clusters/halo'
cat > 'platform/kubernetes/argocd/clusters/halo/phoenix-data.yaml' <<'__EOF__'
# platform/kubernetes/argocd/clusters/halo/phoenix-data.yaml — namespace guardrails + TimescaleDB + Redis + backups.
```

```
# No resources-finalizer: deleting this Application must never delete the databases.
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: phoenix-data
  namespace: argocd
  labels:
    lab.platform/env: halo
    lab.platform/tenant: phoenix
  annotations:
    argocd.argoproj.io/sync-wave: "20"           # before command-center (30); waits until Healthy
spec:
  project: lab
  source:
    repoURL: git@github.com:nikhilgs817/ai-platform-portfolio.git
    targetRevision: main
    path: platform/kubernetes/apps/phoenix-data/overlays/halo
  destination:
    server: https://kubernetes.default.svc
    namespace: phoenix
  syncPolicy:
    automated:
      prune: true                                # StatefulSets and the backup PVC carry Prune=false,Delete=false
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
    managedNamespaceMetadata:
      labels:
        pod-security.kubernetes.io/enforce: restricted
        pod-security.kubernetes.io/enforce-version: latest
        lab.platform/tenant: phoenix
        lab.platform/env: halo
    retry:
      limit: 5
      backoff: {duration: 10s, factor: 2, maxDuration: 3m}
__EOF__
```

#101 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/argocd/clusters/halo'
cat > 'platform/kubernetes/argocd/clusters/halo/kustomization.yaml' <<'__EOF__'
# platform/kubernetes/argocd/clusters/halo/kustomization.yaml
# Everything that runs on cluster "halo" = the child Applications listed here (sync-wave order in each file).
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - projects.yaml           # wave -10  AppProjects
  - sealed-secrets.yaml     # wave -5   platform add-on
  - storage-classes.yaml    # wave -5   platform add-on
  - podinfo.yaml            # wave 10   Week 1 sample
  - phoenix-data.yaml       # wave 20   Phoenix guardrails + TimescaleDB + Redis + backups
__EOF__
```

#102 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'docs/adrs'
cat > 'docs/adrs/0003-phoenix-data-layer.md' <<'__EOF__'
# ADR-0003: Phoenix data layer — official TimescaleDB and Redis images as StatefulSets

- **Status:** Accepted · **Date:** 2026-11-16

## Context
Phoenix needs PostgreSQL (SQLAlchemy + Alembic; the plan calls for TimescaleDB for bar data in L8) and Redis
(planned for the L2 agents). Single node, local-path storage, around-the-clock uptime, free and maintained images.
Bitnami moved its versioned images to the unmaintained `bitnamilegacy` repository in Aug–Sep 2025, leaving only a
small free "latest" set, so Bitnami charts are no longer a pinned, maintained option.

## Decision
- **TimescaleDB 2.31.0 on PostgreSQL 18** (official `timescale/timescaledb`, pinned by digest) as a 1-replica
  StatefulSet, PVC on `local-path-retain`, non-root (uid 70), read-only root filesystem.
- **Redis 8.10.3** (official `redis:alpine`, pinned by digest) as a 1-replica StatefulSet with AOF persistence.
- App credentials keep the CNPG contract: Secret `phoenix-db-app` with key `uri`, so a later move to CloudNativePG
  needs no app change.
- Backups: nightly `pg_dump -Fc` CronJob to a Retain PVC; restore test in the runbook.

## Alternatives
```

```
| Option | Why not now |
|---|---|
| CloudNativePG 1.30 + TimescaleDB | Best for HA and PITR, but TimescaleDB on CNPG needs a custom image. Revisit on AWS |
| Bitnami charts | Versioned images unmaintained since 2025 (see Context) |
| Valkey 9.1 instead of Redis | Drop-in and BSD-licensed; a good swap later. Redis 8 is fine for personal use (AGPLv3 option since 8.0) |

## Consequences
- Plain manifests teach StatefulSets, headless Services and PVC templates (CKA).
- No automatic failover: a node or disk loss means a restore from backup.
__EOF__
```

#103 WSL Ubuntu shell (bash) - copy & paste

```
kubectl kustomize platform/kubernetes/apps/phoenix-data/overlays/halo | grep -E '^kind:' | sort | uniq -c
git add -A && git commit -m "feat(phoenix): data layer (TimescaleDB, Redis, backups, guardrails)" && git pull --rebase && git push
argocd app get root-halo --refresh >/dev/null
until argocd app get phoenix-data >/dev/null 2>&1; do sleep 5; done
argocd app wait phoenix-data --health --timeout 600
kubectl -n phoenix get pods,pvc,secrets
```

Expected: `timescale-0 1/1 Running` (first start 30–90 s: initdb + timescaledb-tune), `redis-0 1/1 Running`. PVCs `data-timescale-0`, `data-redis-0` and `timescale-backups` are **Bound** on `local-path-retain`. Secrets `timescale-auth`, `phoenix-db-app`, `redis-auth` and `phoenix-secrets` exist (created by the controller).

Verify:

#104 WSL Ubuntu shell (bash) - copy & paste

```
kubectl -n phoenix exec timescale-0 -c timescale -- sh -c \
  'PGPASSWORD=$POSTGRES_PASSWORD psql -h 127.0.0.1 -U postgres -d phoenix -c "\dx timescaledb" -c "\du phoenix"'
kubectl -n phoenix exec redis-0 -- redis-cli --no-auth-warning ping
kubectl get pv -o custom-columns=NAME:.metadata.name,CLAIM:.spec.claimRef.name,POLICY:.spec.persistentVolumeReclaimPolicy
kubectl get ns phoenix --show-labels | grep -o 'enforce=restricted'
```

Expected: extension `timescaledb 2.31.0`; role `phoenix` (no superuser attributes); `PONG`; the three PVs with `Retain`; `enforce=restricted`.

If it fails

1. `timescale-0` CrashLoop with `initdb: could not change permissions` or `Permission denied` → the PVC directory isn't writable by uid 70. Check that `fsGroup: 70` is present, and look at the directory on disk: `sudo ls -ld /var/lib/rancher/k3s/storage/*data-timescale-0*`.
2. SealedSecret events show `no key could decrypt secret` → sealed against another controller (or a rebuild without key restore). Re-seal (B2.1), or restore the key (runbook).

3. `phoenix-data` stays *Progressing* on the `timescale-backups` PVC → it isn't mounted by `timescale-0` (see `timescale.yaml` volumes). Appendix T4.

- ☐ Four SealedSecrets committed, values in the password manager · ☐ both StatefulSets Ready, PVCs Bound with Retain

CKA corner: StatefulSets, headless Services and `volumeClaimTemplates` (Workloads + Storage). Practise `kubectl get sts`, `kubectl rollout status sts/timescale`, `kubectl describe pvc`, `kubectl get pv`.

Interview angle: "The data layer is plain official images on StatefulSets, so I can explain every line. Credentials keep the CloudNativePG contract, so the move to an operator on AWS is a swap, not a rewrite."

Phoenix angle: Phoenix's database now lives on a Retain volume with a non-superuser app role, and the init runs once, declaratively.

Drill W3-2 (15 min, `kcka`): resource requests and limits.

1. Run pod `sized` (`nginx:1.29`) with requests `cpu=100m,memory=64Mi` and limit `memory=128Mi` (generate with `--dry-run`, edit, apply).
2. Create a LimitRange in `cka` with default request `cpu=50m` and default limit `memory=256Mi`; create a pod without resources and show what it got.
3. Create ResourceQuota `pods-max` with `pods=3` and try to start a 4th pod. Read the error.
4. Make a pod get **OOMKilled** (`busybox:1.37.0`, limit `32Mi`, `sh -c 'head -c 100m /dev/zero | tail'`) and read the reason with jsonpath.
5. Show pods in `cka` sorted by memory use (`kubectl top`). Clean up the quota and LimitRange afterwards.

B3: Namespace guardrails and NetworkPolicies (Tue Nov 17, block 1)

Goal: default-deny in both directions, then only DNS, app → Timescale, app → Redis and the ingress to the app, with a proof that k3s enforces it.

Why: a NetworkPolicy is a firewall rule for pods, selected by **labels** rather than IPs. Once any policy selects a pod for a direction, everything not explicitly allowed in that direction is dropped. Access is granted by pod label (`lab.platform/access-timescale: "true"`), so "who may talk to the database" is a reviewable line in the chart values. **k3s enforces NetworkPolicy with its embedded kube-router controller**, unless the server runs with `disable-network-policy` (your Week 1 config doesn't). Traefik runs hostNetwork, so its traffic reaches pods from the node (`10.255.255.10`, or the cni0 gateway `10.42.0.1`). kube-router also always admits traffic from the local node (that's how kubelet probes work), but the explicit rule documents intent and keeps working on other CNIs.

Where: [WSL]

#105 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/apps/phoenix-data/base'
cat > 'platform/kubernetes/apps/phoenix-data/base/networkpolicies.yaml' <<'__EOF__'
# platform/kubernetes/apps/phoenix-data/base/networkpolicies.yaml
# Default deny in both directions, then only: DNS, app -> Timescale, app -> Redis, and the ingress to the app.
# Access is granted by POD LABEL (lab.platform/access-*: "true"), set in the chart's podLabels for the app pods.
# k3s enforces these with its embedded kube-router controller (disable-network-policy is NOT set).
apiVersion: networking.k8s.io/v1
```

```
kind: NetworkPolicy
metadata:
  name: default-deny-all
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-dns-egress
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
    - to:
        - namespaceSelector:
            matchLabels: {kubernetes.io/metadata.name: kube-system}
          podSelector:
            matchLabels: {k8s-app: kube-dns}
      ports:
        - {port: 53, protocol: UDP}
        - {port: 53, protocol: TCP}
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: timescale-ingress
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
spec:
  podSelector:
    matchLabels: {app.kubernetes.io/name: timescale}
  policyTypes: [Ingress]
```

```
  ingress:
    - from:
        - podSelector:
            matchLabels: {lab.platform/access-timescale: "true"}
      ports:
        - {port: 5432, protocol: TCP}
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: redis-ingress
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
spec:
  podSelector:
    matchLabels: {app.kubernetes.io/name: redis}
  policyTypes: [Ingress]
  ingress:
    - from:
        - podSelector:
            matchLabels: {lab.platform/access-redis: "true"}
      ports:
        - {port: 6379, protocol: TCP}
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: clients-egress-to-data
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
spec:
  podSelector:
    matchExpressions:
      - {key: lab.platform/access-timescale, operator: In, values: ["true"]}
  policyTypes: [Egress]
  egress:
    - to:
        - podSelector:
```

```
            matchLabels: {app.kubernetes.io/name: timescale}
      ports:
        - {port: 5432, protocol: TCP}
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: clients-egress-to-redis
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
spec:
  podSelector:
    matchExpressions:
      - {key: lab.platform/access-redis, operator: In, values: ["true"]}
  policyTypes: [Egress]
  egress:
    - to:
        - podSelector:
            matchLabels: {app.kubernetes.io/name: redis}
      ports:
        - {port: 6379, protocol: TCP}
---
# The web app only: Traefik runs hostNetwork, so its traffic arrives from the node (halo0 IP or cni0 gateway).
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: command-center-ingress
  annotations:
    argocd.argoproj.io/sync-wave: "-2"
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: command-center
      app.kubernetes.io/component: api
  policyTypes: [Ingress]
  ingress:
    - from:
        - ipBlock: {cidr: 10.255.255.10/32}
```

```
        - ipBlock: {cidr: 10.42.0.1/32}
      ports:
        - {port: 8000, protocol: TCP}
__EOF__
```

#106 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/apps/phoenix-data/base'
cat > 'platform/kubernetes/apps/phoenix-data/base/kustomization.yaml' <<'__EOF__'
# platform/kubernetes/apps/phoenix-data/base/kustomization.yaml — namespace "phoenix": guardrails + data layer.
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
labels:
  - pairs:
      app.kubernetes.io/part-of: phoenix
    includeSelectors: false
resources:
  - guardrails.yaml
  - networkpolicies.yaml
  - timescale.yaml
  - redis.yaml
  - backup.yaml
__EOF__
```

#107 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/scripts'
cat > 'platform/scripts/np-probe.sh' <<'__EOF__'
#!/usr/bin/env bash
# platform/scripts/np-probe.sh — NetworkPolicy check: can a pod with (or without) a label reach timescale or redis?
# Usage: np-probe.sh <namespace> <key=value|none> <timescale|redis>     prints REACHABLE or BLOCKED
set -euo pipefail
NS="${1:?}"; LABEL="${2:?}"; TARGET="${3:?}"
case "$TARGET" in
  timescale)
    IMAGE="timescale/timescaledb:2.31.0-pg18@sha256:0fcff5761363809d1242f2261a3482197b5a5337407f8b97a976fb174206d579"
    CMD="pg_isready -h timescale -p 5432 -t 5" ;;
  redis)
    IMAGE="redis:8.10.3-alpine@sha256:21090af93f8324db2f0f0f4f38a7c60369d47d917e000c3423d89d2ae2883537"
```

```
    CMD="timeout 5 redis-cli -h redis -p 6379 --no-auth-warning ping | grep -qE 'PONG|NOAUTH'" ;;
  *) echo "target must be timescale or redis"; exit 2 ;;
esac
NAME="np-probe-$RANDOM"
EXTRA=""
[ "$LABEL" = none ] || EXTRA="    ${LABEL%%=*}: \"${LABEL#*=}\""
kubectl -n "$NS" apply -f - >/dev/null <<YAML
apiVersion: v1
kind: Pod
metadata:
  name: $NAME
  labels:
    app.kubernetes.io/name: np-probe
$EXTRA
spec:
  restartPolicy: Never
  automountServiceAccountToken: false
  securityContext: {runAsNonRoot: true, runAsUser: 65534, runAsGroup: 65534, seccompProfile: {type: RuntimeDefault}}
  containers:
    - name: probe
      image: "$IMAGE"
      command: ["sh", "-c", "$CMD"]
      resources: {requests: {cpu: 10m, memory: 16Mi}, limits: {memory: 64Mi}}
      securityContext: {allowPrivilegeEscalation: false, readOnlyRootFilesystem: true, capabilities: {drop: [ALL]}}
YAML
phase=Pending
for _ in $(seq 1 40); do
  phase=$(kubectl -n "$NS" get pod "$NAME" -o jsonpath='{.status.phase}')
  case "$phase" in Succeeded|Failed) break ;; esac
  sleep 1
done
kubectl -n "$NS" delete pod "$NAME" --wait=false >/dev/null
if [ "$phase" = Succeeded ]; then r=REACHABLE; else r=BLOCKED; fi
echo "$r   label=$LABEL -> $TARGET"
__EOF__
chmod +x 'platform/scripts/np-probe.sh'
```

#108 WSL Ubuntu shell (bash) - copy & paste

```
git add -A && git commit -m "feat(phoenix): default-deny NetworkPolicies + probe script" && git pull --rebase && git push
argocd app get phoenix-data --refresh >/dev/null && argocd app wait phoenix-data --sync --health --timeout 300
kubectl -n phoenix get networkpolicy
sudo iptables -S | grep -c 'KUBE-NWPLCY'          # > 0: kube-router turned the policies into iptables chains
```

Prove enforcement, with a pod that has no label, then each label:

#109 WSL Ubuntu shell (bash) - copy & paste

```
for l in none lab.platform/access-timescale=true lab.platform/access-redis=true; do
  for t in timescale redis; do platform/scripts/np-probe.sh phoenix "$l" "$t"; done
done
```

Expected:

Output / reference - not a command

```
BLOCKED     label=none -> timescale
BLOCKED     label=none -> redis
REACHABLE   label=lab.platform/access-timescale=true -> timescale
BLOCKED     label=lab.platform/access-timescale=true -> redis
BLOCKED     label=lab.platform/access-redis=true -> timescale
REACHABLE   label=lab.platform/access-redis=true -> redis
```

GitOps bonus: `kubectl -n phoenix delete networkpolicy default-deny-all`, then `kubectl -n phoenix get networkpolicy default-deny-all -w`. selfHeal recreates it within seconds. A hand-made hole in the firewall doesn't stay open.

If it fails

1. Everything `REACHABLE` → policies aren't enforced. Check `grep -i network /etc/rancher/k3s/config.yaml` (must not disable them) and `sudo journalctl -u k3s | grep -i 'network policy'`.
2. Everything `BLOCKED`, even with labels → DNS egress is blocked (the probe can't resolve `timescale`). Check `allow-dns-egress` and the CoreDNS label `k8s-app=kube-dns` (`kubectl -n kube-system get pods --show-labels`). Appendix T6.
3. The probe pod is rejected by `violates PodSecurity` → only if you edited the script's securityContext (`phoenix` enforces `restricted`).

- ☐ Probe table matches · ☐ selfHeal restores a deleted policy

CKA corner: NetworkPolicy (Services & Networking 20%) is a near-certain exam task. Know default-deny, `podSelector` vs `namespaceSelector` vs `ipBlock`, and that `from` entries are OR-ed while fields inside one entry are AND-ed.

Interview angle: "Every tenant namespace is default-deny. Data access is a pod label granted in reviewed values, and I prove enforcement with a probe matrix, not by assuming the CNI supports it."

Phoenix angle: when the internet-facing webhook (`hook`) arrives with the tunnel, it lands in a namespace where it can reach nothing it isn't explicitly given, as Phoenix's own design intended.

Drill W3-3 (15 min, `kcka`): scheduling on a single node. ⚠ Remove the taint at the end. On one node it blocks every new pod in the cluster, Phoenix included.

1. Label node `halo` with `disk=ssd`; run pod `fast` with `nodeSelector: {disk: ssd}`.
2. Run pod `slow` with `nodeSelector: {disk: hdd}`; read why it's Pending.
3. Taint `halo` with `drill=yes:NoSchedule`, run pod `tol` **with** a matching toleration and pod `notol` without; compare. Then **remove the taint**.
4. Run pod `aff` with a *required* node affinity `disk In [ssd]`.
5. Create deployment `spread` (2 replicas) with required pod **anti-affinity** on `kubernetes.io/hostname`; explain why one replica stays Pending here (single-node limit).

B4: The Command Center through the template (Tue Nov 17, block 2) — W3 checkpoint

B4.1 The Application

Goal: `command-center` (wave 30) syncs after `phoenix-data` is Healthy, and runs `serve` + `worker`, with the migration Job before every rollout.

Why: the sync order inside the app is ServiceAccount / ConfigMap / `ghcr-pull` SealedSecret (wave -1) → `command-center-migrate` Job running `phoenix db upgrade` (wave 1, a Sync hook) → Deployments (wave 2). If the migration fails, the old pods keep serving. No `CreateNamespace` here: `phoenix-data` owns the namespace and its labels (two apps managing one namespace's metadata would fight).

Where: [WSL], [BROWSER]

#110 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/argocd/clusters/halo'
cat > 'platform/kubernetes/argocd/clusters/halo/command-center.yaml' <<'__EOF__'
# platform/kubernetes/argocd/clusters/halo/command-center.yaml — Phoenix Command Center through the service chart.
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: command-center
  namespace: argocd
  labels:
    lab.platform/env: halo
    lab.platform/tenant: phoenix
  annotations:
    argocd.argoproj.io/sync-wave: "30"           # after phoenix-data is Healthy
  finalizers:
```

```
    - resources-finalizer.argocd.argoproj.io
spec:
  project: lab
  source:
    repoURL: git@github.com:nikhilgs817/ai-platform-portfolio.git
    targetRevision: main
    path: platform/helm-charts/service
    helm:
      releaseName: command-center
      valueFiles:
        - ../../kubernetes/apps/_shared/halo/values.yaml
        - ../../kubernetes/apps/command-center/base/values.yaml
        - ../../kubernetes/apps/command-center/overlays/halo/values.yaml
  destination:
    server: https://kubernetes.default.svc
    namespace: phoenix                          # created and labelled by phoenix-data, not here
  revisionHistoryLimit: 10
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - PruneLast=true
    retry:
      limit: 5
      backoff: {duration: 10s, factor: 2, maxDuration: 3m}
__EOF__
```

#111 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/argocd/clusters/halo'
cat > 'platform/kubernetes/argocd/clusters/halo/kustomization.yaml' <<'__EOF__'
# platform/kubernetes/argocd/clusters/halo/kustomization.yaml
# Everything that runs on cluster "halo" = the child Applications listed here (sync-wave order in each file).
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - projects.yaml           # wave -10  AppProjects
  - sealed-secrets.yaml     # wave -5   platform add-on
  - storage-classes.yaml    # wave -5   platform add-on
  - podinfo.yaml            # wave 10   Week 1 sample
  - phoenix-data.yaml       # wave 20   Phoenix guardrails + TimescaleDB + Redis + backups
  - command-center.yaml     # wave 30   Phoenix Command Center (service chart)
__EOF__
```

#112 WSL Ubuntu shell (bash) - copy & paste

```
git add -A && git commit -m "feat(command-center): deploy through the paved-road chart" && git pull --rebase && git push
argocd app get root-halo --refresh >/dev/null
until argocd app get command-center >/dev/null 2>&1; do sleep 5; done
argocd app wait command-center --sync --health --timeout 600
kubectl -n phoenix get deploy,pods,svc,ingress,sealedsecret
kubectl -n phoenix logs job/command-center-migrate | tail -3
```

Expected: Deployments `command-center 1/1` and `command-center-worker 1/1`, the migrate Job `Completed` (logs end with `database …` from `phoenix db upgrade`), Ingress `phoenix.halo.localhost` with address `10.255.255.10`.

B4.2 Probes, logs and metrics

#113 WSL Ubuntu shell (bash) - copy & paste

```
for p in livez readyz healthz; do curl -s -o /dev/null -w "$p %{http_code}\n" -H 'Host: phoenix.halo.localhost' http://127.0.0.1/$p; done
kubectl -n phoenix logs deploy/command-center --tail=3
kubectl -n phoenix port-forward svc/command-center 9100:9100 >/dev/null & sleep 2; curl -s localhost:9100/metrics | head -4; kill %1
```

Expected: `livez 200 · readyz 200 · healthz 200`; JSON log lines (`"service": "phoenix"`); Prometheus text starting with `# HELP phoenix_…`. [BROWSER] http://phoenix.halo.localhost: a basic-auth prompt (the `PHOENIX_BASIC_AUTH` you sealed), then Phoenix with **empty** data, the demo clock and the ruleset hash in the top bar.

B4.3 Seed synthetic demo data (one-shot Job)

Why: the lab rule is public or synthetic data only, and `phoenix demo` builds a realistic, invented journal. A **one-shot Job** pinned to the *current* image doesn't change when the app's tag changes later. Argo CD would otherwise try to mutate an immutable Job on every promotion. Rename it (`demo-seed-2`) to run it again.

#114 WSL Ubuntu shell (bash) - copy & paste

```
F=platform/kubernetes/apps/command-center/overlays/halo/values.yaml
TAG=$(yq '.image.tag' $F)
TAG="$TAG" yq -i '.jobs = [{"name": "demo-seed-1", "image": "ghcr.io/nikhilgs817/command-center:" + strenv(TAG),
                            "args": ["demo", "--url", "$(PHOENIX_DATABASE_URL)"]}]' $F
git commit -am "feat(command-center): seed synthetic demo data" && git pull --rebase && git push
argocd app get command-center --refresh >/dev/null && sleep 30
kubectl -n phoenix wait --for=condition=complete job/command-center-demo-seed-1 --timeout=300s
kubectl -n phoenix logs job/command-center-demo-seed-1 | tail -2
```

Expected: `demo database ready (N trades): postgresql+psycopg://…`. [BROWSER] refresh: Today, Journal and Rules show the invented history. `$(PHOENIX_DATABASE_URL)` is expanded by the kubelet from the container's env, so the password never appears in Git or in the pod spec.

B4.4 (Optional) Point the chat at Ollama on Windows

Why: your models stay on the Windows GPU (see Week 1's GPU note). A Service with a hand-written EndpointSlice gives pods a stable in-cluster name for an outside address. Full LLM work (LiteLLM, budgets, agents) is L2, so do this only if you want the chat page live now.

[WIN] Make Ollama listen beyond localhost and allow WSL's NAT range in:

#115 Windows PowerShell - copy & paste

```
[Environment]::SetEnvironmentVariable('OLLAMA_HOST', '0.0.0.0:11434', 'User')     # then quit + restart Ollama
New-NetFirewallRule -DisplayName 'Ollama from WSL (halo)' -Direction Inbound -Protocol TCP -LocalPort 11434 `
  -RemoteAddress 172.16.0.0/12 -Action Allow
(Get-NetIPAddress -AddressFamily IPv4 | Where-Object { $_.PrefixOrigin -eq 'Dhcp' }).IPAddress   # your LAN IP
```

#116 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'platform/kubernetes/apps/phoenix-data/overlays/halo'
cat > 'platform/kubernetes/apps/phoenix-data/overlays/halo/ollama-external.yaml' <<'__EOF__'
# platform/kubernetes/apps/phoenix-data/overlays/halo/ollama-external.yaml — OPTIONAL (Step B4.6).
# Ollama keeps running natively on Windows (GPU); pods reach it through a Service with a manual EndpointSlice.
# Replace 192.168.1.50 with your PC's LAN IPv4 (ipconfig) and give the PC a DHCP reservation so it never changes.
apiVersion: v1
kind: Service
```

```
metadata:
  name: ollama
spec:
  ports:
    - {name: http, port: 11434, targetPort: 11434, protocol: TCP}
---
apiVersion: discovery.k8s.io/v1
kind: EndpointSlice
metadata:
  name: ollama-windows
  labels:
    kubernetes.io/service-name: ollama
addressType: IPv4
ports:
  - {name: http, port: 11434, protocol: TCP}
endpoints:
  - addresses: ["192.168.1.50"]
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: command-center-egress-to-ollama
spec:
  podSelector:
    matchLabels: {app.kubernetes.io/name: command-center}
  policyTypes: [Egress]
  egress:
    - to:
        - ipBlock: {cidr: 192.168.1.50/32}
      ports:
        - {port: 11434, protocol: TCP}
__EOF__
```

[WSL] Put your LAN IP in that file (2 places), uncomment `ollama-external.yaml` in `overlays/halo/kustomization.yaml`, and add to `apps/command-center/overlays/halo/values.yaml` under `env:` → `PHOENIX_LLM_BASE_URL: http://ollama.phoenix.svc.cluster.local:11434/v1`, `PHOENIX_LLM_API_KEY: ollama`, `PHOENIX_LLM_BIG: <your model tag>`, `PHOENIX_LLM_FAST: <your model tag>`. Commit, push, then:

#117 WSL Ubuntu shell (bash) - copy & paste

```
kubectl -n phoenix exec deploy/command-center -- python -c \
  "import urllib.request; print(urllib.request.urlopen('http://ollama:11434/api/tags', timeout=5).read()[:120])"
```

If it fails (B4)

1. `command-center` waits with `waiting for healthy state of …/phoenix-data` → the data layer isn't Healthy (B2), or the Application health check from A1.1 is missing.
2. Pods `CrashLoopBackOff` with `Phoenix will not start` or a pydantic error → a missing env var (`PHOENIX_DATABASE_URL`, or `phoenix-secrets` not decrypted). Appendix T5.
3. Browser `400 Unknown host name` → `PHOENIX_ALLOWED_HOSTS` must be exactly `phoenix.halo.localhost`.

- ☐ **The Command Center runs in "phoenix" through GitOps** (W3 checkpoint) · ☐ demo data visible · ☐ logs JSON, metrics on :9100

CKA corner: probes (startup/liveness/readiness on different paths), Jobs as hooks, Ingress, Services. `kubectl describe pod <api>` shows all three probes, and `kubectl get endpointslices -n phoenix` shows readiness at work.

Interview angle: "The first real tenant runs through the same chart and pipeline as the demo: migrations gate the rollout, one-shot data seeding is declarative, and broker credentials are absent by design."

Phoenix angle: the Command Center is now reachable from any Windows browser on `phoenix.halo.localhost`, survives restarts, and redeploys on every merged commit.

Drill W3-4 (15 min): static pods (k3s and kubeadm).

1. Find k3s's static pod directory and confirm it from the k3s process or logs.
2. Write `static-web.yaml` (`nginx:1.29`, namespace `default`) into it with `--dry-run=client -o yaml`; find the mirror pod's name.
3. `kubectl delete` the mirror pod and watch what happens.
4. Remove the manifest file and confirm the pod is gone.
5. On a **kubeadm** node, where is the static pod path configured, and what's the default directory?

B5: Backups, restore test, runbook (Wed Nov 18, block 1)

Goal: a backup now, a successful restore test into a scratch database, and the runbook committed with the result.

Why: a backup you haven't restored is a hope. Timescale needs `timescaledb_pre_restore()` and `timescaledb_post_restore()` around `pg_restore` (they pause and resume its background jobs). Some warnings during restore are normal for hypertables. The test restores into `phoenix_restore_test` and compares row counts table by table, so production data is never touched.

Where: [WSL]

#118 WSL Ubuntu shell (bash) - copy & paste

```
mkdir -p 'docs/runbooks'
cat > 'docs/runbooks/phoenix-backup-restore.md' <<'__EOF__'
# Runbook: Phoenix database backup and restore

| | |
|---|---|
| Backup | CronJob `phoenix/timescale-backup`, nightly 21:15 (zone in the manifest), `pg_dump -Fc` of database `phoenix` |
| Where | PVC `timescale-backups` (local-path-retain), mounted at `/backups` in the backup Job and in `timescale-0` |
| Retention | 14 days on the PVC. Weekly: copy the newest dump off the box (below) |
| RPO / RTO | 24 h / about 15 min for a restore |

## Check the last backup
```bash
kubectl -n phoenix get cronjob timescale-backup
kubectl -n phoenix get jobs --sort-by=.metadata.creationTimestamp | tail -3
kubectl -n phoenix exec timescale-0 -c timescale -- ls -lh /backups
```

Back up now
```bash
J=backup-manual-$(date +%Y%m%d%H%M)
kubectl -n phoenix create job --from=cronjob/timescale-backup "$J"
kubectl -n phoenix wait --for=condition=complete "job/$J" --timeout=300s
kubectl -n phoenix logs "job/$J"
```

Restore test (into a scratch database; production data untouched)
```bash
kubectl -n phoenix exec -i timescale-0 -c timescale -- sh -s <<'SH'
set -eu
export PGPASSWORD="$POSTGRES_PASSWORD" PGHOST=127.0.0.1 PGUSER=postgres
latest=$(ls -1t /backups/phoenix-*.dump | head -1); echo "restoring $latest"
psql -q -c "DROP DATABASE IF EXISTS phoenix_restore_test" -c "CREATE DATABASE phoenix_restore_test"
psql -q -d phoenix_restore_test -c "CREATE EXTENSION IF NOT EXISTS timescaledb" -c "SELECT timescaledb_pre_restore()"
pg_restore --no-owner --role=postgres -d phoenix_restore_test "$latest"
psql -q -d phoenix_restore_test -c "SELECT timescaledb_post_restore()"
for t in $(psql -Atd phoenix -c "select tablename from pg_tables where schemaname='public' order by 1"); do
```
```

```
  a=$(psql -Atd phoenix -c "select count(*) from public.\"$t\""); b=$(psql -Atd phoenix_restore_test -c "select count(*) from public.\"$t\"")
  [ "$a" = "$b" ] && echo "OK   $t $a" || echo "DIFF $t live=$a restored=$b"
done
psql -q -c "DROP DATABASE phoenix_restore_test"
SH
```
Expected: every table `OK` (a `DIFF` is only acceptable for rows written after the dump).

Real restore (database "phoenix" lost or corrupted)
1. Stop writers: set `replicaCount: 0` and the worker `replicas: 0` in `apps/command-center/overlays/halo/values.yaml`, push.
2. Run the restore block above with `phoenix` instead of `phoenix_restore_test` (DROP/CREATE with `OWNER phoenix`).
3. Revert the replica change (git revert), push; check `/readyz`.

Copy off the box (weekly)
```bash
f=$(kubectl -n phoenix exec timescale-0 -c timescale -- sh -c 'ls -1t /backups/phoenix-*.dump | head -1')
kubectl -n phoenix cp -c timescale "timescale-0:$f" "/mnt/d/lab-backups/$(basename "$f")"
```

Drill log
Date	Test	Result	Notes
YYYY-MM-DD	restore test		
__EOF__

Run the "Back up now" and "Restore test" blocks from the runbook (they're copy-paste ready):

#119 WSL Ubuntu shell (bash) - copy & paste

```
J=backup-manual-$(date +%Y%m%d%H%M)
kubectl -n phoenix create job --from=cronjob/timescale-backup "$J"
kubectl -n phoenix wait --for=condition=complete "job/$J" --timeout=300s && kubectl -n phoenix logs "job/$J"
```

Expected: `backup ok: /backups/phoenix-2026…Z.dump`, followed by the directory listing. The restore test then prints `restoring …` and one `OK <table> <count>` per Phoenix table.

Copy one dump to Windows (off the WSL disk), then record the drill:

#120 WSL Ubuntu shell (bash) - copy & paste

```
WINHOME=$(wslpath "$(/mnt/c/Windows/System32/cmd.exe /c 'echo %USERPROFILE%' 2>/dev/null | tr -d '\r')")
mkdir -p "$WINHOME/lab-backups"
f=$(kubectl -n phoenix exec timescale-0 -c timescale -- sh -c 'ls -1t /backups/phoenix-*.dump | head -1')
kubectl -n phoenix cp -c timescale "timescale-0:$f" "$WINHOME/lab-backups/$(basename "$f")" && ls -lh "$WINHOME/lab-backups"
# edit docs/runbooks/phoenix-backup-restore.md -> "Drill log": date, "restore test", "all OK", notes
git add -A && git commit -m "docs(runbook): Phoenix backup/restore + first drill result" && git pull --rebase && git push
```

If it fails

1. `pg_dump: error: connection … timed out` → the backup pod lacks the `lab.platform/access-timescale: "true"` label (NetworkPolicy). Check `backup.yaml`.
2. `pg_restore` errors on `timescaledb_information` objects or `circular foreign-key` → harmless on Timescale. What matters is the final `OK` table.
3. `kubectl cp` fails with `tar: not found` → the container lacks `tar`. Use `kubectl exec … -- cat "$f" > file` instead.

- ☐ Manual backup OK · ☐ restore test all `OK` · ☐ off-box copy · ☐ runbook drill log filled

CKA corner: CronJobs: `schedule`, `timeZone`, `concurrencyPolicy`, `startingDeadlineSeconds`, history limits, and `kubectl create job --from=cronjob/…`. All exam material.

Interview angle: "Backups are a CronJob with a restore test that diffs every table, documented RPO/RTO, and an off-box copy, because RWO local storage has no replication."

Phoenix angle: the journal has a 24 h RPO in the lab, which is enough for synthetic data and the pattern you'll reuse when real data is allowed.

Drill W3-5 (15 min, `kcka`): Jobs and CronJobs.

1. Job `pow` (`busybox:1.37.0`, `sh -c 'echo $((2**10))'`) with `completions: 3`, `parallelism: 2`; show the 3 pods.
2. Job `fail` that always exits 1, with `backoffLimit: 2`; read the failure reason.
3. CronJob `tick` every minute printing the date; after one run, **suspend** it.
4. Trigger `tick` once by hand with `create job --from`.
5. Add `activeDeadlineSeconds: 20` and `ttlSecondsAfterFinished: 60` to a Job that sleeps 60 s; explain both outcomes.

B6: Resilience tests and retiring the old repo (Wed Nov 18, block 2)

Setup: helpers and a marker in each store (kept outside the app's own tables):

#121 WSL Ubuntu shell (bash) - copy & paste

```
pgq(){ kubectl -n phoenix exec -i timescale-0 -c timescale -- sh -c 'PGPASSWORD=$POSTGRES_PASSWORD psql -h 127.0.0.1 -U postgres -d postgres -At'; }
rq(){ kubectl -n phoenix exec -i redis-0 -- redis-cli --no-auth-warning "$@"; }
pgq <<'SQL'
CREATE TABLE IF NOT EXISTS drill_marker(id serial PRIMARY KEY, note text, at timestamptz DEFAULT now());
INSERT INTO drill_marker(note) VALUES ('before-b6');
SELECT count(*) FROM drill_marker;
SQL
rq SET drill:marker before-b6
API='app.kubernetes.io/name=command-center,app.kubernetes.io/component=api'
restarts(){ kubectl -n phoenix get pod -l "$API" -o jsonpath='{.items[0].status.containerStatuses[0].restartCount}'; echo; }
```

B6a: Delete the app pod

#122 WSL Ubuntu shell (bash) - copy & paste

```
kubectl -n phoenix delete pod -l "$API"
kubectl -n phoenix rollout status deploy/command-center --timeout=120s
curl -s -o /dev/null -w '%{http_code}\n' -H 'Host: phoenix.halo.localhost' http://127.0.0.1/readyz
```

Expected: a new pod is Ready in < 60 s and `200`. Argo CD stays *Synced* (pods are owned by the ReplicaSet, not by Git). `argocd app get command-center | grep Health` → Healthy.

B6b: Delete the Timescale pod, then the Redis pod: data survives

#123 WSL Ubuntu shell (bash) - copy & paste

```
restarts                                        # note the number
kubectl -n phoenix delete pod timescale-0
kubectl -n phoenix get pods -l "$API" -w        # READY drops to 0/1 while the DB is down (readiness), Ctrl+C when 1/1
kubectl -n phoenix wait --for=condition=Ready pod/timescale-0 --timeout=180s
echo "SELECT note FROM drill_marker ORDER BY id;" | pgq
restarts                                        # SAME number: liveness (/livez) never touched the database
kubectl -n phoenix delete pod redis-0
kubectl -n phoenix wait --for=condition=Ready pod/redis-0 --timeout=120s
rq GET drill:marker
```

Expected: `before-b6` from both stores. The API went *unready* and came back with **zero restarts**. This is the liveness/readiness split working.

B6c: `wsl --shutdown`, then start again

[WIN]

#124 Windows PowerShell - copy & paste

```
wsl --shutdown; Start-Sleep 10; Start-ScheduledTask -TaskName 'WSL-halo-keepalive'; Start-Sleep 120
kubectl -n phoenix get pods
curl.exe -s -o NUL -w "%{http_code}`n" http://phoenix.halo.localhost/livez
```

[WSL] (new shell; re-run the Setup block's two helper lines) `echo "SELECT count(*) FROM drill_marker;" | pgq` and `rq GET drill:marker`.
Expected: all pods Running and Ready within 2–3 min, `200`, and both markers intact. The PVs live on the WSL ext4 disk, so a VM restart doesn't touch them.

B6d: Push a bad image tag, roll back with `git revert`

#125 WSL Ubuntu shell (bash) - copy & paste

```
F=platform/kubernetes/apps/command-center/overlays/halo/values.yaml
yq -i '.image.tag = "0000000000000000000000000000000000000000"' $F
git commit -am "test(command-center): deploy a tag that does not exist" && git pull --rebase && git push
argocd app get command-center --refresh >/dev/null; sleep 60
kubectl -n phoenix get pods                                  # command-center-migrate-…  ErrImagePull / ImagePullBackOff
argocd app get command-center | grep -E 'Sync Status|Health Status|Phase'
curl -s -o /dev/null -w '%{http_code}\n' -H 'Host: phoenix.halo.localhost' http://127.0.0.1/readyz   # 200: old pods serve
git revert --no-edit HEAD && git pull --rebase && git push
argocd app terminate-op command-center                       # stop the sync that is waiting on the broken hook
argocd app get command-center --refresh >/dev/null
argocd app wait command-center --sync --health --timeout 300
```

Expected: while broken, the sync is *Running* on the migration hook and the Deployments are **untouched**: the wave-1 migration gates wave 2, so a bad build never replaces a good one. After the revert and `terminate-op`, a fresh sync succeeds. Compare Week 1's podinfo, where the bad image did roll (and was held only by `maxUnavailable: 0`).

B6e: Everything Synced and Healthy

#126 WSL Ubuntu shell (bash) - copy & paste

```
argocd app list
kubectl -n phoenix get pods
```

Expected: `root-halo`, `argocd-projects`, `sealed-secrets`, `storage-classes`, `podinfo`, `phoenix-data` and `command-center` all `Synced` and `Healthy`. Phoenix pods all Running with zero unexpected restarts.

B6f: Retire the old repo

[WSL]

#127 WSL Ubuntu shell (bash) - copy & paste

```
cd /tmp && rm -rf phx-retire && git clone -q git@github.com:nikhilgs817/PHOENIX_TRADER.git phx-retire && cd phx-retire
printf '%s\n\n' '> **Moved:** this code now lives in `ai-platform-portfolio/projects/command-center` (history preserved).' \
  | cat - README.md > README.new && mv README.new README.md
git commit -am "docs: moved to ai-platform-portfolio/projects/command-center" && git push && cd ~ && rm -rf /tmp/phx-retire /tmp/phoenix-import
```

[GITHUB] PHOENIX_TRADER → *Settings → General → Danger Zone → Archive this repository*. Your Windows-local Phoenix (real journal, SQLite) keeps running from wherever it runs today, unchanged. The cluster copy is the synthetic-data tenant.

- ☐ a · ☐ b · ☐ c · ☐ d · ☐ e · ☐ old repo archived

CKA corner: this block is Troubleshooting (30%): `kubectl get events --sort-by=.lastTimestamp`, `kubectl describe pod`, readiness vs liveness, StatefulSet pod identity and PVC re-attachment.

Interview angle: "I ran a resilience script against the first tenant: pod kills, database and cache restarts, a VM restart and a bad release, each with an expected result, all recovering without manual fixes except a `git revert`."

Phoenix angle: "survives any pod being killed", the L1 promise for Phoenix, is now demonstrated, not assumed.

Drill W3-6 (15 min, timed mock, mixed).

1. In `cka`, create a NetworkPolicy `deny-all` that blocks all ingress to every pod in the namespace.
2. Create PVC `scratch` (`local-path`, 1Gi) and a pod `writer` that mounts it at `/data`; explain why the PVC was Pending until the pod existed.
3. Add a readiness probe (`httpGet /` port 80) to deployment `api` with `kubectl patch` or `kubectl edit`.
4. Roll `api` back to its previous revision and print the image now running.
5. Print the names of all pods in all namespaces that are **not** `Running` (field selector).

Part C: CKA drills, woven in

CKA: Thu Dec 3. These two weeks cover the Workloads, Services/Networking and Storage domains on your own cluster. Troubleshooting comes from every "If it fails".

C1: The rules

- **Namespace `cka` only** (created in A0, context `cka`, alias `kcka`). Argo CD manages every namespace listed in an Application and reverts drift within seconds (`selfHeal`). Practising in `podinfo` or `phoenix` would mean Argo CD deleting your objects, or "fixing" your scaled Deployment, mid-drill. `cka` is in no Application and in no AppProject destination, so Argo CD never sees it. No PSA label either, so drills that need looser pods work.
- **Timed:** 15 minutes, 5 tasks, at the end of every block. Answers are in the appendix. Look only after the timer.
- **Exam habits:** `alias k=kubectl` with completion (Week 1), `export do="--dry-run=client -o yaml"`, `export now="--force --grace-period 0"`; `kubectl explain <res>.<field>` instead of guessing; generate YAML with `$do` and edit, rather than writing from scratch.
- **Reset after each drill:** `kcka delete all --all $now; kcka delete cm,secret,quota,limitrange,netpol,pvc --all`. Then check the node has no drill taint: `kubectl describe node halo | grep Taints`.
- **Single-node limits:** anti-affinity, topology spread and real `drain` scenarios need ≥ 2 nodes. Practise those on the kubeadm cluster (Nov 19–22).

C2: Where lab steps teach exam topics

Lab step	CKA topic	Domain
A2 / B4 probes (`/healthz`, `/readyz`, `/livez`, startup)	liveness vs readiness vs startup	Workloads, Troubleshooting
A3 chart: PDB, securityContext, ServiceAccount	PDBs, Pod Security, service accounts	Workloads, Cluster Architecture
A6 / B2 ResourceQuota + LimitRange	quotas, default requests	Workloads
B2 StatefulSets + PVCs on `local-path-retain`	StorageClass, PV/PVC binding, reclaim policy	Storage
B3 NetworkPolicies + probe matrix	NetworkPolicy	Services & Networking
B4 Ingress through Traefik	Ingress, Services, EndpointSlices	Services & Networking
B5 CronJob + manual Job	Jobs, CronJobs	Workloads
B6 resilience	events, describe, logs, rollout	Troubleshooting

Lab step	CKA topic	Domain
W3-4 drill	static pods (k3s `/var/lib/rancher/k3s/agent/pod-manifests`, kubeadm `/etc/kubernetes/manifests`)	Cluster Architecture

C3: Drill log (fill in; it's on the final checklist)

Drill	Date	Time used (min)	Tasks right (of 5)	Missed / slow on
W2-1 pods				
W2-2 deployments, labels				
W2-3 rollouts, scaling				
W2-4 explain, events, logs, exec				
W2-5 dry-run YAML				
W2-6 jsonpath, custom-columns				
W3-1 ConfigMaps, Secrets				
W3-2 requests, limits				
W3-3 scheduling				
W3-4 static pods				
W3-5 Jobs, CronJobs				
W3-6 mixed mock				

Save a copy as `docs/demos/cka-drill-log.md` and commit it as you go.

C4: Print me: "10 minutes of kubectl a day" (7 days)

Day	10 minutes, no notes, in `cka`	Done
1	`k run` 5 pods with different images and labels; `k get po -L app,tier`; delete all with one selector	☐
2	`k create deploy` → `k set image` → `k rollout history/undo/status`; three times, faster each time	☐
3	Generate with `$do`: pod, deploy, svc (`expose`), job, cronjob, configmap, secret; apply all with one `-f`	☐
4	jsonpath: pod IPs, node IPs, images, restart counts; one custom-columns view; `--sort-by` twice	☐
5	ConfigMap + Secret as env **and** volume; `k exec` to prove both; decode a secret	☐
6	NetworkPolicy: default-deny, then allow one labelled pod; test with `wget -T 2` from two pods	☐

Day	10 minutes, no notes, in `cka`	Done
7	Static pod in/out of the manifest dir; CronJob suspend/resume; `create job --from`; `k explain` three fields	☐

Appendix: troubleshooting, glossary, drill answers, checklist, what's next

Troubleshooting

T1: `ImagePullBackOff` from GHCR

`kubectl -n <ns> describe pod <pod>` → *Events*:

Message	Cause	Fix
`401 Unauthorized` / `denied`	PAT expired or revoked, lacks `read:packages`, or the `ghcr-pull` Secret is missing	`kubectl -n <ns> get secret ghcr-pull`; new PAT → `seal-ghcr-pull.sh` → push
`manifest unknown`	Tag doesn't exist (B6d on purpose, or a promote of a failed build)	`git revert` the promotion; check the `image` job pushed
`no key could decrypt secret` (on the SealedSecret)	Sealed for another controller key	Restore the key (`bootstrap-secrets.md`) or re-seal
Pull works for one service, not another	Package not linked to the repo, or created under another owner	Package settings → *Manage Actions access* → add `ai-platform-portfolio`

T2: CI didn't trigger (or triggered for the wrong service)

1. The push didn't touch the caller's `paths:`. `git show --stat HEAD` vs `.github/workflows/svc-<svc>.yml`.
2. The caller file only exists on a branch. `push` triggers come from the workflow file **on the pushed commit**.
3. A commit message containing `[skip ci]` (copied from a promotion commit) skips everything.
4. Everything ran → you changed `_service-ci.yml` or `.github/actions/**`, which are deliberately in every caller's filter.

T3: CI commit loops or promotion races

- **A loop needs all three guards to fail:** a promotion pushed with a PAT or GitHub App token (which *does* trigger workflows), callers whose `paths:` include `platform/kubernetes/**`, and no `[skip ci]`. Keep `GITHUB_TOKEN` in `promote-tag`.
- `rejected … fetch first` five times → several promotions at once. The `promote-halo` concurrency group serialises within one workflow, and the retry loop handles the rest. Re-run the job.
- `newer build … already deployed; not going backwards` → correct behaviour: an older run finished last.

T4: PVC stuck `Pending`

1. `local-path(-retain)` is `WaitForFirstConsumer`: *Pending until a pod mounts it* is normal. Argo CD reports it as Progressing and **blocks later sync waves**. Mount it in a long-running pod (as `timescale-0` does for backups), or put it in the same wave as its consumer.
2. `kubectl describe pvc` → `waiting for first consumer` (normal) vs provisioner errors: `kubectl -n kube-system logs deploy/local-path-provisioner`.
3. Quota: `exceeded quota: tenant-budget, requested: requests.storage` → raise it in `guardrails.yaml` via Git.
4. Re-using a `Retain` volume after a PVC delete: the PV is `Released`. Clear `spec.claimRef` (`kubectl patch pv <pv> -p '{"spec":{"claimRef":null}}'`) so a new PVC can bind it.

T5: `CrashLoopBackOff` from a missing env var

`kubectl -n phoenix logs <pod> --previous`:

- pydantic `validation error … Field required` or `Phoenix will not start` → a Secret key or env var is missing. `kubectl -n phoenix get secret phoenix-db-app -o jsonpath='{.data}' | jq 'keys'`.
- `CreateContainerConfigError` (not a crash) → the referenced Secret or key doesn't exist yet. Is the SealedSecret decrypted? `kubectl -n phoenix get sealedsecret` → `SYNCED True`.
- Template service: every setting is `APP_*`; `kubectl -n <ns> exec deploy/<svc> -- env | grep APP_`.

T6: NetworkPolicy blocking too much

1. DNS first: every namespace with default-deny egress needs `allow-dns-egress`. `kubectl -n phoenix exec deploy/command-center -- python -c "import socket;print(socket.gethostbyname('timescale'))"`.
2. Labels: the client pod needs `lab.platform/access-timescale: "true"` (a **string**, quoted). `kubectl -n phoenix get pods --show-labels`.
3. Both directions: egress on the client **and** ingress on the server must allow it. `np-probe.sh` tests exactly that.
4. Ingress from Traefik: allowed from `10.255.255.10/32` and `10.42.0.1/32`. If you change `cluster-cidr`, update both NetworkPolicies and `_shared/halo/values.yaml`.

T7: Argo CD sync-wave or ordering problems

1. Child app waits forever: `argocd app get root-halo` → `waiting for healthy state of argoproj.io/Application/<child>`. Fix the child (`argocd app get <child>`). Without A1.1's Lua check, waves don't wait at all.
2. Migration hook fails on first install with `serviceaccount … not found` → the hook was `PreSync`. The chart uses a `Sync` hook in wave 1 for exactly this reason.
3. Sync stuck *Running* on a hook → `argocd app terminate-op <app>`, fix in Git, refresh.
4. `one or more objects failed to apply … field is immutable` on a Job → a one-shot Job's spec changed. Rename it (`demo-seed-2`) instead.
5. Two apps fighting over namespace labels → only one Application (here `phoenix-data`) may use `CreateNamespace` + `managedNamespaceMetadata` for a namespace.

T8: Copier and generator issues

- `Template is not a git repository` warnings are harmless (local path). `copier update` later needs the template in its own Git repo or tagged (What's next).
- Generated workflow contains literal `[[ service_name ]]` → the file didn't end in `.jinja` in the template.
- `new-service.sh` says the name is invalid → lowercase letters, digits and dashes, 3–30 characters, starting with a letter.

T9: Phoenix-specific

- `400 Unknown host name` → `PHOENIX_ALLOWED_HOSTS` must contain the exact host you browse to.
- Seed Job fails on duplicate rows → it ran twice. Reset: scale the app to 0 via Git, `DROP DATABASE phoenix; CREATE DATABASE phoenix OWNER phoenix;` (superuser), revert the scale (migrations re-run), then rename the seed Job.
- The chat says the model is unavailable → expected unless B4.4 is done; the chat still answers fact checks from data.

T10: Sealed Secrets after a rebuild

`kubectl -n kube-system logs deploy/sealed-secrets-controller | grep -i 'error\|key'`. If a new key was generated, follow `bootstrap-secrets.md` → *Key lost*, then **back up the new key**.

Glossary

Term	Plain words
Paved road	The supported, automated way to build and run a service; leaving it is allowed but costs more
Copier	Template engine that creates a project and records the answers, so the project can be updated when the template changes
Reusable workflow	A GitHub Actions workflow other workflows call with `uses:` (like a Jenkins shared-library pipeline step, versioned in the repo)
Composite action	A reusable *step* bundle (`action.yml`, `using: composite`)
Path filter	`on.push.paths`: run a workflow only when matching files change (the monorepo's main tool)
GHCR	GitHub Container Registry, `ghcr.io/<owner>/<image>`
Promotion	Moving a built artifact into an environment; here, a Git commit that sets `image.tag` in an overlay
SealedSecret	A Secret encrypted with the controller's public key; safe in Git, decrypted only in the cluster
StatefulSet	Controller for pods with stable names and their own volumes (databases)
Headless Service	`clusterIP: None`: DNS returns pod IPs directly
Retain (reclaim policy)	Deleting the PVC keeps the PV and its data
NetworkPolicy	Label-selected firewall rules for pod traffic; default-deny + allow-lists
Sync hook	A resource Argo CD runs at a sync phase (here a Job in the Sync phase, wave 1)

Term	Plain words
Liveness / readiness / startup probe	Restart if stuck / route traffic only if ready / give slow starters time before liveness applies
RPO / RTO	How much data you can lose / how long recovery takes

Drill answers

All in namespace `cka` with `alias k=kubectl` and `export do="--dry-run=client -o yaml"`. Set the namespace once: `k config use-context cka`.

W2-1

1. `k run web --image=nginx:1.29 -l tier=frontend`
2. `k get pod web -o jsonpath='{.status.podIP}{" "}{.spec.nodeName}{"\n"}'`
3. `k run busy --image=busybox:1.37.0 $do -- sleep 3600 > /tmp/busy.yaml && k apply -f /tmp/busy.yaml`
4. `k exec busy -- wget -qO- "$(k get pod web -o jsonpath='{.status.podIP}')" | head -4`
5. `k delete pod web busy --force --grace-period 0`

W2-2

1. `k create deploy api --image=nginx:1.29 --replicas=3`
2. `k get pods -l app=api -L app,pod-template-hash`
3. `k patch deploy api -p '{"spec":{"template":{"metadata":{"labels":{"env":"lab"}}}}}' && k rollout status deploy/api`
4. `k get pods -l app=api,env=lab --no-headers | wc -l`
5. `k get pods -l 'tier in (frontend,backend)'`

W2-3

1. `k set image deploy/api nginx=nginx:1.28 && k annotate deploy/api kubernetes.io/change-cause="nginx 1.28" --overwrite`
2. `k rollout history deploy/api && k rollout history deploy/api --revision=2`
3. `k set image deploy/api nginx=nginx:9.9.9; k rollout status deploy/api --timeout=30s; k get pods -l app=api` (`ErrImagePull`), `k describe pod <new> | tail -5`
4. `k rollout undo deploy/api && k get deploy api -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'`
5. `k scale deploy/api --replicas=5 && k scale deploy/api --replicas=2`

W2-4

1. `k explain deployment.spec.strategy.rollingUpdate.maxSurge` → default **25%**
2. `k get events --field-selector type=Warning --sort-by=.lastTimestamp`
3. `k run crash --image=busybox:1.37.0 -- sh -c 'echo boom; exit 1'; sleep 20; k logs crash --previous`
4. `k exec web -- printenv HOSTNAME`
5. `k run huge --image=nginx:1.29 --overrides='{"spec":{"containers":[{"name":"huge","image":"nginx:1.29","resources":{"requests":{"cpu":"500"}}}]}}'; k describe pod huge | grep -A3 Events` → `0/1 nodes are available: 1 Insufficient cpu`

W2-5

1. `k create deploy shop --image=nginx:1.29 --replicas=2 --port=80 $do > /tmp/shop.yaml`
2. `k set resources -f /tmp/shop.yaml --local --requests=cpu=50m,memory=32Mi -o yaml > /tmp/shop2.yaml && mv /tmp/shop2.yaml /tmp/shop.yaml`
3. `echo --- >> /tmp/shop.yaml && k create svc clusterip shop --tcp=80:80 $do >> /tmp/shop.yaml`
4. `k create job once --image=busybox:1.37.0 $do -- echo done > /tmp/once.yaml`
5. `k apply -f /tmp/shop.yaml -f /tmp/once.yaml && k get deploy,svc,job`

W2-6

1. `k get pods -A -o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}'`
2. `k -n kube-system get pods -o custom-columns=NAME:.metadata.name,IMAGE:.spec.containers[0].image,NODE:.spec.nodeName`
3. `k get node halo -o jsonpath='{.status.addresses[?(@.type=="InternalIP")].address}{"\n"}'`
4. `k get pods -A --sort-by='.status.containerStatuses[0].restartCount'`
5. `k get pods -A -o jsonpath='{.items[*].spec.containers[*].image}' | tr ' ' '\n' | sort -u`

W3-1

1. `k create cm app-cfg --from-literal=MODE=lab --from-literal=LEVEL=debug`
2. `k run cfg-env --image=busybox:1.37.0 $do -- sleep 3600 > p.yaml`; under the container add `envFrom: [{configMapRef: {name: app-cfg}}]`; `k apply -f p.yaml && k exec cfg-env -- printenv MODE`
3. `k create secret generic db-cred --from-literal=user=lab --from-literal=pass=s3cr3t`; pod with `volumes: [{name: db, secret: {secretName: db-cred}}]` and `volumeMounts: [{name: db, mountPath: /etc/db, readOnly: true}]`; `k exec sec-vol -- cat /etc/db/pass`
4. In `sec-env`: `env: [{name: DB_USER, valueFrom: {secretKeyRef: {name: db-cred, key: user}}}]`
5. `k get secret db-cred -o jsonpath='{.data.pass}' | base64 -d; echo`

W3-2

1. `k run sized --image=nginx:1.29 $do > s.yaml`; add `resources: {requests: {cpu: 100m, memory: 64Mi}, limits: {memory: 128Mi}}`; `k apply -f s.yaml`
2. LimitRange `spec.limits: [{type: Container, defaultRequest: {cpu: 50m}, default: {memory: 256Mi}}]`; `k run bare --image=nginx:1.29 && k get pod bare -o jsonpath='{.spec.containers[0].resources}'`
3. `k create quota pods-max --hard=pods=3`; run pods until `Error from server (Forbidden): … exceeded quota: pods-max`
4. Pod with limit `32Mi`, command `sh -c 'head -c 100m /dev/zero | tail'`; `k get pod oom -o jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'` → `OOMKilled`
5. `k top pods --sort-by=memory`; then `k delete quota pods-max; k delete limitrange --all`

W3-3

1. `k label node halo disk=ssd`; pod with `nodeSelector: {disk: ssd}` → Running
2. `nodeSelector: {disk: hdd}` → `k describe pod slow` → `0/1 nodes are available: 1 node(s) didn't match Pod's node affinity/selector`
3. `k taint node halo drill=yes:NoSchedule`; toleration `tolerations: [{key: drill, operator: Equal, value: "yes", effect: NoSchedule}]` → `tol` Running, `notol` Pending; **`k taint node halo drill=yes:NoSchedule-`**

4. `affinity: {nodeAffinity: {requiredDuringSchedulingIgnoredDuringExecution: {nodeSelectorTerms: [{matchExpressions: [{key: disk, operator: In, values: [ssd]}]}]}}}`
5. `podAntiAffinity.requiredDuringSchedulingIgnoredDuringExecution` on `app=spread` with `topologyKey: kubernetes.io/hostname` → the second replica has no other node. Cleanup: `k label node halo disk-`

W3-4

1. `sudo ls /var/lib/rancher/k3s/agent/pod-manifests` and `sudo journalctl -u k3s | grep -m1 pod-manifest` (k3s runs the kubelet with `--pod-manifest-path=/var/lib/rancher/k3s/agent/pod-manifests`)
2. `k run static-web --image=nginx:1.29 -n default $do | sudo tee /var/lib/rancher/k3s/agent/pod-manifests/static-web.yaml`; mirror pod `static-web-halo` in `default`
3. `k -n default delete pod static-web-halo` → recreated immediately (the kubelet owns it, not the API)
4. `sudo rm /var/lib/rancher/k3s/agent/pod-manifests/static-web.yaml` → the pod disappears
5. kubeadm: `staticPodPath` in `/var/lib/kubelet/config.yaml`, default `/etc/kubernetes/manifests`

W3-5

1. `k create job pow --image=busybox:1.37.0 $do -- sh -c 'echo $((2**10))' > j.yaml`; add `completions: 3, parallelism: 2`; `k apply -f j.yaml && k get pods -l job-name=pow`
2. Job `fail` with `backoffLimit: 2` and `exit 1` → `k describe job fail` → `BackoffLimitExceeded`
3. `k create cronjob tick --image=busybox:1.37.0 --schedule='* * * * *' -- date`; after one run `k patch cronjob tick -p '{"spec":{"suspend":true}}'`
4. `k create job tick-manual --from=cronjob/tick`
5. `activeDeadlineSeconds: 20` → the Job fails with `DeadlineExceeded` after 20 s; `ttlSecondsAfterFinished: 60` → the finished Job and its pods are deleted 60 s later

W3-6

1. `{apiVersion: networking.k8s.io/v1, kind: NetworkPolicy, metadata: {name: deny-all}, spec: {podSelector: {}, policyTypes: [Ingress]}}`
2. PVC `storageClassName: local-path`, 1Gi; pod mounting it at `/data` → the PVC binds when the pod is scheduled (`WaitForFirstConsumer`)
3. `k patch deploy api --type=json -p '[{"op":"add","path":"/spec/template/spec/containers/0/readinessProbe","value":{"httpGet":{"path":"/","port":80}}}]'`
4. `k rollout undo deploy/api && k get deploy api -o jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'`
5. `k get pods -A --field-selector=status.phase!=Running`

Final checklist

- ☐ **The template creates a new service that CI builds and Argo CD deploys** (A6: timed < 15 min, removed cleanly)
- ☐ **The Command Center runs in "phoenix" through GitOps** (B4: chart, CI, promotion, Argo CD)
- ☐ **Timescale and Redis data survive pod kills and a WSL restart** (B6b, B6c markers)

- ☐ **No plaintext secrets in Git** (`repo-guards` green; `git grep -nE '^kind: Secret$' platform/kubernetes` is empty; every value in the password manager; sealing key backed up)
- ☐ **The CKA drill log is filled in** (C3, 12 rows)
- ☐ Restore test passed and recorded (B5) · ☐ NetworkPolicy probe matrix matches (B3) · ☐ old Phoenix repo archived (B6f)

What's next (out of scope here)

- **Week 4, supply chain:** Harbor replaces GHCR (proxy cache + tag immutability), Trivy scans in CI, Syft SBOMs, cosign signatures, and a **Kyverno** policy that blocks unsigned images in tenant namespaces. The chart's security defaults already pass the baseline/restricted policies.
- **Argo Rollouts** canary for template services (the chart's Deployment becomes a Rollout behind a flag).
- **Observability (Week 7):** Prometheus scrapes the `prometheus.io/*` annotations and Phoenix's `:9100`; Loki ingests the JSON logs; Tempo/OTel traces; OpenCost per namespace.
- **L2:** the Phoenix `agent` (MCP + models) and LiteLLM in front of llama.cpp/Ollama on the Radeon, using the Redis provisioned here. The internet-facing `hook` arrives with the tunnel, inside the default-deny namespace.
- **Terraform** for AWS GPU nodes; the `aws` overlays (CloudNativePG or RDS there; gp3 StorageClass).
- **Paved road v2:** publish the chart as an OCI artifact and pin a chart version per service; move the template into its own repo so `copier update` can roll fixes out; revisit Argo CD Image Updater for multi-environment promotion; add mypy to Phoenix.
- **January publication checklist:** gitleaks over the full history (already in CI), review the Phoenix `policy.yaml` lessons and `docs/*.pdf`, make sure the repo holds no account data (`.gitignore` already blocks `*.csv`, `*.db`, `exports/`), then flip the repo to public.